

# The AFS-TrEDM Hash Function Family

An Aligned TrEDM Sponge Using the AFS- $p$ -S6[1600,  $n_r$ ] Public Permutation

## Algorithm Specification

*Algorithm Specification (ICCS Submission · S6 Linear-Layer Version)*

Version 2.0 / S6 mainline

Date: 2026-06-29

Algorithm instances: AFS-TrEDM-512, AFS-TrEDM-768, AFS-TrEDM-1024

|                                      |                                                                             |
|--------------------------------------|-----------------------------------------------------------------------------|
| <b>Family name</b>                   | The AFS-TrEDM hash function family (S6 linear-layer version)                |
| <b>Underlying public permutation</b> | AFS- $p$ -S6[1600, $n_r$ ] (a 1600-bit ARX-Feistel-type permutation)        |
| <b>Sponge mode</b>                   | Aligned Truncated Encrypted Davies–Meyer (aligned TrEDM) sponge             |
| <b>Supported digest lengths</b>      | 512 and 1024 bits (mandatory); 768 bits (optional)                          |
| <b>Tiered round counts</b>           | 512 uses 6+6=12 rounds, 768 uses 10+10=20 rounds, 1024 uses 12+12=24 rounds |
| <b>Nonlinear layer</b>               | AFS-64 ARX large S-box (AFS64_t5_k2)                                        |
| <b>Linear diffusion layer</b>        | AFS-LMDS-1600-S6, a six-direction rotating single-MDS layer                 |

*This document is the algorithm specification of the AFS-TrEDM hash function family (S6 linear-layer version). It covers five parts: algorithm description, design rationale, security claims and analysis, performance evaluation, and feature statement.*

# Contents

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| 1 Algorithm Description .....                                                             | 5  |
| 1.1 Overview .....                                                                        | 5  |
| 1.2 Glossary .....                                                                        | 5  |
| 1.2.1 Terms and Abbreviations .....                                                       | 5  |
| 1.2.2 Algorithm Parameters and Other Variables .....                                      | 6  |
| 1.2.3 Basic Operations and Functions .....                                                | 7  |
| 1.3 Algorithm Instances and Parameters .....                                              | 8  |
| 1.4 Auxiliary Functions: Message Parsing, Padding, and Frame Encoding .....               | 9  |
| 1.5 Nonlinear Layer: the AFS-64 S-box .....                                               | 10 |
| 1.6 Linear Diffusion Layer: AFS-LMDS-1600-S6 .....                                        | 12 |
| 1.6.1 The GF(32) Field and Constant Multiplication .....                                  | 13 |
| 1.6.2 The Cauchy MDS Matrix $C$ and Column Mixing $M_{\text{col}}$ .....                  | 13 |
| 1.6.3 The AFS-Specific Lane Circular Shift $\rho_{\text{AFS}}$ .....                      | 13 |
| 1.6.4 The Long-Period Lane Permutation $\pi_{\text{AFS}}$ .....                           | 14 |
| 1.6.5 The Per-Lane Invertible Ring Diffusion $\mu$ .....                                  | 14 |
| 1.6.6 The Single-MDS Building Block $L_p$ .....                                           | 14 |
| 1.6.7 Six-Direction Rotation $S_6$ and the Per-Round Linear Layer $L$ .....               | 15 |
| 1.7 Round-Lane Constants $RC_{32}$ .....                                                  | 16 |
| 1.8 The Public Permutation AFS- $p$ -S6[1600, $n_r$ ] .....                               | 16 |
| 1.9 Initial Value (IV) .....                                                              | 17 |
| 1.10 Aligned-TrEDM Absorbing and Squeezing .....                                          | 18 |
| 1.11 The CryptHash Interface .....                                                        | 20 |
| 1.12 Test Vectors .....                                                                   | 20 |
| 2 Design Rationale .....                                                                  | 22 |
| 2.1 Overall Structure and Cryptographic Properties of the Components .....                | 22 |
| 2.1.1 The Aligned-TrEDM Sponge Mode .....                                                 | 22 |
| 2.1.2 The Six-Direction Rotating Single-MDS Linear Diffusion Layer AFS-LMDS-1600-S6 ..... | 25 |
| 2.1.3 The Nonlinear Layer AFS-64 S-box .....                                              | 26 |
| 2.1.4 Design Motivation for the Tiered Round Counts .....                                 | 26 |
| 2.2 Parameter Selection .....                                                             | 27 |
| 2.2.1 Digest Length $d \in \{512, 768, 1024\}$ .....                                      | 27 |
| 2.2.2 Choice of Capacity $c$ ( $c = a = d + 64$ ) .....                                   | 27 |
| 2.2.3 Tiered Round Counts and the $g/h$ Split .....                                       | 28 |
| 2.2.4 Choice of the AFS64_t5_k2 Instance .....                                            | 28 |
| 2.2.5 AFS-LMDS-1600-S6 Linear-Layer Parameters .....                                      | 28 |
| 2.3 Other Considerations .....                                                            | 29 |
| 2.3.1 Public, Deterministic Round-Lane Constants .....                                    | 29 |
| 2.3.2 Public, Deterministic Initial Values .....                                          | 30 |
| 2.3.3 Big-Endian Bit Order and Implementation-Friendly Frame Encoding .....               | 30 |
| 3 Security Claims and Analysis .....                                                      | 30 |
| 3.1 Security Claims .....                                                                 | 30 |
| 3.2 Theoretical Security .....                                                            | 31 |
| 3.2.1 Model and Basic Notation .....                                                      | 31 |
| 3.2.2 Classical Collision-Resistance Derivation .....                                     | 32 |
| 3.2.3 Classical Preimage-Resistance Derivation .....                                      | 32 |
| 3.2.4 Classical Second-Preimage-Resistance Derivation .....                               | 32 |
| 3.2.5 Five-Layer Structural Reduction Line .....                                          | 33 |
| 3.2.6 Relation-Level keyless-CI Reduction .....                                           | 34 |
| 3.2.7 The Alignment Lemma and Cutting Off the Misalignment Channel .....                  | 35 |
| 3.2.8 Frame Encoding, Capacity Alignment, and Scope of Applicability .....                | 36 |

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 3.3 Concrete Security .....                                                        | 36 |
| 3.3.1 Differential and Linear Analysis .....                                       | 37 |
| 3.3.2 Length-Extension-Attack Resistance .....                                     | 38 |
| 3.3.3 Free-Start, Semi-Free-Start, and Near Collisions .....                       | 39 |
| 3.3.4 Meet-in-the-Middle, Algebraic, and Integral Analysis .....                   | 39 |
| 3.3.5 Rotational Analysis .....                                                    | 39 |
| 3.3.6 Quantum Security .....                                                       | 40 |
| 3.3.7 Front-Half-Round and Aligned-Split Risk Verification .....                   | 42 |
| 3.4 Statistical-Randomness Test Results .....                                      | 42 |
| 4 Performance Evaluation .....                                                     | 43 |
| 4.1 Design-Level Performance Analysis .....                                        | 43 |
| 4.2 Test Platform .....                                                            | 44 |
| 4.3 Software-Implementation Computational Efficiency .....                         | 44 |
| 4.4 Same-Device Comparison with SHA3-512 .....                                     | 48 |
| 4.5 Reproducibility .....                                                          | 48 |
| 5 Statement of Features .....                                                      | 48 |
| 5.1 Innovativeness .....                                                           | 48 |
| 5.2 Simplicity .....                                                               | 49 |
| 5.3 Flexibility .....                                                              | 49 |
| 5.4 Implementation Performance Advantages on Different Platforms .....             | 49 |
| 6 Implementation Status and Code Structure .....                                   | 50 |
| 6.1 Reference Implementation (Reference_Implementation) .....                      | 50 |
| 6.2 Optimized Implementation (Optimized_Implementation) .....                      | 50 |
| 6.3 Additional Implementation (Additional_Implementation) .....                    | 50 |
| 6.4 Tools and Automated Build .....                                                | 50 |
| 7 References .....                                                                 | 51 |
| Appendix A Initial Values (IV) .....                                               | 53 |
| A.1 The IV of AFS-TrEDM-512 .....                                                  | 53 |
| A.2 The IV of AFS-TrEDM-768 .....                                                  | 53 |
| A.3 The IV of AFS-TrEDM-1024 .....                                                 | 53 |
| Appendix B Six-Phase Three-Round Window Verification .....                         | 53 |
| Appendix C AFS-TrEDM-S6 Randomness Test Report .....                               | 55 |
| C.1 Report Scope and Test Objects .....                                            | 55 |
| C.1.1 Test Objects .....                                                           | 55 |
| C.1.2 Report Scope .....                                                           | 55 |
| C.1.3 Evaluation Criteria and Decision Baseline .....                              | 55 |
| C.2 Introduction to the Test Software .....                                        | 56 |
| C.2.1 NIST SP 800-22 STS .....                                                     | 56 |
| C.2.2 TestU01 .....                                                                | 56 |
| C.2.3 Avalanche and Bit-Independence Test Program .....                            | 56 |
| C.3 Description of Test Items .....                                                | 56 |
| C.3.1 NIST SP 800-22 STS Test Items .....                                          | 56 |
| C.3.2 TestU01 SmallCrush Test Items .....                                          | 57 |
| C.3.3 The Avalanche Criterion (SAC) and the Bit-Independence Criterion (BIC) ..... | 57 |
| C.4 Test-Sample Generation Method .....                                            | 58 |
| C.4.1 Test Instances and Input Families .....                                      | 58 |
| C.4.2 NIST STS Samples .....                                                       | 58 |
| C.4.3 TestU01 Samples .....                                                        | 58 |
| C.4.4 Avalanche/Bit-Independence Samples .....                                     | 58 |
| C.4.5 Seed Pre-Registration and Anti-Cherry-Picking Strategy .....                 | 58 |
| C.5 Test Results .....                                                             | 58 |
| C.5.1 Summary of NIST SP 800-22 STS Results .....                                  | 58 |
| C.5.2 Summary of TestU01 SmallCrush Results .....                                  | 59 |

|                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------|----|
| C.5.3 Summary of Avalanche and Bit-Independence Results .....                                              | 60 |
| C.6 Conclusion .....                                                                                       | 61 |
| C.7 Security Boundary and Limitations .....                                                                | 61 |
| C.8 Reproducibility Information .....                                                                      | 62 |
| Appendix D Dedicated Verification of the 512 6+6 Front-Half-Round and Aligned-Split Risk .....             | 62 |
| D.1 Verification Scope and Evidence-Chain Overview .....                                                   | 62 |
| D.2 Model, Thresholds, and Fail-Closed Acceptance Rules .....                                              | 64 |
| D.3 Individual Verifications .....                                                                         | 65 |
| D.3.1 C1: 512 Arbitrary-Starting-State Split Verification .....                                            | 65 |
| D.3.2 C2/C3: Semi-Free-Start and keyless-CI Low-Activity Relation Verification .....                       | 66 |
| D.3.3 C4: Rotational-Structure Verification .....                                                          | 66 |
| D.3.4 C5/C6: Supplementary Evidence and Explicit Non-Claims .....                                          | 66 |
| D.4 Claim Ledger .....                                                                                     | 67 |
| D.5 Evidence Availability and Reproduction .....                                                           | 67 |
| Appendix E Relation-Level keyless-CI Reduction, Structure-Level Security Bounds, and Constant Tables ..... | 68 |
| E.1 Scope of This Appendix and Its Relation to Appendix D .....                                            | 68 |
| E.2 Notation, the Dual-Permutation TrEDM Core, and the Relation Families .....                             | 68 |
| E.3 Classical Main Theorems (with Explicit Coefficients) .....                                             | 69 |
| E.4 Lemma E-1 (Chain-Divergence Localization) and Its Complete Proof .....                                 | 69 |
| E.5 Lemma E-2 (Alignment Cutoff) and Its Complete Proof .....                                              | 70 |
| E.6 Proposition E-1 (Frame-Encoding Suffix-Free) and Its Complete Proof .....                              | 70 |
| E.7 Structure-Level Idealization Model, Scope, and Non-Claims .....                                        | 71 |
| E.8 Basic Notation and the Block Chain .....                                                               | 71 |
| E.9 Lemma E-3 (Single-Core Hit Probability) .....                                                          | 71 |
| E.10 Theorem E-4 (Classical Collision Resistance) .....                                                    | 72 |
| E.11 Theorem E-5 (Classical Preimage Resistance) .....                                                     | 72 |
| E.12 Theorem E-6 (Classical Second-Preimage Resistance) .....                                              | 72 |
| E.13 Parameter Derivation of the Generic Quantum-Attack Complexity .....                                   | 73 |
| E.14 Generic Quantum Lifting: Correct Point Counting for the Dual Oracle and a Conservative Proof .....    | 73 |
| E.15 The Compressed-Oracle/Quantum-Indistinguishability Path and Its Applicability Boundary .....          | 74 |
| E.16 Constant Tables, Layered Summary of Security Strength, and Scope of Applicability .....               | 75 |

# 1 Algorithm Description

This chapter describes the AFS-TrEDM hash function family (S6 version), including all instance parameters, notation conventions, auxiliary functions, the nonlinear layer AFS-64 S-box, the linear diffusion layer AFS-LMDS-1600-S6, the round constants, the underlying permutation AFS- $p$ -S6[1600,  $n_r$ ], the initial values, and the aligned TrEDM absorbing and squeezing process, as well as the external CryptHash() interface. It first specifies the definition and properties of each component, and then gives a reproducible implementation as pseudocode (algorithm boxes).

## 1.1 Overview

AFS-TrEDM is a fixed-output-length, permutation-based hash function family designed for the next-generation commercial-cryptography algorithm solicitation. It supports the two mandatory digest lengths of 512 and 1024 bits, and provides an optional 768-bit instance. Its construction combines the aligned Truncated Encrypted Davies–Meyer (aligned TrEDM) sponge mode with the 1600-bit Addition–Rotation–XOR (ARX)-Feistel-type public permutation AFS- $p$ -S6[1600,  $n_r$ ]. The aligned mode strictly locks the feed-forward window, the chain-value window, and the digest-output window onto the same segment of bits in the state — namely the capacity alignment window  $S$ .

The intra-round linear diffusion layer of AFS-TrEDM (S6) uses the lane-friendly AFS-LMDS-1600-S6 six-direction rotating single-Maximum-Distance-Separable (MDS) linear layer. The underlying permutation uses tiered round counts according to the digest length (512 uses 6+6, 768 uses 10+10, 1024 uses 12+12), so the underlying permutation family is denoted AFS- $p$ -S6[1600,  $n_r$ ] ( $n_r = 12 / 20 / 24$ ). The family inherits the aligned TrEDM sponge mode, the AFS-64 nonlinear layer, the padding and frame encoding, the MSB-first bit order, and the SplitMix64-based public round-constant schedule of §1.7. Each instance's IV-derivation domain string, reference implementation, optimized implementation, and all known-answer test vectors (KATs) are bound to the above parameters; modifying any parameter, IV, padding, bit order, linear layer, or round count invalidates the KATs.

## 1.2 Glossary

This chapter uniformly defines the terms, abbreviations, parameter variables, and basic operations and functions used throughout the document, and fixes the bit and byte conventions.

### 1.2.1 Terms and Acronyms

|                            |                                                                                                                                                             |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>bit</b>                 | A binary digit, taking the value 0 or 1.                                                                                                                    |
| <b>byte</b>                | A sequence of 8 bits.                                                                                                                                       |
| <b>bit string</b>          | A finite-length sequence of bits; the set of all such strings is denoted $\{0,1\}^*$ .                                                                      |
| <b>state</b>               | An array of bits repeatedly updated during the computation, of width $b=1600$ bits; it has two views, a “bit string” and a “ $5 \times 5 \times 64$ array”. |
| <b>state array</b>         | The $5 \times 5 \times 64$ three-dimensional view of the state, denoted $A$ .                                                                               |
| <b>lane</b>                | A 64-bit word $A[x,y]$ of the state array along the $z$ -axis.                                                                                              |
| <b>rate</b>                | The number of bits $r$ absorbed/squeezed per permutation call.                                                                                              |
| <b>capacity</b>            | The permutation width minus the rate, $c=b-r$ .                                                                                                             |
| <b>width</b>               | The fixed bit length $b$ of the input/output of the underlying permutation.                                                                                 |
| <b>sponge construction</b> | A construction defined by an underlying permutation, a padding rule, and a rate, with arbitrary-length input and output, comprising the two phases          |

|                                     |                                                                                                                                                                                                   |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     | of absorbing and squeezing.                                                                                                                                                                       |
| <b>absorbing / squeezing</b>        | The phases of injecting input into the state / extracting output from the state.                                                                                                                  |
| <b>digest</b>                       | The output of a hash function, also called the hash value, of fixed length.                                                                                                                       |
| <b>hash function</b>                | A bit-string function with fixed output length.                                                                                                                                                   |
| <b>message</b>                      | The input to a hash function.                                                                                                                                                                     |
| <b>feed-forward</b>                 | XORing the permutation input (or its projection) back into its output.                                                                                                                            |
| <b>Encrypted Davies–Meyer (EDM)</b> | A dual-permutation feed-forward compression core of the form $F(x)=h(g(x) \oplus \lfloor x \rfloor)$ .                                                                                            |
| <b>alignment / alignment window</b> | The design condition that locks the feed-forward, chain-value propagation, and squeezing onto the same segment of state bits (the capacity window $S$ , $ S =c=a$ ).                              |
| <b>indifferentiability</b>          | The property that the construction is indistinguishable from a random oracle to any distinguisher (the Maurer–Renner–Holenstein framework); a concept distinct from “indistinguishability” below. |
| <b>indistinguishability</b>         | The property that two systems are indistinguishable to any distinguisher (e.g., bounds in the QROM).                                                                                              |
| <b>domain separation</b>            | Using a frame-encoding suffix so that the inputs of different instances/uses do not overlap.                                                                                                      |
| <b>ARX</b>                          | An Addition–Rotation–XOR-type construction.                                                                                                                                                       |
| <b>MDS</b>                          | A Maximum-Distance-Separable matrix.                                                                                                                                                              |
| <b>S-box</b>                        | A nonlinear substitution component; in this algorithm, the AFS-64 ARX large S-box.                                                                                                                |
| <b>GF(32)</b>                       | The 32-element finite field $\text{GF}(2^5)$ .                                                                                                                                                    |
| <b>AFS</b>                          | A Family of ARX-Based Large-State S-boxes.                                                                                                                                                        |
| <b>TrEDM</b>                        | Truncated Encrypted Davies–Meyer.                                                                                                                                                                 |
| <b>LMDS</b>                         | Linear MDS (linear MDS diffusion layer).                                                                                                                                                          |
| <b>IV</b>                           | Initial Value.                                                                                                                                                                                    |
| <b>CI</b>                           | Constrained-Input (keyless-CI reduction).                                                                                                                                                         |
| <b>RPM</b>                          | Random Permutation Model.                                                                                                                                                                         |
| <b>XOF</b>                          | eXtendable-Output Function.                                                                                                                                                                       |
| <b>FIPS</b>                         | Federal Information Processing Standard (U.S.).                                                                                                                                                   |
| <b>ICCS</b>                         | Institute of Commercial Cryptography Standards.                                                                                                                                                   |
| <b>FNV-1a / SplitMix64</b>          | A public mixing function (used for IV and round-constant derivation).                                                                                                                             |

### 1.2.2 Algorithm Parameters and Other Variables

|                         |                                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b><math>b</math></b>   | Public permutation width (bits), fixed at 1600.                                                                         |
| <b><math>r</math></b>   | Rate (bits).                                                                                                            |
| <b><math>c</math></b>   | Capacity (bits), $c=b-r$ .                                                                                              |
| <b><math>d</math></b>   | Digest length / output length (bits), $d \in \{512, 768, 1024\}$ (aligned with the definition of $d$ in FIPS 202 §2.2). |
| <b><math>a</math></b>   | Number of feed-forward bits (bits), $a= S =c$ .                                                                         |
| <b><math>n_r</math></b> | Total number of rounds (12 / 20 / 24).                                                                                  |
| <b><math>i_r</math></b> | Round index ( $0 \leq i_r < n_r$ ).                                                                                     |

|                                                                   |                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $A$                                                               | State array ( $5 \times 5 \times 64$ ).                                                                                                                                                                                                                                                                   |
| $X$                                                               | State bit string / chain value.                                                                                                                                                                                                                                                                           |
| $S$                                                               | Capacity alignment window, occupying state bit positions $state[r \dots b-1]$ , $ S =c$ . Note: in this specification $S$ denotes the alignment window, in contrast to FIPS 202, where $S$ denotes the full state string — here the full state string is denoted $X$ and the state array is denoted $A$ . |
| $M$                                                               | Message.                                                                                                                                                                                                                                                                                                  |
| $F$                                                               | The frame-encoded bit string ( $ F $ is a multiple of $r$ ).                                                                                                                                                                                                                                              |
| $B_k$                                                             | The $k$ -th $r$ -bit frame block of $F$ .                                                                                                                                                                                                                                                                 |
| $T$                                                               | Capacity backup value ( $T = \text{proj}_s(X)$ , used for the EDM feed-forward).                                                                                                                                                                                                                          |
| $Z$                                                               | Digest output (the result of sponge squeezing, $ Z =d$ ).                                                                                                                                                                                                                                                 |
| $H$                                                               | Abstract hash function (denoted $H(M)$ in security arguments).                                                                                                                                                                                                                                            |
| $IV$                                                              | Initial value (the 25 64-bit words derived by <code>derive_IV</code> ).                                                                                                                                                                                                                                   |
| $g, h$                                                            | The first/second half of the public permutation AFS- $p$ -S6 (the two permutations of the absorbing-core EDM; the first $s=n_r/2$ rounds form $g$ and the last $s$ rounds form $h$ ).                                                                                                                     |
| $\kappa$                                                          | The 32-bit round-lane constant of AFS-64 (XOR-injected at each basic step).                                                                                                                                                                                                                               |
| RC32                                                              | The round-constant function $\text{RC32}(i_r, l)$ , deriving $\kappa$ per (round, lane).                                                                                                                                                                                                                  |
| $dir$                                                             | The current direction of the six-direction rotation, $dir = \text{DIR\_SCHEDULE}[i_r \bmod 6] \in \{\infty, 0, 1, 2, 3, 4\}$ .                                                                                                                                                                            |
| $w_D, w_L$                                                        | The weight lower bound (in bits) of a single optimal differential/linear trail.                                                                                                                                                                                                                           |
| $\ell, \ell_{\max}$                                               | The number of frame blocks / the maximum number of frame blocks under the permitted message length.                                                                                                                                                                                                       |
| $\sigma$                                                          | The total number of absorbing-core evaluations induced by a classical adversary.                                                                                                                                                                                                                          |
| $N$                                                               | The upper bound on core candidates in the reduction ( $N = \sigma + 2\ell_{\max}$ ).                                                                                                                                                                                                                      |
| $\varepsilon_{\text{str}}$                                        | The residual risk by which concrete AFS- $p$ -S6 deviates from the structure-level ideal (dual-random-permutation) baseline.                                                                                                                                                                              |
| $K_{\text{coll}}, K_{\text{pre}}, K_{\text{spr}}, K_{\text{bad}}$ | The small constants for chain localization and bad-event counting in the union bound.                                                                                                                                                                                                                     |

### 1.2.3 Basic Operations and Functions

|                                                     |                                                |
|-----------------------------------------------------|------------------------------------------------|
| <code>//</code>                                     | Bit-string concatenation.                      |
| $\oplus$                                            | Bitwise XOR.                                   |
| $\boxplus$                                          | Addition modulo $2^{32}$ .                     |
| $\ll / \gg$                                         | Left/right shift.                              |
| $\text{ROTR32}(\cdot, r) / \text{ROTR64}(\cdot, r)$ | 32/64-bit circular right rotation by $r$ bits. |
| $\lceil \cdot \rceil / \lfloor \cdot \rfloor$       | Ceiling/floor.                                 |
| $\log_2, \min, \text{mod}$                          | Base-2 logarithm, minimum, and modulo.         |
| $ X $                                               | The length of the bit string $X$ .             |
| $\cdot$                                             | Multiplication over $\text{GF}(32)$ .          |

|                                                                                            |                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trunc<sub>d</sub>(X)</b>                                                                | Take the leftmost $d$ bits of $X$ (digest truncation; consistent with FIPS 202).                                                                                                                                                                                                                                                                                        |
| <b>projs(X)</b>                                                                            | Project the state onto the capacity alignment window $S$ .                                                                                                                                                                                                                                                                                                              |
| <b>left<sub>d</sub>(X)</b>                                                                 | = Trunc <sub>d</sub> (X) (an alias for digest extraction).                                                                                                                                                                                                                                                                                                              |
| <b>enc64 / enc16</b>                                                                       | 64/16-bit big-endian integer encoding.                                                                                                                                                                                                                                                                                                                                  |
| <b>AFS-<math>p</math>-S6[1600, <math>n_r</math>]</b>                                       | The 1600-bit public permutation, iterating round $n_r$ times.                                                                                                                                                                                                                                                                                                           |
| <b>round(<math>A</math>, <math>i_r</math>) / permute(<math>A</math>, <math>n_r</math>)</b> | Round function / full permutation.                                                                                                                                                                                                                                                                                                                                      |
| <b>AFS-64 (AFS64_t5_k2)</b>                                                                | An ARX-Feistel-type 64-bit S-box.                                                                                                                                                                                                                                                                                                                                       |
| <b><math>L</math> (<math>L_P</math>)</b>                                                   | The S6 linear diffusion layer = $\mu \circ \pi_{\text{AFS}} \circ \rho_{\text{AFS}} \circ M_{\text{col}}$ (GF(32) Cauchy MDS column mixing, lane circular shift, long-period lane permutation, per-lane ring diffusion).                                                                                                                                                |
|                                                                                            | <b><math>\tau_{\text{dir}}</math></b><br>Rewire the lane coordinates according to the current direction $\text{dir}$ (a directional reinterpretation with compile-time fixed wiring); its inverse $\tau_{\text{dir}}^{-1}$ restores the canonical coordinates after the single MDS linear layer, i.e., $L = \tau_{\text{dir}}^{-1} \circ L_P \circ \tau_{\text{dir}}$ . |
| <b>derive_IV(<math>d</math>, <math>r</math>, <math>c</math>, <math>n_r</math>)</b>         | Initial-value derivation.                                                                                                                                                                                                                                                                                                                                               |
| <b>frame<sub>d</sub>(<math>M</math>, <math>r</math>, <math>d</math>, <math>c</math>)</b>   | Frame encoding.                                                                                                                                                                                                                                                                                                                                                         |
| <b>parse(<math>F</math>, <math>r</math>)</b>                                               | The message is parsed into $r$ -bit blocks.                                                                                                                                                                                                                                                                                                                             |
| <b>AFS-TrEDM-<math>d</math>(<math>M</math>)</b>                                            | A hash algorithm with $d \in \{512, 768, 1024\}$ .                                                                                                                                                                                                                                                                                                                      |
| <b>Conventions</b>                                                                         |                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Bit and Byte Conventions</b>                                                            | Bits are denoted by $\{0,1\}$ ; the external CryptHash interface uses an MSB-first partial-byte convention; the internal 1600-bit state has two equivalent views, a “bit string” and a “ $5 \times 5 \times 64$ array”.                                                                                                                                                 |
| <b>GF(32) Bit-Slicing</b>                                                                  | The five coefficients of a field element $s = s_0 + s_1\alpha + \dots + s_4\alpha^4$ are each carried by one of the five lanes; a single GF(32) symbol represents 64 parallel 5-bit field elements (see §1.6.1 for details).                                                                                                                                            |

### 1.3 Algorithm Instances and Parameters

The three instances of the AFS-TrEDM family share the same bit-ordering convention, the same AFS-64 S-box (AFS64\_t5\_k2), the same AFS-LMDS-1600-S6 linear layer, and the same round-constant derivation rule, differing only in the rate  $r$ , the capacity (i.e., the alignment window)  $c$ , the digest length  $d$ , and the tiered round counts  $n_r$ . Table 1 gives the complete parameters of the three instances; the 768-bit instance is optional.

Table 1. Parameters of the three instances of the AFS-TrEDM family

| Instance       | Digest $d$<br>(bits) | State $b$<br>(bits) | Rate $r$<br>(bits) | Capacity<br>$c$<br>(bits) | Feed-<br>forward<br>$a$<br>(bits) | $g+h$<br>(rounds) | State     |
|----------------|----------------------|---------------------|--------------------|---------------------------|-----------------------------------|-------------------|-----------|
| AFS-TrEDM-512  | 512                  | 1600                | 1024               | 576                       | 576                               | 6+6 (12)          | Mandatory |
| AFS-TrEDM-768  | 768                  | 1600                | 768                | 832                       | 832                               | 10+10 (20)        | Optional  |
| AFS-TrEDM-1024 | 1024                 | 1600                | 512                | 1088                      | 1088                              | 12+12 (24)        | Mandatory |

For every instance we have  $b = r + c = 1600$  bits and  $a = c$ . The aligned capacity window  $S$  occupies state bit positions  $\text{state}[r \dots 1599]$ , i.e., the rightmost  $c$  bits of the 1600-bit state; the feed-forward window, the chain-value window, and the digest-output window are all located at  $S$  (the digest being the leftmost  $d$  bits of  $S$ ). This

specification sets  $a = |S| = c$ , so that the part of the capacity not involved in the feed-forward,  $c \setminus S$ , degenerates to the empty set, and  $S$  coincides with the entire capacity window. Under the lane-serialization convention of §1.2, the  $S$  windows of the three instances are, respectively,  $A[16...24]$  for AFS-TrEDM-512,  $A[12...24]$  for AFS-TrEDM-768, and  $A[8...24]$  for AFS-TrEDM-1024.

Naming of the underlying permutation family. This specification defines the tiered-round permutation family AFS- $p$ -S6[1600,  $n_r$ ], whose state width is fixed at 1600 bits, whose intra-round linear layer is AFS-LMDS-1600-S6, and whose round-count parameter  $n_r$  takes the values 12 / 20 / 24 across the instances. The three instances shall each have independent initial values, KATs, and implementation identifiers, and shall not share a single set of test and implementation parameters.

## 1.4 Auxiliary Functions: Message Parsing, Padding, and Frame Encoding

AFS-TrEDM applies frame encoding (framing) to the message before absorbing. This specification first gives the two auxiliary functions — message parsing and padding/frame encoding — which are then invoked by the main algorithm in §1.10. Let the rate be  $r$  bits (taking the values 1024 / 768 / 512 across instances).

AFS-TrEDM adopts a concrete, fixed-length frame-encoding instance in order to satisfy the requirements of injective parsability, block-level suffix-freeness, cross-instance domain separation, and engineering friendliness. The fixed suffix is taken to be 128 bits because a 64-bit message length plus four 16-bit instance/version fields is exactly 16 bytes, and is smaller than the minimum rate of all instances (512 bits); the final frame therefore spans at most two rate blocks, which facilitates a byte-aligned constant-time implementation. This 128-bit suffix is designed with four practical purposes.

First, length commitment.  $\text{enc64}(|M|)$  writes the bit length of the original message into the final 64-bit field at a fixed position; even if two messages are related at the bit level by “one being the other padded with some zeros,” the original length can be recovered from the tail during decoding, which then locates the unique 1-padding bit and the original message boundary. Proposition E-1 in Appendix E relies precisely on the fact that “the final 128 bits of metadata lie at a fixed position, in which  $\text{enc64}(|M|)$  uniquely determines the message length” to prove the injectivity and block-level suffix-freeness of  $\text{frame\_d}$ .

Second, cross-instance domain separation.  $\text{enc16}(d) \parallel \text{enc16}(r) \parallel \text{enc16}(c)$  writes the output length, rate, and capacity of the three tiers 512, 768, 1024 into the message frame; even if the same byte string enters different instances, the tail metadata actually absorbed differs, so that the inputs of different instances/uses do not overlap after frame encoding.

Third, version isolation.  $\text{enc16}(2)$  writes the algorithm version into the frame; should the padding, linear layer, round count, bit order, or absorbing rule change in the future, the version field prevents different versions from being mixed at the message-encoding level.

Fourth, engineering friendliness. 128 bits is exactly 16 bytes, naturally laid out as an 8-byte message length, a 2-byte  $d$ , a 2-byte  $r$ , a 2-byte  $c$ , and a 2-byte version; when the message is byte-aligned, the single 1-bit is exactly one 0x80 byte, and the 128-bit suffix is placed byte-aligned at the tail of the final rate block. §2.3.3 further explains that this arrangement makes it convenient for the reference implementation to provide a byte-aligned fast path.

Message parsing  $\text{parse}(M, r)$ . Parse the message  $M$  into a number of  $r$ -bit blocks and a tail block of length in  $[0, r-1]$ . When  $|M|$  is a multiple of  $r$ , the tail block is empty.

### Algorithm 1 $\text{parse}(M, r)$

**Input:** bit string  $M$ , positive integer  $r$

**Output:** blocks  $M_0, \dots, M_{\ell-1}$  and tail block  $\tilde{M}_\ell$

```

 $\ell \leftarrow \lfloor |M| / r \rfloor$ 
for  $i = 0$  to  $\ell - 1$  do
   $M_i \leftarrow M[i \cdot r : (i+1) \cdot r - 1]$ 
end for
 $\tilde{M}_\ell \leftarrow M[\ell \cdot r : |M| - 1]$ 
return  $M_0, \dots, M_{\ell-1}, \tilde{M}_\ell$ 

```

Frame encoding  $\text{frame}_d(M)$ . In AFS-TrEDM, padding and frame encoding are unified into one: after the message, a single 1-bit, several 0-bits, and then a fixed 128-bit metadata suffix are appended, making the overall length a multiple of  $r$ . The metadata suffix binds the message length and the API-level instance parameters ( $d, r, c$ , version); the underlying round count, S-box, and linear-layer identity are bound separately by the IV domain string.

$$\text{frame}_d(M) = M \| 1 \| 0^j \| \text{enc}_{64}(|M|) \| \text{enc}_{16}(d) \| \text{enc}_{16}(r) \| \text{enc}_{16}(c) \| \text{enc}_{16}(\text{version})$$

where  $\text{version} = 2$ ;  $\text{enc}_{64}$  and  $\text{enc}_{16}$  are 64-bit and 16-bit big-endian encodings, respectively;  $j$  is the smallest non-negative integer that makes the overall length a multiple of  $r$ . The fixed suffix after the first padding 1-bit is 128 bits long ( $64 + 16 + 16 + 16 + 16$ ). All the 0-bits after 0x80 and before the metadata suffix belong to 0; there is no separate “8-bit gap” field. The maximum message length supported by this interface is  $2^{64} - 1$  bits.

**Algorithm 2**  $\text{frame}_d(M, r, d, c)$

**Input:** bit string  $M$ , rate  $r$ , digest length  $d$ , capacity  $c$

**Output:** the frame-encoded bit string  $F$ , with  $|F|$  a multiple of  $r$

```

suffix  $\leftarrow \text{enc}_{64}(|M|) \text{ // } \text{enc}_{16}(d) \text{ // } \text{enc}_{16}(r) \text{ // } \text{enc}_{16}(c) \text{ // } \text{enc}_{16}(2) \text{ // } 128 \text{ bits total}$ 
 $j \leftarrow (-|M| - 1 - 128) \bmod r$ 
 $F \leftarrow M \text{ // } 1 \text{ // } 0^j \text{ // suffix}$ 
return  $F$ 

```

Frame encoding is parsed only once at the API boundary, not repeated for each block. When  $(|M| \bmod r) + 1 + 128 \leq r$ , the final frame occupies only one block; otherwise it spans two blocks. When  $|M|$  is a multiple of 8, the 1-bit is exactly the most significant bit of the next byte (a single 0x80 byte), and the 128-bit suffix lies at the tail of the final block, byte-aligned, on which the reference implementation provides a byte-aligned fast path.

## 1.5 Nonlinear Layer: the AFS-64 S-box

The selected S-box is AFS64\_t5\_k2, taken from the 64-bit ARX large-state S-box family AFS-64<sup>[3]</sup> proposed by Yan, Wei, Rodríguez-Aldama, Pasalic, et al. Its structure code is  $A8 = 11000011$ , and its rotation sequence is  $K8 = [17, 24, 1, 1, 16, 31, 24, 0]$ .

The 64-bit input  $W$  is split into a high 32-bit branch  $x$  and a low 32-bit branch  $y$ , i.e.,  $W = x \| y$ . All additions are modulo  $2^{32}$ . Given a 32-bit lane constant  $\kappa$ , the S-box consists of 8 ARX-Feistel basic steps (steps 0/1/6/7 are modular additions and steps 2/3/4/5 are pure XORs, corresponding to the structure code 11000011), as shown in Eq. (1) and Algorithm 3.

### (1) The 8-step definition of the AFS-64 S-box:

```

x = x  $\boxplus$  ROTR32(y,17); x = x  $\oplus$   $\kappa$ ;      // step 0: modular addition
y = y  $\boxplus$  ROTR32(x,24); y = y  $\oplus$   $\kappa$ ;      // step 1: modular addition
x = x  $\oplus$  ROTR32(y,1);                       // step 2: XOR
y = y  $\oplus$  ROTR32(x,1);                       // step 3: XOR
x = x  $\oplus$  ROTR32(y,16);                      // step 4: XOR
y = y  $\oplus$  ROTR32(x,31);                      // step 5: XOR
x = x  $\boxplus$  ROTR32(y,24); x = x  $\oplus$   $\kappa$ ;      // step 6: modular addition
y = y  $\boxplus$  ROTR32(x,0); y = y  $\oplus$   $\kappa$ ;      // step 7: modular addition
return x  $\ll$  y;

```

Here  $\boxplus$  denotes addition modulo  $2^{32}$  and  $\oplus$  is bitwise XOR. Step 7 uses  $\text{ROTR32}(x, 0)$ , which is the identity rotation, corresponding to the value 0 at position 7 in K8; this step is written explicitly in the form  $\text{ROTR32}(x, 0)$  so that the round structure aligns exactly with the K8 table indices and corresponds line-by-line to the reference implementation. The four modular-addition steps are the sole source of nonlinearity; the round constant  $\kappa$  is injected via  $\oplus c$ , and in a differential pair the two trails are identical and cancel under XOR, so the single-box differential weight is independent of the round constant. Algorithm 3 gives the pseudocode that corresponds one-to-one with the reference implementation.

**Algorithm 3** AFS64( $W, \kappa$ )

**Input:** 64-bit word  $W$ , 32-bit round-lane constant  $\kappa$

**Output:** 64-bit word  $W'$

```

x  $\leftarrow$   $W \gg 32$ ; y  $\leftarrow$   $W \bmod 2^{32}$            $\triangleright$  high/low 32-bit branches
x  $\leftarrow$  (x  $\boxplus$  ROTR32(y,17))  $\oplus$   $\kappa$ 
y  $\leftarrow$  (y  $\boxplus$  ROTR32(x,24))  $\oplus$   $\kappa$ 
x  $\leftarrow$  x  $\oplus$  ROTR32(y, 1)
y  $\leftarrow$  y  $\oplus$  ROTR32(x, 1)
x  $\leftarrow$  x  $\oplus$  ROTR32(y,16)
y  $\leftarrow$  y  $\oplus$  ROTR32(x,31)
x  $\leftarrow$  (x  $\boxplus$  ROTR32(y,24))  $\oplus$   $\kappa$ 
y  $\leftarrow$  (y  $\boxplus$  ROTR32(x, 0))  $\oplus$   $\kappa$ 
return (x  $\ll$  32)  $\vee$  y

```

Figure 1 shows the branch flow of these 8 steps and the rotation amount of each step in dataflow form, which can be compared line-by-line with Algorithm 3 to check the correspondence between the structure code A8 and the rotation sequence K8; the figure also indicates that the entire nonlinearity of the S-box comes only from the modular additions of steps 0, 1, 6, and 7.

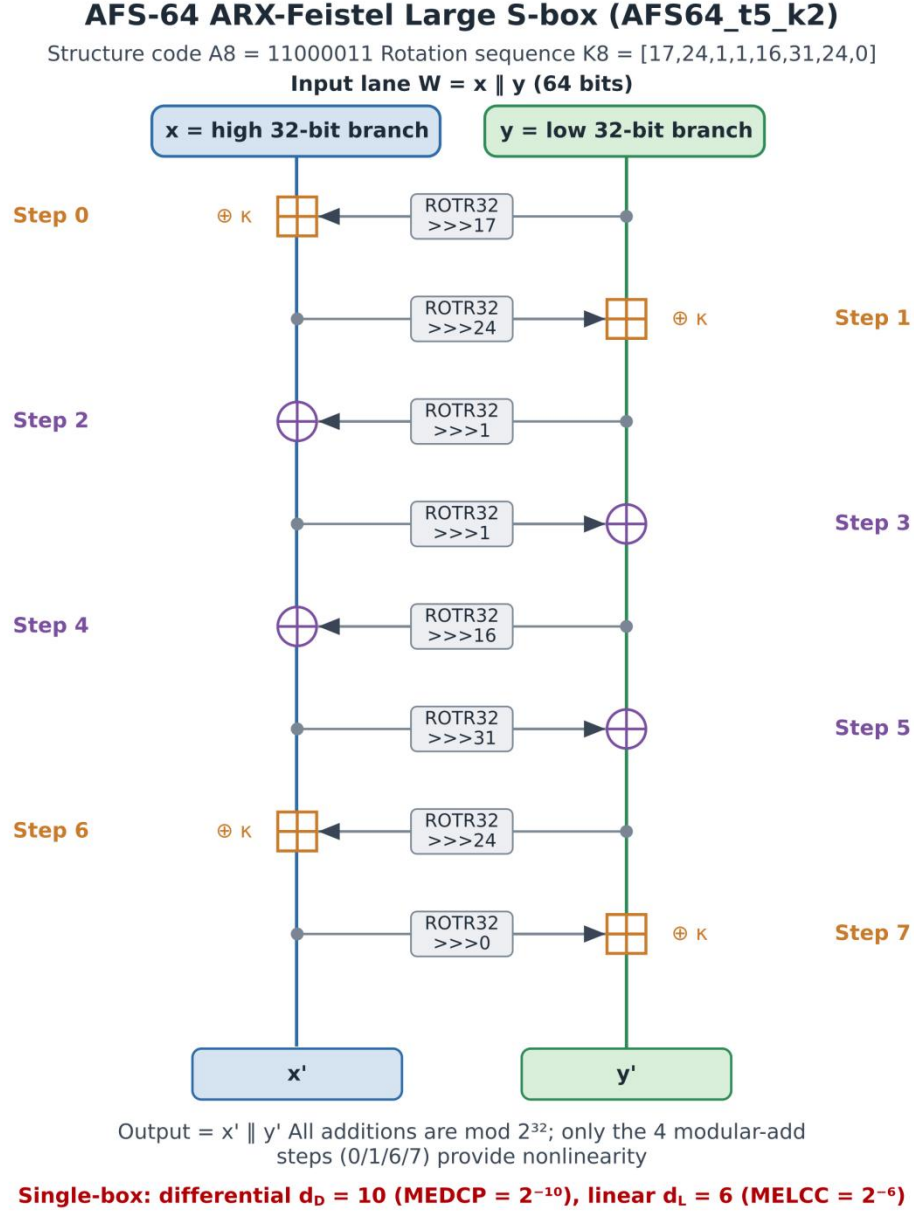


Fig. 1. The 8-step structure of the AFS-64 large S-box (AFS64\_t5\_k2)

## 1.6 Linear Diffusion Layer: AFS-LMDS-1600-S6

The linear diffusion layer of AFS- $p$ -S6[1600,  $n_r$ ] is the six-direction rotating single-MDS scheme AFS-LMDS-1600-S6 (six-direction rotating single-MDS). It takes the 25 64-bit lanes as its natural state view, and each round performs one bit-sliced GF(32) Cauchy MDS column mixing, one AFS-specific lane circular shift, one 1+24 long-period lane permutation, and one per-lane invertible ring diffusion, while rotating the acting direction of the MDS among six projective directions on a 6-round period. This layer only changes the linear diffusion across lanes and within lanes; it does not redefine the state partition or the  $S$  window; the position of  $S$  is determined by the outer aligned-TrEDM specification. This section first gives the single-MDS building block  $L_p$ , and then the six-direction rotation  $L$ .

### 1.6.1 The GF(32) Field and Constant Multiplication

$\text{GF}(32)=F_2[\alpha]/(\alpha^5+\alpha^2+1)$ . The xtime operation of multiplying by the primitive element  $\alpha$  is (since  $\alpha^5 = \alpha^2 + 1$ ):

$$\alpha \cdot (S_0, S_1, S_2, S_3, S_4) = (S_4, S_0, S_1 \oplus S_4, S_2, S_3)$$

The multiplication  $m_c : S \mapsto c \cdot S$  by an arbitrary 5-bit constant  $c$  is a  $5 \times 5$  linear map over  $\text{GF}(2)$ , which can be implemented as a combination of xtime and bitwise selection, or expanded into a fixed XOR network. Since each coefficient in the bit-sliced representation is a 64-bit lane, constant multiplication is executed bitwise in parallel over the 64 bit-slices. The reference implementation uses generic xtime/multiply code to ensure readability and audibility; the optimized implementation can emit a constant-multiplication XOR network from a code generator. The design report gives the multiplication-network cost of the eight classes of constants (01, 02, 05, 06, 09, 10, 11, 13) used by the  $C$  matrix.

### 1.6.2 The Cauchy MDS Matrix $C$ and Column Mixing $M_{\text{col}}$

Take two disjoint sets of  $\text{GF}(32)$  elements  $a=(1F,0C,1A,09,06)$  and  $b=(07,02,0D,1E,08)$ , each with distinct internal elements, and define the Cauchy matrix  $C_{i,j}=(a_i+b_j)^{-1}$  ( $0 \leq i, j < 5$ ). Its concrete values over  $\text{GF}(32)$  are given in Table 2; the inverse matrix  $C^{-1}$  is given in the design report.

Table 2. The  $\text{GF}(32)$  Cauchy MDS matrix  $C$  used by AFS-LMDS-1600 (hexadecimal)

| $C[i,j]$ | $j=0$ | $j=1$ | $j=2$ | $j=3$ | $j=4$ |
|----------|-------|-------|-------|-------|-------|
| $i=0$    | 11    | 13    | 02    | 01    | 05    |
| $i=1$    | 10    | 06    | 01    | 02    | 09    |
| $i=2$    | 13    | 11    | 05    | 09    | 02    |
| $i=3$    | 06    | 10    | 09    | 05    | 01    |
| $i=4$    | 01    | 09    | 10    | 11    | 06    |

Column-symbol MDS mixing  $M_{\text{col}}$ . For each column coordinate  $x$ , combine the five lanes of that column into a bit-sliced  $\text{GF}(32)$  column symbol  $U_x = A[x,0] + \alpha A[x,1] + \alpha^2 A[x,2] + \alpha^3 A[x,3] + \alpha^4 A[x,4]$ . Form the five column symbols into a column vector  $U = (U_0, \dots, U_4)^T$ , compute  $V = C \cdot U$ , and split the five coefficients of  $V_i$  back into five lanes and write them back to the state. This step is executed in parallel over the 64 bit positions. Note: the symbol-level branch number of  $C$  is 6, but a  $\text{GF}(32)$  symbol is composed of the same bit-sliced coefficient of the five lanes in the same column, so the symbol-level branch number 6 is not directly equal to the lane-level or bit-level active-S-box lower bound; the latter is given by the solver verification cited in §3.3.1.

### 1.6.3 The AFS-Specific Lane Circular Shift $\rho_{\text{AFS}}$

Each lane undergoes a fixed circular right rotation  $A[x, y] \leftarrow \text{ROTR64}(A[x, y], \rho[x, y])$ ; the offset table is given in Table 3. The offsets are spread over 0...63 and, together with the subsequent lane permutation, make the total bit rotation amount (the cumulative sum) accumulated along the 24-lane long orbit coprime to 64, so that the lane-position period and the bit-position period do not close prematurely. Note: the coprimality requirement is on the cumulative rotation amount summed along the long orbit, not on a single offset  $\rho[x, y]$ ; a single offset may be even (e.g., 24/14/6/8/10/34/28, etc., in Table 3). §1.6.4 shows that the cumulative offset of this 24-orbit is 23, and  $\text{gcd}(23, 64) = 1$ .  $\rho_{\text{AFS}}$  and the  $\mu$  of §1.6.5 are the main source of bit-level diffusion speed ( $M_{\text{col}}$  runs bitwise in parallel over the 64 bit-slices and does not itself propagate across bit positions).

Table 3. The lane circular-shift offset table  $\rho[x, y]$  of AFS-LMDS-1600

| $\rho[x, y]$ | $y = 0$ | $y = 1$ | $y = 2$ | $y = 3$ | $y = 4$ |
|--------------|---------|---------|---------|---------|---------|
| $x = 0$      | 0       | 1       | 3       | 7       | 19      |
| $x = 1$      | 5       | 9       | 23      | 11      | 24      |
| $x = 2$      | 14      | 6       | 25      | 8       | 21      |
| $x = 3$      | 10      | 15      | 29      | 37      | 34      |
| $x = 4$      | 28      | 33      | 39      | 43      | 27      |

#### 1.6.4 The Long-Period Lane Permutation $\pi_{\text{AFS}}$

Define the lane-coordinate permutation  $(x, y) \mapsto ((x + y) \bmod 5, 3x \bmod 5)$ , i.e., move  $A[x, y]$  to  $A[(x+y) \bmod 5, 3x \bmod 5]$ . It corresponds to the linear matrix  $P = \begin{bmatrix} 1 & 1 \\ 3 & 0 \end{bmatrix}$  over  $\mathbb{F}_5^2$ , with  $\det(P) = -3 \equiv 2 \pmod{5} \neq 0$ , and is therefore a full permutation over the 25 lane positions. Its orbit structure is  $1 + 24$ :  $(0, 0)$  is a fixed point, and the remaining 24 nonzero lanes form a single long period. The cumulative  $\rho$  offset along this 24-orbit is  $23 \pmod{64}$ , and  $\gcd(23, 64) = 1$ . This 24-orbit is the basis for the long-orbit constraint in multi-round active-S-box analysis.

#### 1.6.5 The Per-Lane Invertible Ring Diffusion $\mu$

For each 64-bit lane  $X$ , perform:

$$X \leftarrow X \oplus \text{ROTR64}(X, 17) \oplus \text{ROTR64}(X, 32)$$

This step works only within a lane and is used to increase bit-position diffusion. It corresponds to the polynomial  $m(z) = 1 + z^{17} + z^{32}$  in the ring  $R = \mathbb{F}_2[z]/(z^{64} + 1)$ ; since  $m(z)$  has 3 terms (an odd number) and takes the value 1 at  $z = 1$ , it is coprime to  $z^{64} + 1 = (z + 1)^{64}$  and is therefore a unit, with inverse  $X \leftarrow \text{ROTR64}(X, 30) \oplus \text{ROTR64}(X, 47) \oplus \text{ROTR64}(X, 62)$ ; hence  $\mu$  is invertible.

#### 1.6.6 The Single-MDS Building Block $L_P$

The single-layer MDS building block  $L_P$  is defined as the composition of four steps:

$$L_P = \mu \circ \pi_{\text{AFS}} \circ \rho_{\text{AFS}} \circ M_{\text{col}}$$

That is, it performs column-symbol MDS mixing, lane circular shift, long-period lane permutation, and per-lane ring diffusion in sequence. Since  $M_{\text{col}}$ ,  $\rho_{\text{AFS}}$ ,  $\pi_{\text{AFS}}$ , and  $\mu$  are all invertible linear transformations, their composition  $L_P$  is an invertible linear transformation on the 1600-bit state (see §3.2 and the design report for the proof). Algorithm 4 gives the pseudocode of  $L_P$ .

##### Algorithm 4 L\_P\_core(A)

**Input:** the state  $A$  of 25 64-bit lanes

**Output:** the updated state  $A$

```

// Mcol: form GF(32) symbols column by column, left-multiply by the Cauchy matrix C
for i = 0 to 4 do
  Vi ←  $\sum_{j=0 \dots 4} C[i, j] \cdot U_j$     ( $U_j$  = the five-lane slice symbol of column j)
end for; A ← V
//  $\rho_{\text{AFS}}$ : lane circular right rotation
for (x, y) do A[x, y] ← ROTR64(A[x, y],  $\rho[x, y]$ ) end for
//  $\pi_{\text{AFS}}$ : long-period lane permutation
for (x, y) do A'[(x+y)%5, 3x%5] ← A[x, y] end for; A ← A'
//  $\mu$ : per-lane ring diffusion
for i = 0 to 24 do A[i] ← A[i] ⊕ ROTR64(A[i], 17) ⊕ ROTR64(A[i], 32) end for

```

return A

### 1.6.7 Six-Direction Rotation S6 and the Per-Round Linear LayerL

In the affine plane  $F_5 \times F_5$  there are 6 projective directions in total: the vertical direction  $\infty$  and the finite slopes  $dir \in \{0, 1, 2, 3, 4\}$ . For each direction  $dir$ , define the lane-coordinate transformation  $\tau_{dir}$ :

$$\tau_{\infty}(x,y)=(x,y); \tau_{dir}(x,y)=((y-dir \cdot x) \bmod 5, x), \quad dir \in \{0,1,2,3,4\}$$

For finite  $dir$ , the transformation matrix  $T_{dir} = \begin{bmatrix} -dir & 1 \\ 1 & 0 \end{bmatrix}$ ,  $\det(T_{dir}) = -1 \equiv 4 \pmod{5} \neq 0$ , so each  $\tau_{dir}$  is a full permutation over the 25 lane positions, changing only the lane index and not the bit order within a lane. Define the 6-round direction schedule  $dir = [\infty, 0, 1, 2, 3, 4]$ , indexed by  $i_r \bmod 6$  (see Table 4). The linear layer of round  $i_r$  is:

$$L = \tau_{dir}^{-1} \circ L_P \circ \tau_{dir}$$

That is, first reinterpret the lane coordinates according to  $\tau_{dir}$ , then perform exactly the same single-MDS linear layer as  $L_P$ , and finally restore the canonical coordinates according to  $\tau_{dir}^{-1}$ . Since  $\tau$  and  $\tau^{-1}$  are both lane permutations, they can be merged into the input/output lane selection table of  $M_{col}$ , the  $\pi_{AFS}$  wiring table, and the code generator, adding no arithmetic instructions. Therefore the per-round linear-layer cost of S6 is the same as  $L_P$ . Under the  $g/h$  split, the direction schedule increases continuously by the global round number  $i_r$  and is not reset in the  $h$  segment. The  $g$  and  $h$  of the 512 tier are each 6 rounds, so each half-permutation covers the six directions once, and the full 12-round permutation covers the six directions twice; for the 768 tier, each 10-round half-permutation covers one full round of six directions plus 4 additional directions; for the 1024 tier, each 12-round half-permutation covers the six directions twice.

Table 4. The S6 six-round direction schedule

| Round number $i_r \bmod 6$ | Direction $dir$ | Line family mixed with priority |
|----------------------------|-----------------|---------------------------------|
| 0                          | $\infty$        | $x = \text{const}$              |
| 1                          | 0               | $y = \text{const}$              |
| 2                          | 1               | $y - x = \text{const}$          |
| 3                          | 2               | $y - 2x = \text{const}$         |
| 4                          | 3               | $y - 3x = \text{const}$         |
| 5                          | 4               | $y - 4x = \text{const}$         |

#### Algorithm 5 L(A, $i_r$ )

**Input:** state A, round number  $i_r$

**Output:** the updated state A

```

dir ← [∞, 0, 1, 2, 3, 4][  $i_r \bmod 6$  ]
if dir ≠ ∞ then A ←  $\tau_{dir}(A)$            ▷ lane reordering (compile-time wiring)
A ← L_P_core(A)                         ▷ Algorithm 4
if dir ≠ ∞ then A ←  $\tau_{dir}^{-1}(A)$ 
return A

```

The overall structure of AFS-LMDS-1600-S6 is shown in Figure 2: the upper part is the four-stage pipeline of the single-MDS building block  $L_P$  ( $M_{col} \rightarrow \rho_{AFS} \rightarrow \pi_{AFS} \rightarrow \mu$ ), and the lower part is the six-direction schedule  $dir = [\infty, 0, 1, 2, 3, 4]$  (by  $i_r \bmod 6$ ); 6 rounds suffice to cover all 6 projective directions of the affine plane.

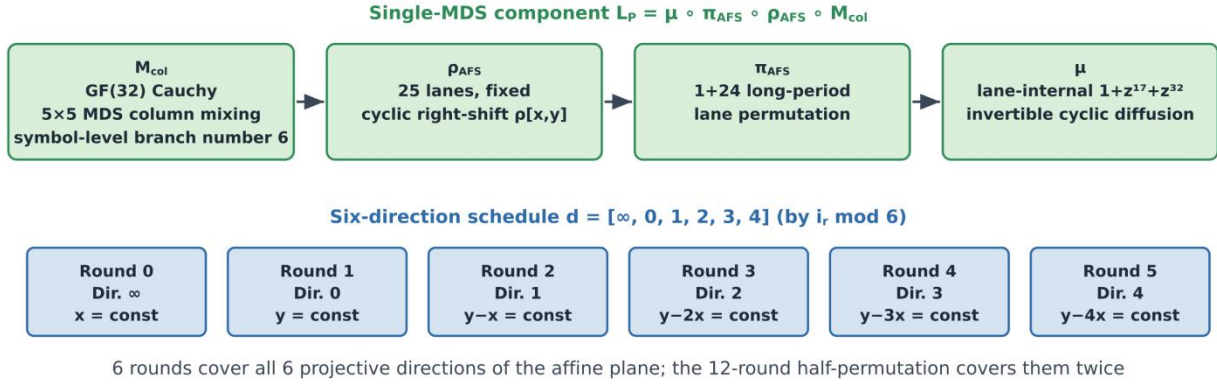
**AFS-LMDS-1600-S6: single-MDS component  $L_p$  and six-direction rotation schedule**

Fig. 2. The AFS-LMDS-1600-S6 six-direction rotating single-MDS linear layer

**1.7 Round-Lane Constants RC32**

In each AFS-64 call, the nonlinear layer XOR-injects into the 8 basic steps a 32-bit public constant  $RC32(i_r, l)$  associated with (round  $i_r$ , lane  $l$ ). This constant is deterministically derived per position from the round number  $i_r$  and the lane index  $l$ , so that each (round, lane) position injects a distinct constant, thereby breaking the rotational/translational regularity of the ARX structure. The round-lane constant derivation of this specification differs both from SHA-2 (which takes the fractional parts of the square/cube roots of several primes) and from SHA-3/Keccak (which generates the  $t$ -step round constants from a linear-feedback shift register (LFSR)): this algorithm derives the 32-bit constant per (round, lane) position based on the public mixing function SplitMix64. The three share the same design goal (publicly recomputable constants, distinct per position, breaking the ARX rotational/translational regularity), but differ in generation mechanism; SplitMix64 is chosen to obtain a table-free, per-position parallelly recomputable “nothing-up-my-sleeve” derivation. For round number  $i_r$  and lane  $l$ , let  $idx = 25 \cdot i_r + l$ , then:

$$z = \text{SplitMix64}(0xB4C5F1D62E1738A9 + idx \cdot 0x9E3779B97F4A7C15)$$

$$RC32(i_r, l) = \text{low32}(z \oplus (z \gg 32))$$

where SplitMix64( $x$ ) is the standard mixing function, all 64-bit operations are modulo  $2^{64}$ , and low32( $\cdot$ ) takes the lowest 32 bits:

```
SplitMix64(x):
    x = x + 0x9E3779B97F4A7C15
    x = (x ⊕ (x >> 30)) · 0xBF58476D1CE4E5B9
    x = (x ⊕ (x >> 27)) · 0x94D049BB133111EB
    return x ⊕ (x >> 31)
```

The seed  $0xB4C5F1D62E1738A9$  is the publicly fixed round-lane constant seed; the increment  $0x9E3779B97F4A7C15$  is the 64-bit fixed-point encoding of the golden ratio. AFS-TrEDM-512 / 768 / 1024 use the first  $25 \cdot 12 = 300$ ,  $25 \cdot 20 = 500$ , and  $25 \cdot 24 = 600$  constants, respectively. It has been verified that the constants used by the three tiers are all nonzero and non-repeating within their respective (round, lane) domains (see the reference-implementation tool `check_round_constants.py`). The publicly deterministic derivation of the constants eliminates the possibility of a hidden backdoor (see §2.3.1 for the design motivation).

**1.8 The Public Permutation AFS- $p$ -S6[1600,  $n_r$ ]**

AFS- $p$ -S6[1600,  $n_r$ ] consists of  $n_r$  rounds acting on the 1600-bit state ( $n_r = 12 / 20 / 24$ ). Each round first applies

one AFS-64 S-box to each of the 25 lanes (injecting the corresponding  $\text{RC32}(i_r, l)$  constant), and then applies the AFS-LMDS-1600-S6 linear layer  $L$  of that round (Algorithm 5). The single-round structure is shown in Figure 3; the round function and full permutation are given in Algorithm 6.

### Single-round structure of the underlying permutation AFS-p-S6[1600, $n_r$ ]

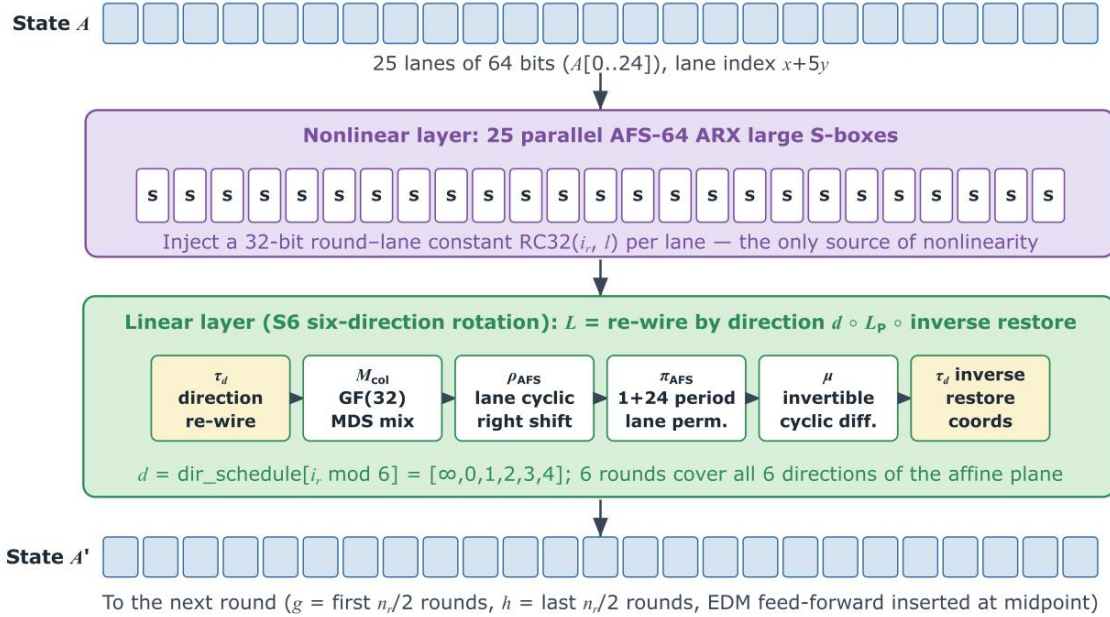


Fig. 3. The single-round structure of AFS-p-S6[1600,  $n_r$ ]

#### Algorithm 6 AFS-p-S6[1600, $n_r$ ]: round( $A, i_r$ ) and permute( $A, n_r$ )

**Input:** state  $A$ , round number  $i_r$  or total round count  $n_r$

**Output:** the updated state  $A$

```

round( $A, i_r$ ):
  for  $l = 0$  to  $24$  do  $A[l] \leftarrow \text{AFS64}(A[l], \text{RC32}(i_r, l))$  end for    ▷ nonlinear layer
   $A \leftarrow L(A, i_r)$                                               ▷ linear layer (Algorithm 5)
  return  $A$ 
permute( $A, n_r$ ):
  for  $i_r = 0$  to  $n_r - 1$  do  $A \leftarrow \text{round}(A, i_r)$  end for
  return  $A$ 

```

The  $g/h$  split. The aligned-TrEDM mode requires splitting the single permutation into two halves,  $g$  and  $h$ . Let the split round count be  $s$  (6 for 512, 10 for 768, 12 for 1024, i.e.,  $s = n_r/2$ ); then  $g$  consists of rounds  $0 \dots s-1$  and  $h$  consists of rounds  $s \dots 2s-1$ :

$$g(A) = \text{round}_{0 \dots s-1}(A); \quad h(A) = \text{round}_{s \dots 2s-1}(A); \quad P = h \circ g$$

$g$  and  $h$  share the same RC32 constants and the same  $L$  schedule, with the round number increasing continuously between  $g$  and  $h$  (so that the S6 direction schedule and the round constants are continuous across the two halves). This is precisely the  $P = h \circ g$  form required by the aligned-TrEDM absorbing of §1.10.

## 1.9 Initial Value (IV)

The IV of each instance is a public 1600-bit constant (25 64-bit lanes), derived by using the FNV-1a64 hash of

the instance-specific domain string as a seed and taking the first 25 outputs of SplitMix64. The domain string of the S6 version explicitly contains the tiered round count  $n_r$  and the linear-layer identifier:

"AFS-TrEDM-v2|d=<d>|r=<r>|c=<c>|b=1600| $n_r$ =< $n_r$ >|sbox=AFS64\_t5\_k2|linear=AFS-LMDS-1600-S6"

where  $\langle d \rangle$ ,  $\langle r \rangle$ ,  $\langle c \rangle$ ,  $\langle n_r \rangle$  are replaced by the decimal literals of the corresponding instance parameters (for example, for AFS-TrEDM-512,  $d=512$ ,  $r=1024$ ,  $c=576$ ,  $n_r=12$ ). Since the  $(d, r, c, n_r)$  of the three tiers are mutually distinct and the linear-layer identifier is AFS-LMDS-1600-S6, the three instances derive three mutually distinct IVs bound to S6. The IV derivation is given in Algorithm 7, the complete IV data is listed in Appendix A, and the optimized implementation compiles it directly into the binary as a constant.

**Algorithm 7 IV derivation:  $\text{derive\_IV}(d, r, c, n_r)$**

**Input:** instance parameters  $d, r, c, n_r$

**Output:** the initial value IV[0...24] (25 64-bit words)

```

s ← "AFS-TrEDM-v2|d=<d>|r=<r>|c=<c>|b=1600| $n_r$ =< $n_r$ >|sbox=AFS64_t5_k2|linear=AFS-LMDS-
1600-S6"
x ← FNV-1a64(s)                                ▷ 64-bit FNV-1a hash
for i = 0 to 24 do
  x ← x + 0x9E3779B97F4A7C15
  z ← (x ⊕ (x >> 30)) · 0xBF58476D1CE4E5B9
  z ← (z ⊕ (z >> 27)) · 0x94D049BB133111EB
  IV[i] ← z ⊕ (z >> 31)
end for; return IV

```

## 1.10 Aligned-TrEDM Absorbing and Squeezing

The absorbing-squeezing process of AFS-TrEDM absorbs the message block by block into the rate part of the state, compresses with a truncated Encrypted Davies–Meyer (TrEDM) core in each absorbing step, and finally extracts the digest from the capacity alignment window  $S$ . Let the state  $A$  be initialized to the IV of the corresponding instance, the rate region lie in the low  $r$  bits of  $A$  (lanes  $A[0 \dots r/64 - 1]$ ), and the aligned capacity window  $S = \text{state}[r \dots 1599]$  (lanes  $A[r/64 \dots 24]$ ). Denote by  $\text{proj}_S(A)$  the subvector on  $S$ , and by  $X[S]$  the part of the state located at  $S$ . The overall absorbing-squeezing dataflow is shown in Figure 4.

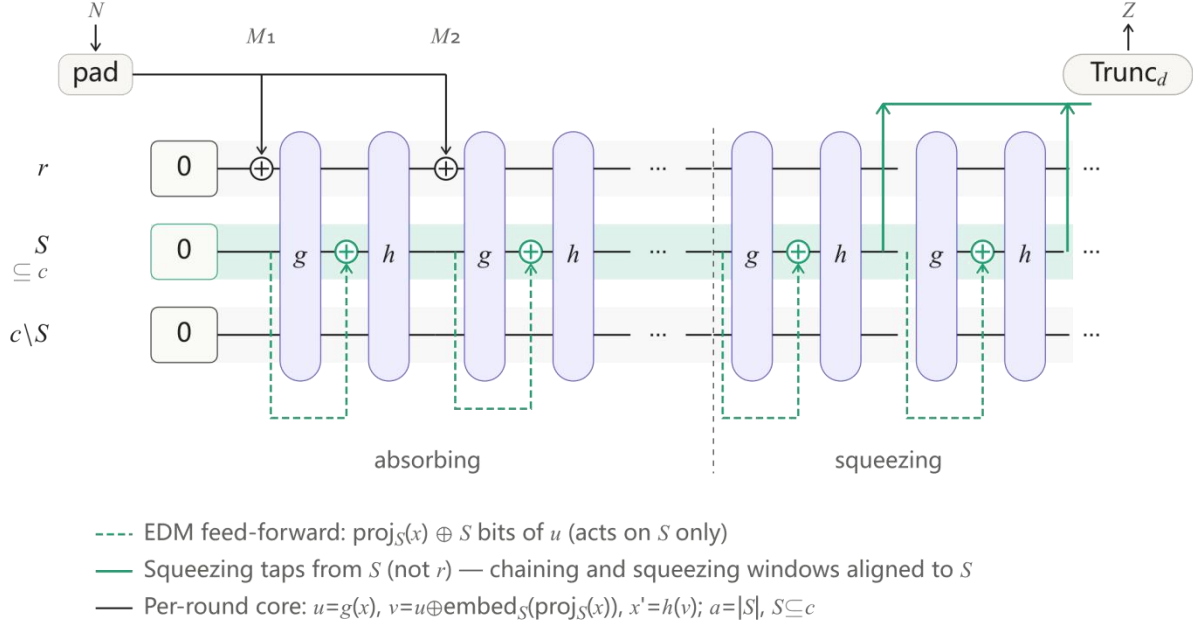


Fig. 4. The aligned-TrEDM absorbing and squeezing dataflow

The absorbing core (five steps per block). For each  $r$ -bit block  $B$  after frame encoding:

$$X[0 \dots r-1] \leftarrow X[0 \dots r-1] \oplus B; \quad T \leftarrow \text{proj}_S(X); \quad X \leftarrow g(X); \quad X[S] \leftarrow X[S] \oplus T; \quad X \leftarrow h(X)$$

That is: (i) XOR the block into the rate part; (ii) back up the current value of the capacity window  $T$ ; (iii) apply the first-half permutation  $g$ ; (iv) XOR the backup  $T$  back into the capacity window (the EDM feed-forward); (v) apply the second-half permutation  $h$ . After all blocks have been absorbed, the digest is the leftmost  $d$  bits of the capacity window:

$$\text{Digest} = \text{left}_d(\text{proj}_S(X_{\text{final}}))$$

Therefore the feed-forward window (the write position of step (iv)), the chain-value window (the state-transfer position from one permutation call to the next), and the squeezing window (the digest-extraction position) are strictly aligned to the same bit set  $S$  — which is exactly the meaning of the word “aligned.” Algorithm 8 gives the complete CryptHash pseudocode consistent with the reference implementation.

**Algorithm 8** AFS-TrEDM-d(M) ( $d \in \{512, 768, 1024\}$ )

**Input:** bit string  $M \in \{0,1\}^*$ ,  $|M| \leq 2^{64} - 1$

**Output:** digest  $Z \in \{0,1\}^d$

$(r, c, n_r) \leftarrow \text{select by } d: (1024, 576, 12) / (768, 832, 20) / (512, 1088, 24)$

$s \leftarrow n_r / 2; \quad A \leftarrow \text{IV}_d$

$F \leftarrow \text{frame}_d(M, r, d, c)$

$B_0, \dots, B_{L-1} \leftarrow \text{parse}(F, r)$   
 of  $r$ , no tail block)

for  $k = 0$  to  $L - 1$  do

$X[0 \dots r-1] \leftarrow X[0 \dots r-1] \oplus B_k$

$T \leftarrow \text{proj}_S(X)$

$X \leftarrow g(X)$

$X[S] \leftarrow X[S] \oplus T$

$X \leftarrow h(X)$

end for

▷ initialize

▷ Algorithm 2

▷ Algorithm 1 ( $F$  is a multiple

▷ absorb

▷ back up the capacity window

▷ first  $s$  rounds

▷ EDM feed-forward

▷ last  $s$  rounds

```

Z ← leftd( projs(X) )
return Z

```

▷ squeeze

*Note: the reference implementation places the rate region in lanes  $A[0 \dots r/64 - 1]$  and the capacity window  $S$  in lanes  $A[r/64 \dots 24]$ ; digest extraction returns the leftmost  $d$  bits of  $S$ , i.e., lanes  $A[r/64 \dots r/64 + d/64 - 1]$ .*

## 1.11 The CryptHash Interface

Each implementation instance provides the CryptHash interface specified by the ICCS API\_CryptHash:

```

int CryptHash(int          digest_len_bits,
               const unsigned char *msg,
               unsigned long long msg_len_bits,
               unsigned char  *digest);

```

The interface wrapper accepts only the fixed digest length (512 / 768 / 1024) corresponding to its instance directory. It returns 0 on success; when digest is a null pointer, when msg is null but the length is nonzero, or when the digest length is not supported by the instance, it returns the corresponding nonzero error code. The digest is written out in the big-endian bit order of §1.2 (MSB-first partial byte).

## 1.12 Test Vectors

The Test\_Vectors directory contains the four classes of known-answer test (KAT) files mandated by ICCS §3.5: KAT\_2\_12, KAT\_2\_23, KAT\_2\_33, and KAT\_Loop, each covering the three instances. All KATs are driven by the ICCS-provided KAT\_CryptHash.c and generated with consistency checking by the reference/optimized implementations. Table 5 gives three short-message sample digests (taken from the reference implementation quick\_test) for quick verification: the empty message, the 3-byte “abc” (24 bits), and the 11-bit string 10101100001 (byte sequence AC 20, only the high 3 bits valid).

Table 5. Short-message sample digests of the three instances (hexadecimal, big-endian)

| Instance      | Input                                   | Digest Z                                                                                                                                                                                                     |
|---------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AFS-TrEDM-512 | Empty message                           | 38BDE78163BE3C7740F0BF5ABD657D5EABA7D8FED7DEA8E92B8D99074<br>5ACDD710517981E5ADB3BE95E88095A95530394F9C7B4D46A149A98A3<br>AFF4E0AEA001F7                                                                     |
| AFS-TrEDM-512 | "abc"                                   | 3C053A45577D4C777467F696284C840C41ADE7E97FE352AC30C489B67<br>7BB175D2568EDA7A77105400207C34561A69840D0ADB4174BE4A6A448<br>6806EA6A4E3308                                                                     |
| AFS-TrEDM-512 | 11-bit string<br>10101100001<br>(AC 20) | 2D9DE10E26588EE9B202F1276090C83E859415714724D3D4DDC35B19E<br>D313C9058C265128849E45AD814E339DEB187E32C3C89D69A94C7A0CA<br>61F9C77C8AA5F9                                                                     |
| AFS-TrEDM-768 | Empty message                           | 6B562A13E98F88A52EC1F7DF523E0A40D1388FB6BFA7D1BB2F08180CA<br>89690F9A0688B175BABC0B9995B99EB98B7830A3465C0242C82DE96E1<br>1212EC3ECF2C2F8D6EB171479FAA666D73C2802C49C6BFF1CAA04F224<br>34628E463F3D806749734 |
| AFS-TrEDM-768 | "abc"                                   | C127B916E603AE3209AA6BE020A7954C4E4CDBDB827F18285DD7E384478<br>D024C9FD5694B447A310CB6A6855EE74E9676631D4746F09C73489429C8<br>20F6CB8B1B0BE6C65466B667BC1E994636BA24E2120DE38D83991B77C96<br>56EDF3C82323B02 |
| AFS-TrEDM-768 | 11-bit string<br>10101100001<br>(AC 20) | 8607DD83E6A913E3931FE9673FD0E11226FB6790EDF482475EFA0019B7B<br>4668EEFC4964943ED2FB4432775052B730B1F3B84D4F8DF25FD891AA121<br>9BE329C8EA7B1723F41E2587468522EEB5C7F1893FD8FF38A7FE9B1666E<br>1103EC6D43DC345 |

| Instance       | Input                                   | Digest Z                                                                                                                                                                                                                                                                         |
|----------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AFS-TrEDM-1024 | Empty message                           | FDD4B35AAF2EA1361B9DC209F8E7B7AF35CD82F9899D06352451FCC65C8<br>008F642C1AEAAA159F455498220BD6CED824773C3077833F80EA7597B11<br>11ED6FF995264763993FDB624B044D66559E33D69E4EE71D0EFAC45E7BC<br>49FF76AD54B298E613F542687A33304C75139E9D52AC1A80042658AAE4<br>A6DBEA6EB7DBE1A484AE  |
| AFS-TrEDM-1024 | "abc"                                   | 89946087EFCECA389424F1419030553D5DC6D3F89A41F3E410363E7DC66<br>3AB75E56FFBB74F462630FB46BCF4300BEFD1BBFADA3DA9D270B13A45CC<br>F0561B07B95EA1EB5FF611F825C4D083850B6C54D0C0C1566791BBC8291<br>B493977C3D3211E9D0880873310D6A39A4108661450ABF3D054238BA77F<br>C88388388B9EBADA050F |
| AFS-TrEDM-1024 | 11-bit string<br>10101100001<br>(AC 20) | 0BE8C0339797459E352DA1C786E36152C57DA0F54EA960AC667D0C9B167<br>1EA2A4895E9907C4E6DC582DE8EFC475AFFBA20C14123AE8765039A972A<br>20A0BC6CD21D73599FE0C4FF3E356EE68E954674115A3530EDBB45DCDFC<br>8603F3FD33B168788CC0735849AE8F52D0C4AD86D4EDF88AC92C98F02F4<br>E96538AD9A0C249413A8 |

## 2 Design Rationale

This chapter explains the design rationale of AFS-TrEDM (S6 version) at three levels: the overall structure and the cryptographic properties of the components, parameter selection, and other considerations.

### 2.1 Overall Structure and Cryptographic Properties of the Components

#### 2.1.1 The Aligned-TrEDM Sponge Mode

The absorbing-squeezing mode of AFS-TrEDM starts from a structure-level security goal: in the idealized model of the dual public permutation  $(g, h)$ , it breaks the invertibility of the absorbing phase via a mid-section EDM feed-forward, and aligns the feed-forward window, the chain-value window, and the digest-output window all to the capacity window  $S$ . The standard sponge divides the  $b$ -bit state into a rate  $r$  and a capacity  $c$ , injects the message by XOR into  $r$ , and usually extracts the digest from the rate side; its preimage and second-preimage analysis is subject to a capacity  $c/2$ -type bottleneck. The goal of AFS-TrEDM is, while keeping the round cost of a single full permutation, to make the classical second-preimage resistance reach  $\min\{d, c - \log_2 \ell_{\max}\}$  rather than the  $c/2$ -type bottleneck (where  $\ell_{\max}$  is the maximum number of blocks absorbed by this interface under the permitted message length, i.e., the number of frame blocks of the longest message; the maximum message length of this interface is  $2^{64} - 1$  bits, so  $\ell_{\max} < 2^{56}$  for every instance. From  $c = d + 64$  we get  $c - \log_2 \ell_{\max} > d$ , so this bound is dominated by  $d$ , i.e., the classical second-preimage resistance is actually  $d$  bits; see §3.1 for its full definition).

Around this idea, two complementary structural ideas can be abstracted. The first is to introduce an internal feed-forward on the capacity side and also extract the digest from the capacity side, so that an output hit must pass through the internal-window constraint; the second is to split the full iterated permutation into two halves,  $g$  and  $h$ , and insert the EDM feed-forward between them, thereby obtaining a non-invertible absorbing core without increasing the number of full-permutation calls. Adopting either idea alone cannot fully cover the design goals of AFS-TrEDM: if the feed-forward window and the output window are misaligned, an invertible manipulation channel may still remain on the rate side; if the  $g/h$  split core is simply equated with a single permutation call, the relation-arity baseline of the quantum tightness proof is distorted.

References <sup>[1]</sup> and <sup>[2]</sup> provide two complementary clues, respectively: the former emphasizes internal feed-forward and internal squeezing on the capacity, so that an output hit must pass through the internal-window constraint; the latter emphasizes splitting the iterated permutation into  $g$  and  $h$  and inserting a DM/EDM feed-forward between the two halves to break the invertibility of the absorbing phase at low cost. At the same time, if the feed-forward window, the chain-value window, and the digest-output window are decoupled from one another, a manipulable channel may still remain. AFS-TrEDM summarizes this problem as the window-misalignment risk and adopts “alignment” as a mode constraint: the feed-forward, chain-value propagation, and final squeezing must all fall on the same capacity bit set  $S$ .

AFS-TrEDM is constructed accordingly: the absorbing phase uses a  $g/h$  split EDM feed-forward core with  $a=c$ ; the squeezing phase no longer extracts the digest from the rate side, but takes the leftmost  $d$  bits from the capacity alignment window  $S$ . This alignment brings three benefits: first, to manipulate the digest, an adversary must manipulate the  $S$  window, and  $S$  is exactly the chain-value window closed by the feed-forward at every round; second, the confluence path of a long-message second preimage must hit the  $c$ -bit capacity window in the target chain, with a complexity term of  $2^{(c - \log_2 \ell)}$  rather than  $2^{(c/2)}$ ; third, in the quantum setting one can separately account a  $d$ -bit digest search and a  $c$ -bit capacity multi-target search into the generic Grover/BHT complexity, rather than forcibly reducing the  $g/h$  split core to a single permutation call.

Figure 5 compares the four constructions in terms of feed-forward and squeezing positions, in the form of a

sponge dataflow; Table 6 compares item by item the common baseline of the standard sponge, the two improvement lines, and the treatment of AFS-TrEDM. Its last column shows that the step of “moving the squeezing window into the capacity subset  $S$  so that it aligns with the feed-forward and the chain value” is precisely the key to simultaneously resolving the main weaknesses of the two preceding lines.

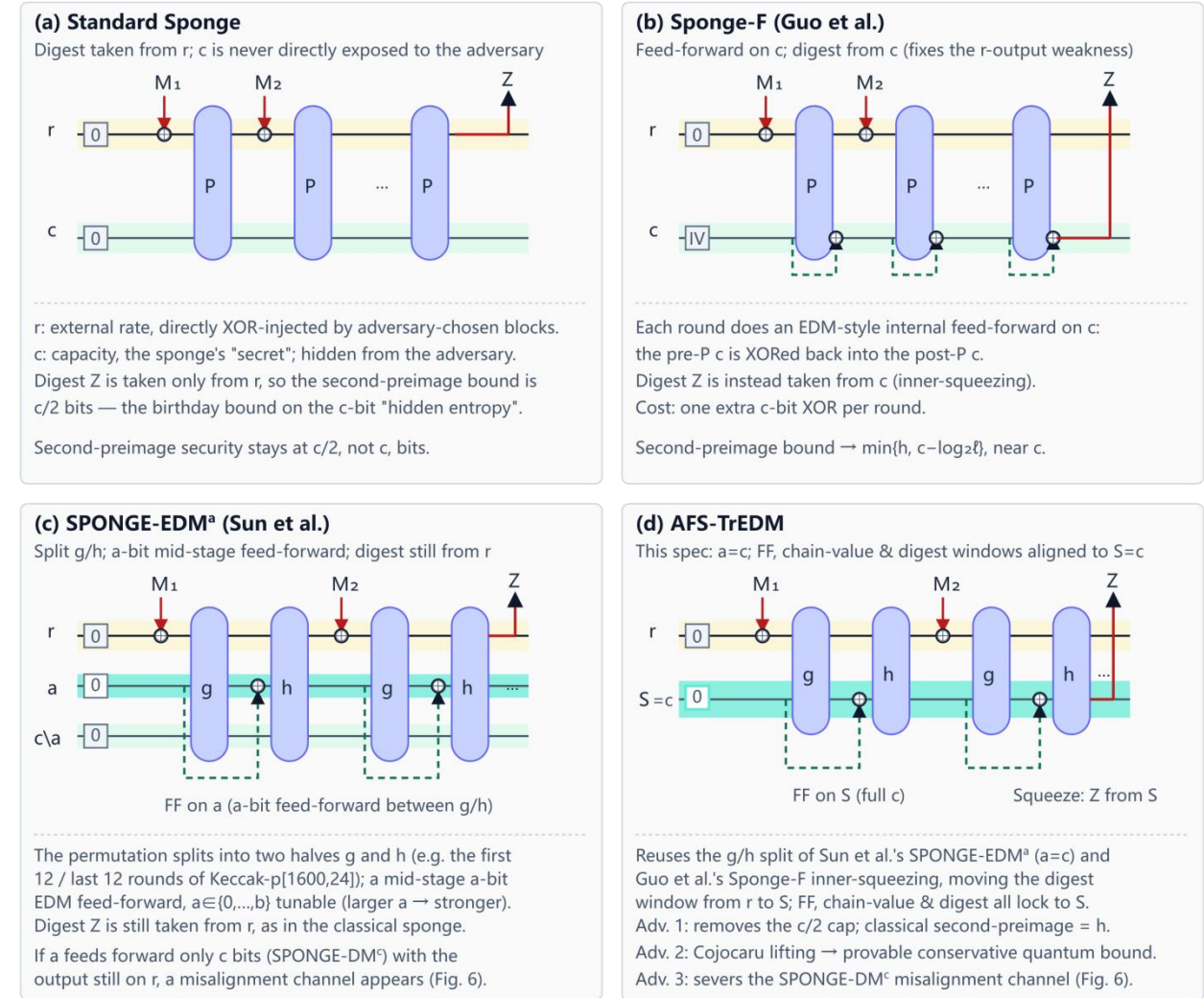


Fig. 5. Lineage comparison of the two feed-forward sponge improvement lines and AFS-TrEDM

Table 6. Comparison of the two feed-forward sponge improvement lines and AFS-TrEDM

| Construction                   | Advantages                                                                                                                       | Limitations                                                                                                                                                               | Fusion and improvement in AFS-TrEDM                                                                                                                                                                |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Standard sponge <sup>[4]</sup> | Simple structure, no feed-forward overhead;<br>indifferentiable under permutation idealization;<br>thoroughly analyzed in SHA-3. | (Second-)preimage is usually limited by a $c/2$ -type capacity bottleneck; the generic attack matching this bound exploits the backward computability of the permutation. | Serves as the common baseline; the EDM feed-forward on the capacity window $S$ breaks the invertibility of the absorbing phase and raises the second-preimage capacity term to $c - \log_2 \ell$ . |

| Construction                                                   | Advantages                                                                                                                                                                                                       | Limitations                                                                                                                                                                                             | Fusion and improvement in AFS-TrEDM                                                                                                                                                                                                                                |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Internal feed-forward + internal squeezing line <sup>[1]</sup> | The internal EDM feed-forward and internal squeezing on the capacity make an output hit pass through the capacity-side constraint; the security argument can be developed around the truncated compression core. | If each core evaluation must be equivalent to a single full-permutation call, it cannot directly cover the $g/h$ split core of AFS-TrEDM; the quantum tightness bound cannot be transferred as is.      | Adopts the internal-squeezing and window-alignment ideas; combined with the $g/h$ split implementation of AFS- $p$ -S6, and additionally gives a structure-level analysis based on the dual core.                                                                  |
| $g/h$ split EDM line <sup>[2]</sup>                            | $g/h$ can take the first and second halves of the same iterated permutation, adding no full-permutation calls and only one extra $a$ -bit XOR; suitable for engineering implementation and tiered-round design.  | If the feed-forward window and the output window are misaligned, a preimage-type attack channel may remain on the rate side; the quantum tightness reduction of the dual core needs separate treatment. | Retains the $g/h$ split with $a=c$ ; moves the squeezing window into $S$ , aligning it with the feed-forward and the chain value; the classical bound is derived at the AFS structure level, and the quantum strength is stated under the generic-attack baseline. |

In summary, the core improvement of AFS-TrEDM lies in the structural principle of window alignment, rather than a simple superposition of several existing constructions: it realizes a low-overhead EDM feed-forward via the  $g/h$  split, uses the capacity window  $S$  to simultaneously carry the feed-forward, the chain value, and the digest output, and uses  $c=d+64$  to absorb the  $\log_2 \ell$  loss in the long-message second preimage. The S6 version further replaces the linear layer of the underlying permutation with AFS-LMDS-1600-S6 (see §2.1.2) and adopts tiered round counts according to the digest length (see §2.1.4).

Figure 6 further contrasts the two cases of misaligned feedback and aligned feedback: in SPONGE-DM<sup>c</sup>, the feed-forward acts on the capacity but the digest is extracted from the rate, the two being decoupled and leaving a misalignment attack channel<sup>[2]</sup>; AFS-TrEDM aligns and locks the feed-forward, chain-value, and digest windows to the capacity subset  $S$ , cutting off this channel.

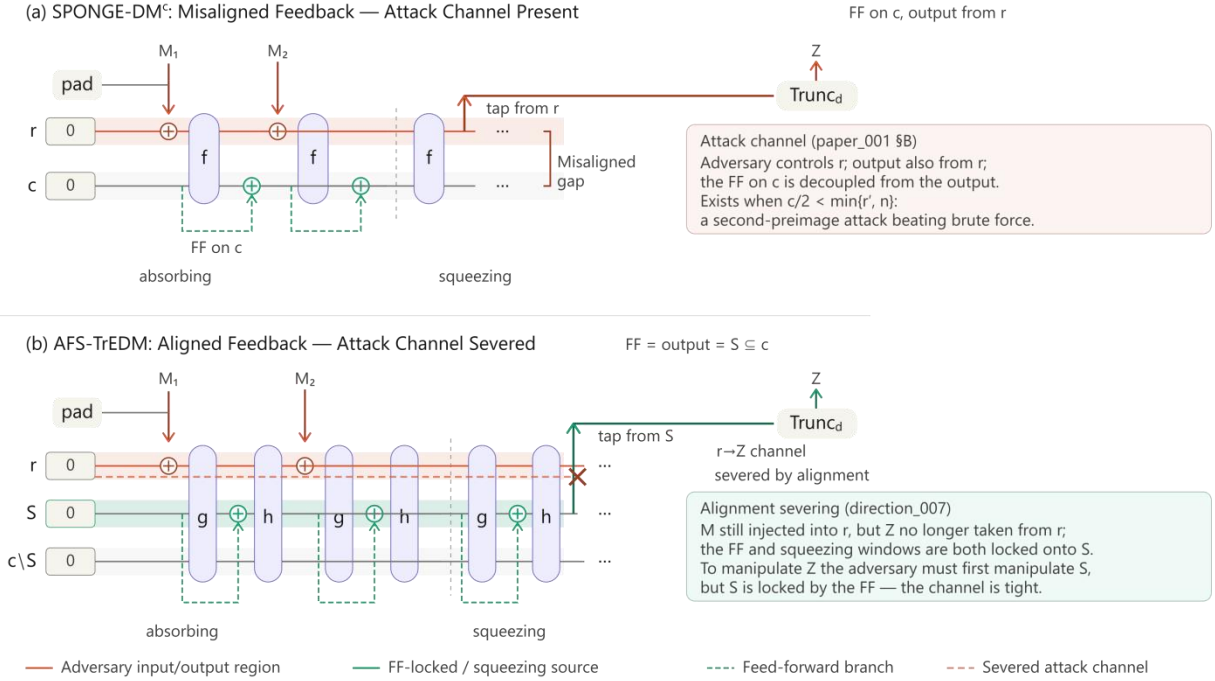


Fig. 6. Attack-channel comparison of misaligned feedback and aligned feedback

### 2.1.2 The Six-Direction Rotating Single-MDS Linear Diffusion Layer AFS-LMDS-1600-S6

The linear diffusion layer of AFS- $p$ -S6 does not follow the  $\theta + \rho + \pi$  combination of Keccak- $p$ <sup>[4]</sup>, nor does it use a per-row three-shift circular-right-rotation-XOR layer; instead, it adopts the six-direction rotating single-MDS structure of §1.6. Its design motivation comes from solver observations of the multi-round behavior of a single-layer MDS:

- The multi-round “mid-narrowing” of the single-MDS building block  $L_p$ .  $L_p$  realizes mixing with a single-layer column-direction MDS and removes row-direction mixing, so the single-layer diffusion is directionally anisotropic: there is MDS mixing in the column direction, while the row direction can only rely on  $\pi_{\text{AFS}}$  for redistribution in subsequent rounds. The solver quantitatively exposes this under the activity-preserving model — the 3-round active-S-box count for a fixed direction  $L_p$  is as low as 15, exhibiting a mid-valley of  $6 \rightarrow 4 \rightarrow 14$  round by round (only 4 active lanes in the middle round), and extrapolating over any 3 consecutive rounds corresponds to  $N_{12} \geq 60$ , below the design goal.
- A zero-cost countermeasure: direction rotation. The direct countermeasure is to add an orthogonal row-direction MDS layer in each round (the two-layer candidate  $L_C$ ), but  $L_C$  costs about 424 64-bit XORs per round, which is tight under the 24-round main performance budget, and its dense matrix makes the multi-round solver unable to decide within the time limit. Therefore this design does not add an expensive second MDS layer within a single round, but instead rotates the acting direction of the MDS among six projective directions across consecutive rounds: if any low-activity set fits direction  $dir$  at round  $i_r$ , it will be projected onto a different parallel line family to receive MDS mixing at rounds  $r+1$  and  $r+2$ , so the mid-narrowing cannot persist across directions.

Solver experiments confirm (see §3.3.1 and the design report): under the S6 schedule, the 3-round active-S-box lower bound is raised from  $L_p$ 's  $\leq 15$  (with a valley of 4) to  $\geq 12$  without a mid-valley. While keeping the cost of  $L_p$  (about 237 64-bit XORs per round) unchanged, S6 raises the multi-round active-S-box lower bound to  $N_{24}^D \geq 96$ ,  $N_{24}^L \geq 96$ , which is a scheme balancing cost and multi-round goals.  $\tau_{dir}$  and  $\tau_{dir}^{-1}$  are lane reorderings that can be

merged into compile-time wiring, adding no arithmetic instructions.

GF(32) Cauchy and lane-friendliness.  $M_{\text{col}}$  chooses a Cauchy MDS matrix over  $\text{GF}(32)=F_2[\alpha]/(\alpha^5+\alpha^2+1)$ : every square minor of a Cauchy matrix is nonzero, thus guaranteeing the MDS (optimal branch number) property; C contains only eight classes of small constants, so the hardware XOR-network cost is low. The whole layer takes the 25 64-bit lanes as its natural (lane-major) view, with no bit gather/scatter and no  $25 \times 25$  mask table, which is friendly to 64-bit word-level software and single-instruction-multiple-data (SIMD).  $\rho_{\text{AFS}}$  and  $\mu$  provide bit-level diffusion, and  $\pi_{\text{AFS}}$  provides a long-period lane permutation to break up regularity.

### 2.1.3 The Nonlinear Layer AFS-64 S-box

The AFS-64 family (Yan et al.<sup>[3]</sup>, obtained by a convergent satisfiability-modulo-theories (SMT)–mixed-integer-linear-programming (MILP) search) acts on the 64-bit lane with a pure ARX-Feistel structure (only 32-bit modular addition, circular rotation, and XOR, with no table lookups or bit-level operations). This specification replaces  $\chi$  with a single AFS-64 call, which brings:

1. Software-friendly: the 8-step ARX operations execute directly on 32-bit words, with no bit-level extraction and no table lookups, suiting the integer ALU and SIMD units of modern CPUs.
2. Hardware-friendly and constant-time: composed only of additions, XORs, and constant injections, with no table lookups and no data-dependent branches; it is both easy to implement in hardware and runs in a time that does not vary with the input data (i.e., constant-time, naturally resistant to timing side channels).
3. Higher algebraic degree: a single AFS-64 call produces an algebraic degree on the 64-bit lane far higher than that of  $\chi$ , significantly raising the threshold for algebraic attacks and integral/division-property distinguishers.
4. Tight differential/linear bounds: a single full call of AFS64\_t5\_k2 has a maximum expected differential probability  $\text{MEDCP} \leq 2^{-10}$  and a maximum expected linear correlation  $\text{MELCC} \leq 2^{-6}$  (see §2.2.4), comparable to or better than 4-round Alzette<sup>[13]</sup> at the same implementation cost.

### 2.1.4 Design Motivation for the Tiered Round Counts

The design goal of the tiered round counts of AFS-TrEDM is, while fully exploiting the word-level implementation advantages of the AFS-64 ARX large S-box, to make the three digest-length instances 512, 768, 1024 each reach the corresponding differential, linear, and structural security thresholds. Since the security goals of the three instances increase with the digest length, the underlying permutation does not use a single fixed round count, but configures the round count by tier: shorter-digest instances prioritize throughput advantage once they satisfy the security boundary, while longer-digest instances retain a higher round count to match a higher security strength. In other words, the tiered round count is not merely performance tuning, but a layered balance between security margin and implementation performance under the combined effect of a “strong single box, high rate, and high-diffusion linear layer.”

Round reduction must be supported by verification of the differential/linear active-S-box lower bound. There are two key constraints. First, the linear direction is the bottleneck for round determination: a single active S-box of AFS64\_t5\_k2 has a differential weight of  $w_D=10$  and a linear-correlation weight of  $w_L=6$ , so at the same number of active boxes, the security weight given by a linear trail is lower than that of a differential trail, and whether the linear direction meets the threshold determines whether a given tier’s round count is acceptable. Second, the wide-trail window bound of the S6 linear layer accumulates over three-round windows: it has been verified that the differential and linear active-S-box counts of any three-round window satisfy  $N_3^L \geq 12$ , so the weight lower bound of a single linear trail over R rounds is:  $w_L(R) \geq \lfloor R/3 \rfloor \times 12 \times 6$ .

For 12/20/24 rounds we obtain 4/6/8 three-round windows respectively, i.e., linear weight lower bounds of 288/432/576 bits; the corresponding differential weight lower bounds are 480/720/960 bits. Compared with the three tiers' collision-resistance thresholds  $d/2 = 256/384/512$ , all three tiers meet the requirement; compared further with the recommended capacity threshold  $c/2 = 288/416/544$ , the linear direction of all three tiers also reaches or exceeds this threshold. See § 3.3.1 for detailed active-box counts, three-round window verification, and differential/linear weights.

Accordingly, and combined with the uniform  $g/h$  split requirement, AFS-TrEDM selects  $6+6=12$  rounds for 512,  $10+10=20$  rounds for 768, and  $12+12=24$  rounds for 1024. This tiering makes instances with higher security goals use more rounds, while keeping shorter-digest instances from increasing rounds excessively once the threshold is met, thereby retaining the implementation-performance advantage of the AFS-64 ARX large S-box. It should be noted that the above wide-trail window bound mainly constrains a single differential/linear trail over the full permutation; the  $6+6$  split of the 512 tier additionally has a specific risk that this bound cannot isolate on its own: an attacker may enter  $g_6$  from an arbitrary internal state, seek cancellation at the capacity feed-forward boundary, and still satisfy the digest or  $S$  window constraint after passing through  $h_6$ . This split/feed-forward risk has been separately verified under the arbitrary-starting-state split model, with both the differential and linear directions reaching the 512-tier threshold of  $c/2 = 288$ ; see § 3.3.7 for the conclusion and the evidence boundary.

## 2.2 Parameter Selection

### 2.2.1 Digest Length $d \in \{512, 768, 1024\}$

The ICCS submission requirements<sup>[19]</sup> specify 512 bits and 1024 bits as mandatory instances. AFS-TrEDM-512 provides 256-bit collision resistance equivalent to SHA3-512; AFS-TrEDM-1024 provides 512-bit collision resistance and 1024-bit (second-)preimage resistance, matching the target profile of a “next-generation 1024-bit hash algorithm.” The 768-bit instance is optional, positioned at an intermediate tier between 512 and 1024 on the performance-security trade-off curve. By contrast, the current commercial-cryptography hash standard SM3<sup>[18]</sup> has a digest length of 256 bits; the AFS-TrEDM family targets next-generation digest-length and security-strength requirements of 512 bits and above.

### 2.2.2 Choice of Capacity $c$ ( $c = a = d + 64$ )

Under the aligned-TrEDM mode, the classical collision resistance is given by  $d/2$ ; the classical preimage resistance is dominated by the  $d$ -bit digest hit; the classical second-preimage resistance is jointly determined by the final-digest-hit term and the target-chain capacity-window-hit term, with parameter baseline  $\min\{d, c - \log_2 \ell_{\max}\}$ . Here  $\ell_{\max}$  is the maximum number of  $r$ -bit blocks produced by  $\text{frame}_d(M)$  within the permitted message-length range.

AFS-TrEDM supports  $|M| \leq 2^{64} - 1$  bits.  $\text{frame}_d$  appends a 1-bit, several 0-bits, and a fixed 128-bit metadata suffix after the message, so the maximum number of blocks satisfies

$$\ell_{\max}(r) = \lceil ((2^{64} - 1) + 1 + 128)/r \rceil = \lceil (2^{64} + 128)/r \rceil.$$

Since the minimum rate of the three-tier instances is 512 bits, uniformly  $\ell_{\max} < 2^{56}$ . When  $c=d+64$ ,

$$c - \log_2 \ell_{\max} > d + 8,$$

$$\min\{d, c - \log_2 \ell_{\max}\} = d.$$

Therefore, within the permitted maximum message-length range, the capacity loss  $\log_2 \ell_{\max}$  of the long-message second preimage is already absorbed by the extra 64 bits of capacity, and the classical second-preimage target of

all three tiers is  $d$  bits. See Table 7 for more precise block counts and capacity margins of the three tiers.

Table 7. *Maximum block count and classical second-preimage capacity margin*

| Instance       | $d$<br>(bits) | $r$<br>(bits) | $c=d+64$<br>(bits) | $\ell_{\max}$ upper<br>bound (blocks) | $c-\log_2 \ell_{\max}$ lower<br>bound (bits) | Classical<br>second-<br>preimage<br>target (bits) |
|----------------|---------------|---------------|--------------------|---------------------------------------|----------------------------------------------|---------------------------------------------------|
| AFS-TrEDM-512  | 512           | 1024          | 576                | $2^{54}+1 < 2^{55}$                   | $> 521$                                      | 512                                               |
| AFS-TrEDM-768  | 768           | 768           | 832                | $(2^{56}+2)/3 < 2^{55}$               | $> 777$                                      | 768                                               |
| AFS-TrEDM-1024 | 1024          | 512           | 1088               | $2^{55}+1 < 2^{56}$                   | $> 1032$                                     | 1024                                              |

The security-strength table in the main text uses the uniform conservative bound  $\ell_{\max} < 2^{56}$ , so it can be written that the three tiers satisfy  $c-\log_2 \ell_{\max} > 520, > 776, > 1032$ , respectively; the precise upper bounds in Table 7 only give larger margins. Thus the three sets of parameters  $b=1600=r+c$  are fully determined, and the feed-forward window  $a$  is all locked to  $c$ .

### 2.2.3 Tiered Round Counts and the $g/h$ Split

Each tier’s total round count is split evenly into  $g$  (the first  $s$  rounds) and  $h$  (the last  $s$  rounds,  $s=n_r/2$ ), inserting the EDM feed-forward in the middle of the full permutation without increasing the number of full-permutation calls. The tiers split as  $6+6/10+10/12+12$ . Under S6, this split brings two engineering benefits: first,  $g$  and  $h$  share the first and second halves of the same public permutation, the same round constants, and the same S6 direction schedule, so the implementation only needs one code loop run  $s$  times to realize both, avoiding the double footprint of a dual-permutation implementation; second, each half-permutation covers the full six-direction schedule several times, so the EDM feed-forward is sandwiched between two sub-permutations that diffuse the directions in a balanced way. The security sufficiency of the tiered round counts is given in §3.3.1: the wide-trail window bounds of the three tiers (linear directions of 288/432/576 bits for 512/768/1024) all satisfy their respective  $d/2$  thresholds and, by the window bound, reach the  $c/2$  recommended threshold.

### 2.2.4 Choice of the AFS64\_t5\_k2 Instance

AFS-64 forms one S-box round by cascading 8 Feistel-type basic steps. Yan et al. describe an ARX-Feistel chain with a “configuration code”  $A8 \in \{0,1\}^8$  (1 for a modular-addition step, 0 for a pure-XOR step), and estimate the differential/linear bounds one by one for valid configurations of Hamming weight  $\leq 4$  via a convergent SMT-MILP search, requiring the implementation cost not to exceed Alzette. This specification selects  $A8 = 11000011$  (with 4 modular additions, at the same cost as Alzette); this near-palindromic configuration is superior to Alzette’s 10101010 repeating structure on the tight differential/linear bounds. Given the configuration, the rotation sequence  $K8 = [17, 24, 1, 1, 16, 31, 24, 0]$  determines the rotation amounts of the 8 basic steps. A single full call of this instance (corresponding to 4 ARX-Feistel iterations) gives  $MEDCP \leq 2^{-10}$  and  $MELCC \leq 2^{-6}$ , denoted by the single-box differential weight  $w_D=10$  and the single-box linear-correlation weight  $w_L=6$ ; this value is taken from the rigorous search results of Yan et al., cross-checked for consistency with the original differential/linear trail files of the authors’ public repository, and verified by a fixed-key experiment with  $2^{28}$  random samples (deviation  $\leq 10^{-5}$ , insensitive to the round constant). The name t5\_k2 refers to the 2nd rotation candidate of type 5 in Yan et al.’s table, corresponding one by one with the hardcoded constants of the reference implementation.

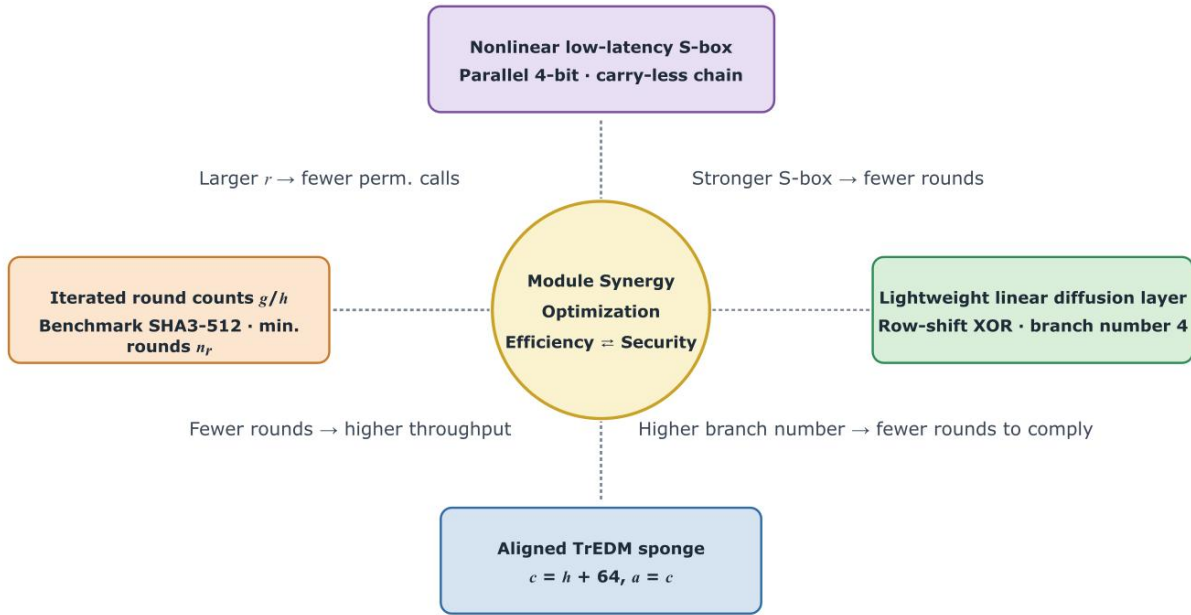
### 2.2.5 AFS-LMDS-1600-S6 Linear-Layer Parameters

The linear-layer parameters are chosen as follows: the  $GF(32)$  field is taken as  $F_2[\alpha]/(\alpha^5 + \alpha^2 + 1)$ , so that one bit-sliced symbol carries 64 parallel 5-bit field elements, and the primitive polynomial has low weight with a

cheap xtime; the Cauchy matrix guarantees the MDS property and contains only eight classes of small constants; the  $\rho_{\text{AFS}}$  offsets are spread over 0...63 so that the long-orbit cumulative bit rotation is coprime to 64;  $\pi_{\text{AFS}}$  is taken with a 1+24 long period to maximize the cross-lane mixing period; the  $1 + z^{17} + z^{32}$  of  $\mu$  is a three-term polynomial with an odd number of terms to guarantee invertibility and provide bit-level diffusion; the six-direction schedule  $[\infty, 0, 1, 2, 3, 4]$  covers all projective directions of the affine plane and rotates on a 6-round period. Each parameter is chosen under the joint constraints of the solver active-S-box lower bound, invertibility, and the cost budget (see the design report for details).

Figure 7 summarizes the synergistic relationship of the four modules — the nonlinear layer, the linear layer, the tiered round counts, and the aligned-TrEDM sponge — in balancing efficiency and security: a stronger S-box and a higher multi-round active-S-box lower bound can both reduce the number of rounds needed to meet the threshold, a larger rate can reduce the number of permutation calls, and fewer rounds bring higher throughput; the parameters are jointly optimized within this three-dimensional space.

### Efficiency–Security Balance from Module Synergy Optimization



Synergy principle: first ensure the 256-bit collision-resistance compliance threshold, then jointly optimize over the three dimensions: security margin, throughput, and area/latency.

Fig. 7. The efficiency-security balance of the four-module joint optimization

## 2.3 Other Considerations

The external interface is fully consistent with the ICCS-provided API\_CryptHash, using MSB-first partial-byte input with no additional auxiliary libraries. All public constants (round-lane constants, IV, Cauchy matrix,  $\rho$  offsets, direction schedule) are deterministically derived by public rules or explicitly listed, can be independently verified by any third party, and eliminate the possibility of a hidden backdoor.

### 2.3.1 Public, Deterministic Round-Lane Constants

The round-lane constants  $\text{RC32}(i_r, l)$  are deterministically derived by SplitMix64 using the documented public seed 0xB4C5F1D62E1738A9 and increment 0x9E3779B97F4A7C15 (formula in §1.7), introducing no randomness. SplitMix64 is a 64-bit mixing function (Steele, Lea, Flood, 2014). It has passed the evaluation of

statistical test suites such as TestU01 BigCrush, PractRand, and dieharder, with no recordable statistical bias over the  $2^{64}$  output range, and has been chosen by the standard libraries of several languages as the default mixing kernel of a splittable pseudorandom number generator (PRNG). It is chosen as the constant source because it simultaneously satisfies: sufficiently good avalanche and single-output independence, public deterministic single-seed reproducibility, and extremely low implementation overhead. The increment  $0x9E3779B97F4A7C15$  is the 64-bit fixed-point encoding of the golden ratio ( $\lfloor 2^{64}/\varphi \rfloor$ ), keeping the input seeds of adjacent  $\text{idx}$  far apart with no short-period collisions. The seed is a single literal publicly locked once at standardization submission; any third party can regenerate each  $\text{RC32}(i_r, l)$  from the formula and compare it one by one with the hardcoded values of the reference implementation.

### 2.3.2 Public, Deterministic Initial Values

The IV is a public 1600-bit constant, derived per instance by the documented FNV-1a64 + SplitMix64 chain (see §1.9). This provides mutually distinct IVs for the three instances, introduces no secret material, and at the same time binds the IV identity to the instance parameters  $(d, r, c, n_r)$  and the linear-layer identifier, achieving cross-instance and cross-version domain separation.

### 2.3.3 Big-Endian Bit Order and Implementation-Friendly Frame Encoding

Bytes are output MSB-first, partial bytes are aligned to the high bits of the byte, and the state bit order uses a big-endian layout, consistent with the ICCS API\_CryptHash convention. Frame encoding places a single 1-bit,  $0^j$ , and the 128-bit metadata suffix together, so the last frame is exactly a multiple of  $r$ ; frame encoding is parsed only once at the API boundary. When the message is byte-aligned, the 1-bit is exactly a single  $0x80$  byte and the 128-bit suffix is byte-aligned, facilitating an implementation fast path.

## 3 Security Claims and Analysis

This chapter is organized into four parts: §3.1 gives the security claims of the three instances under classical and quantum computing conditions; §3.2 gives the theoretical security, i.e., the argument line for the overall structure and component security; §3.3 gives the concrete security, covering collision resistance, preimage resistance, second-preimage resistance, as well as differential, linear, rotational, algebraic, integral, meet-in-the-middle, and other cryptanalysis<sup>[20]</sup>; §3.4 gives the statistical-randomness test results.

Levels of the security claims. This chapter strictly distinguishes three classes of statements in its formulation: (i) conclusions that can be mathematically proven, such as the indistinguishability reduction direction of the aligned sponge and the lower bound of the wide-trail active-S-box count; (ii) conclusions decided as UNSAT by the solver on reduced instances with reproducible archiving completed, such as the six-phase window verification; (iii) heuristic arguments or analysis items still in progress, such as differential/linear hull refinement and rotational-attack refinement. For class (ii), the precise statement of this specification is “solver UNSAT decision + reproducible archive,” not “rigorous mathematical proof”; this chapter does not conflate solver decisions with mathematical proofs.

### 3.1 Security Claims

The security strength of the AFS-TrEDM family is claimed separately under classical and quantum computing conditions. For an instance with digest length  $d$ , the classical collision-attack target is  $d/2$  bits, the classical preimage-attack target is  $d$  bits, and the classical second-preimage-attack target is  $\min\{d, c - \log_2 \ell_{\max}\} = d$  bits. Under quantum computing conditions, the security strength is given by the known generic quantum-attack complexity: collision attack under the  $d/3$  baseline, and preimage and second-preimage attacks under the  $d/2$  baseline. Table 8 gives the claimed strengths of the three instances.

Table 8. Security claims of the three AFS-TrEDM instances (bit security strength)

| Security property                | Analysis baseline                               | 512   | 768 | 1024  |
|----------------------------------|-------------------------------------------------|-------|-----|-------|
| Classical collision attack       | $d/2$                                           | 256   | 384 | 512   |
| Classical preimage attack        | $d$                                             | 512   | 768 | 1024  |
| Classical second-preimage attack | $\min\{d, c - \log_2 \ell_{\max}\} = d$         | 512   | 768 | 1024  |
| Quantum collision attack         | $d/3$                                           | 170.7 | 256 | 341.3 |
| Quantum preimage attack          | $d/2$                                           | 256   | 384 | 512   |
| Quantum second-preimage attack   | $\min\{d/2, (c - \log_2 \ell_{\max})/2\} = d/2$ | 256   | 384 | 512   |

The classical collision resistance comes from the birthday bound of the  $d$ -bit digest: the main term of a collision among any  $q$  candidate digests is  $q^2/2^{(d+1)}$ , so reaching a constant success probability requires about  $2^{(d/2)}$  attempts. The classical preimage resistance comes from a direct hit on a specified  $d$ -bit digest, with main term  $q/2^d$ , so reaching a constant success probability requires about  $2^d$  attempts. The classical second-preimage resistance must also consider the multi-target hit on the  $\ell$  capacity windows along the target message chain, but  $c=d+64$  and  $\ell_{\max} < 2^{56}$ , so the complexity of the capacity path is no less than  $2^{(d+8)}$ , and is ultimately still dominated by the  $2^d$  digest-hit term.

The quantum collision resistance adopts the generic quantum collision-search baseline  $2^{(d/3)}$  most favorable to the attacker. The quantum preimage attack adopts the Grover search baseline  $2^{(d/2)}$ . The quantum second-preimage attack considers both the final-digest hit and the multi-target hit on the target-chain capacity window, with complexity  $\min\{2^{(d/2)}, 2^{((c - \log_2 \ell_{\max})/2)}\}$ ; from  $(c - \log_2 \ell_{\max})/2 > d/2 + 4$ , the capacity multi-target path is still no less than the direct digest-search path.

The above quantum strength is a claimed baseline given jointly by the generic quantum-attack complexity and the AFS structure parameters. This specification does not claim that the  $g/h$  split TrEDM core has been directly proven by a tight quantum reduction to a single permutation call; whether the actual permutation AFS- $p$ -S6 has a structural shortcut below the generic complexity is supported by the differential, linear, rotational, algebraic, and meet-in-the-middle analyses of §3.3 and the split-boundary verification of Appendix D.

### 3.2 Theoretical Security

This section gives the structure-level security-strength derivation of AFS-TrEDM under classical computing conditions, and explains the claimed baseline of the security strength under quantum computing conditions. The object of analysis is the  $g/h$  split aligned-TrEDM core defined in §1.10; the concrete permutation deviation of the underlying AFS- $p$ -S6 is not eliminated by a random model in this section, but is explicitly retained as the  $\epsilon_{\text{str}}$  term and supported by the instance-level cryptanalysis of §3.3 and Appendix D. The classical analysis of this section is at two levels: §3.2.1–§3.2.4 give the three classes of claimed bounds by direct probability/union bounds; §3.2.5–§3.2.7 give structural and relation-level reductions for the same set of bounds, precisely reducing the claimed bounds to the keyless-CI<sup>[1]</sup> relation of the underlying permutation, thereby connecting with the permutation-layer evidence of §3.3 and Appendix D.

#### 3.2.1 Model and Basic Notation

Let  $b=1600$ ,  $b=r+c$ ,  $a=c$ , and  $S=\text{state}[r...1599]$  be the capacity alignment window. Denote by  $\pi_S(X)$  the  $c$ -bit projection of the state  $X$  onto the window  $S$ , and by  $\iota_S$  the linear embedding that embeds  $c$  bits back into the  $S$  window with the remaining positions filled with 0. The single absorbing core of AFS-TrEDM is defined as

$$C_{g,h}(X) = h(g(X) \oplus \iota_S(\pi_S(X))).$$

To describe both the digest hit and the capacity-window hit, define

$$D_d(X) = L_d(\pi_S(C_{g,h}(X))), \quad D_c(X) = \pi_S(C_{g,h}(X)).$$

For the  $r$ -bit blocks  $B_1, \dots, B_L$  parsed from  $\text{frame}_d(M)$ , the chain value is updated as follows:

$$W_i = X_{i-1} \oplus (B_i \parallel 0^c), \quad X_i = C_{g,h}(W_i), \quad H(M) = D_d(W_L).$$

The structure-level idealization baseline is as follows: when no repeated core input,  $h$  input repetition, capacity-window alignment, or concrete-permutation structural bad event is triggered,  $D_d$  behaves as a uniform  $d$ -bit value for a new input, and  $D_c$  behaves as a uniform  $c$ -bit value for a new input. The deviation of the actual AFS- $p$ -S6 from this baseline is uniformly denoted by  $\varepsilon_{\text{str}}$ , and is constrained in the concrete-security chapter by differential, linear, rotational, algebraic, meet-in-the-middle, and split-boundary evidence.

### 3.2.2 Classical Collision-Resistance Derivation

Suppose a classical adversary induces a total of  $\sigma$  absorbing-core evaluations. If two distinct messages produce the same  $d$ -bit digest, then either a final-digest birthday collision occurs, or a capacity-window alignment, an IV-capacity hit, or another internal-chain alignment bad event occurs in the absorbing chain. By the union bound,

$$\text{Adv}_{\text{coll,cl}}(\sigma) \leq \sigma(\sigma - 1)/2^{d+1} + K_{\text{coll}} \cdot \sigma^2/2^c + \varepsilon_{\text{str,coll}}(\sigma).$$

The first term is the birthday term on the  $d$ -bit digest; the second term covers the internal-alignment events on the  $c$ -bit capacity window, where  $K_{\text{coll}}$  is a small constant related to the encoding and chain localization; the third term is the concrete-permutation structural deviation. Since  $c=d+64$ , the capacity term is 64 bits lower than the digest birthday term and does not determine the dominant complexity. Therefore the classical collision resistance is  $d/2$  bits, namely 256, 384, and 512 bits for the three tiers.

### 3.2.3 Classical Preimage-Resistance Derivation

Given a target digest  $Z \in \{0,1\}^d$ , each new candidate final core input  $W$  hits  $D_d(W)=Z$  with probability  $2^{-d}$ . If the adversary tries to bypass the final-digest search via an intermediate link, it must hit some  $c$ -bit capacity-window constraint. For  $\sigma$  candidates and at most  $\ell_{\text{max}}$  connectable positions,

$$\text{Adv}_{\text{pre,cl}}(\sigma) \leq \sigma/2^d + K_{\text{pre}} \cdot \sigma \ell_{\text{max}}/2^c + \varepsilon_{\text{str,pre}}(\sigma).$$

By §2.2.2,  $\ell_{\text{max}} < 2^{56}$  and  $c=d+64$ , so

$$\sigma \ell_{\text{max}}/2^c < \sigma/2^{d+8}.$$

The capacity-connection path is strictly weaker than the direct-digest-hit path, so reaching a constant success probability requires about  $2^d$  attempts. The classical preimage resistance is  $d$  bits, namely 512, 768, and 1024 bits for the three tiers.

### 3.2.4 Classical Second-Preimage-Resistance Derivation

Suppose the target message  $M$  contains  $\ell$  blocks after  $\text{frame}_d$ , with target chain  $X_0, X_1, \dots, X_\ell$ . The success paths for a second-preimage adversary to output  $M' \neq M$  with  $H(M')=H(M)$  fall into two classes: first, a direct hit on the final  $d$ -bit digest; second, confluence from a different chain value to some internal capacity window of the target chain. By the union bound,

$$\text{Adv}_{\text{spr,cl}}(\sigma, \ell) \leq \sigma/2^d + K_{\text{spr}} \cdot \sigma \ell/2^c + K_{\text{bad}} \cdot \ell^2/2^c + \varepsilon_{\text{str,spr}}(\sigma, \ell).$$

The first term corresponds to a direct digest hit; the second term corresponds to a multi-target hit among the  $\ell$  target capacity windows; the third term covers bad events such as internal capacity repetition of the target chain itself and IV-capacity hits;  $\varepsilon_{\text{str}}^{\text{spr}}$  is the concrete-permutation deviation. Ignoring low-order bad events, the second-preimage complexity exponent is

$$S_{\text{spr,cl}} = \min\{d, c - \log_2 \ell\}.$$

$$\min\{d, c - \log_2 \ell_{\max}\} = d.$$

Therefore, the classical second-preimage resistance is  $d$  bits, namely 512, 768, and 1024 bits for the three tiers.

### 3.2.5 Five-Layer Structural Reduction Line

§3.2.1–§3.2.4 have directly given the three classes of claimed bounds under the ideal-permutation model. This section (§3.2.5–§3.2.7) does not set up new security claims, but gives structural and relation-level reductions for the same set of bounds: it first decomposes the claimed bounds into layer-by-layer verifiable structural sources (a five-layer reduction), and then formalizes them as the keyless-CI of the underlying permutation with respect to three simple relations, precisely reducing “hash-layer security” to “verifiable properties of the underlying permutation.” This serves three purposes: as an independent corroboration of the direct derivation; to provide a precise reduction target for the differential/linear analysis of §3.3 and the solver evidence of Appendix D; and to clarify the structural source of preimage-type resistance (capacity and feed-forward, not single-trail weight). The reduction in this section is a classical reduction; its quantum baseline I (see §3.3.6 and Appendix E.14) is a conservative secondary result, and the quantum security strength is claimed by the generic-attack complexity, not by baseline I.

Layer 1 — the ideal-permutation model. Model the underlying permutation AFS- $p$ -S6[1600,  $n_r$ ] as an ideal random permutation  $P$ . Component-level evidence (i.e., that  $P$  is statistically close to a random permutation, with no exploitable differential/linear/rotational/algebraic structure) is provided by the concrete cryptanalysis of §3.3; this layer abstracts these component-level conclusions into the premise that “ $P$  behaves as a good permutation.”

Layer 2 — standard-sponge indistinguishability as a capacity baseline. The aligned-TrEDM sponge inherits the absorbing/squeezing skeleton of the standard sponge and inserts the EDM feed-forward inside the aligned capacity window. The standard-sponge indistinguishability result provides the relevant capacity barrier and a design baseline under the ideal-permutation model, but it is not used here as a direct proof of  $d$ -bit preimage or second-preimage resistance for the aligned-TrEDM core. The  $g/h$  split, the capacity-side feed-forward, and capacity-side squeezing are accounted for explicitly by the core-hit and capacity-window analyses of §3.2.2–§3.2.4 and Appendix E.9–E.12.

Layer 3 — deriving the claimed classical bounds from core-hit and capacity-window counting. The claimed bounds are therefore not obtained by applying a standard-sponge indistinguishability theorem as a black box. Collision resistance is bounded by the  $d$ -bit digest birthday term together with retained  $c$ -bit capacity-bad-event terms. Preimage and second-preimage resistance are derived from the single-core hit probability and the multi-target capacity-window counting of §3.2.2–§3.2.4 and Appendix E.9–E.12: each fresh final digest hit costs  $2^d$ , and each chain-confluence path must hit a  $c$ -bit  $S$  window, with at most  $\ell_{\max}$  targets. With  $c=d+64$  and  $\ell_{\max} < 2^{56}$ , the capacity paths remain above the  $d$ -bit digest-search term. Thus the dominant classical strengths are  $d/2$  for collision and  $d$  for preimage and second preimage.

Layer 4 — truncation. The digest is given by  $\text{digest} = \text{left}_d(\text{proj}_S(X_{\text{final}}))$ , where  $|S| = c = d + 64$ , i.e., the  $c$  bits of the capacity window are truncated to  $d$  bits<sup>[10]</sup>. The extra 64 bits are a truncation reserve margin, ensuring that truncation does not reduce any security strength below  $d$ ; at the same time, truncation breaks direct observation of the full capacity state, working with Layer 5 to block the state-recovery path.

Layer 5 — EDM feed-forward closure. The aligned EDM feed-forward couples the output of the absorbing permutation  $g$  with the chain value on the window  $S$  (followed by the squeezing permutation  $h$  after  $X[S] \oplus = \text{proj}_S(g(X))$ ); this feed-forward makes it impossible for an attacker to reduce “finding a preimage” to inverting a single permutation: to recover the internal state across the capacity barrier, the feed-forward coupling constraint must be satisfied simultaneously, so the  $c$ -bit capacity cannot be bypassed. This layer is the structural

source of the preimage and second-preimage resistance reaching  $d$  bits — it does not depend on the weight of a single differential trail, but on the structure of the capacity and the feed-forward.

A key distinction. The standard-sponge capacity barrier, the  $d$ -bit digest birthday term, and the single-trail differential/linear evidence of §3.3.1 are independent checks for collision-type behavior. The  $d$ -bit preimage and second-preimage claims, however, rest on the aligned-TrEDM structural analysis—single-core digest hits, capacity-window multi-target hits, and EDM feed-forward closure—not on the standard-sponge indistinguishability bound alone and not on a single differential trail weight. Treating either indistinguishability or trail weight as a direct proof of  $d$ -bit preimage resistance would be an over-claim; this specification avoids that interpretation.

### 3.2.6 Relation-Level keyless-CI Reduction

The division of labor between differential analysis and structural reduction. The reduction of §3.2.5 shows that the permutation-layer evidence on which the three classes of resistance depend is not the same: the collision resistance (threshold  $d/2$ ) is given by the differential/linear active-S-box weight bound of §3.3.1, addressing the collision-type problem of “finding two inputs for a given output difference”; the preimage and second-preimage resistance (thresholds  $d$  and  $\min\{d, c - \log_2 \ell\}$ ) do not depend on differential weight, but are determined by the structural reduction given in this section. In other words, active-S-box analysis only supports collision resistance, and preimage-type resistance comes from the structure of the aligned-TrEDM sponge rather than trail weight — consistent with the discussion of Layers 3–5 in §3.2.5. This section formalizes the latter class of resistance as a keyless-CI reduction of the underlying composite permutation with respect to three simple relations, giving relation-level proofs of the corresponding bounds of §3.2.1–§3.2.4; see Appendix E for the complete theorems and proofs. This reduction is a classical reduction; for the quantum baseline see §3.3.6 and Appendix E.14.

The reduction paradigm. The reduction adopts the ideal-permutation-based keyless correlation-intractability framework<sup>[1]</sup>; it does not assume that “no one can find” a bad input, but constructively builds, from any hash adversary, an adversary that attacks the underlying permutation relation, thereby upper-bounding the hash security by the keyless-CI advantage of the underlying relation plus an error term. The underlying object is taken to be the composite permutation  $P=h \circ g$ .

The single absorbing compression core and three relations. Denote the truncated Encrypted Davies–Meyer core of a single absorbing as

$$C(X)=d\left(g(X)\oplus\iota_S(\pi_S(X))\right), \quad U=C(X)$$

where  $\pi_S$  takes the capacity window  $S$ ,  $\iota_S$  is its inverse embedding, and the digest  $\text{Digest}=\text{left}_d(\pi_S(U_{\text{final}}))$ . Three simple relations are defined on  $P$ : the collision relation  $R_{\text{co}}$  (two core outputs equal on the digest or the internal window with distinct inputs), the preimage relation  $R_{\text{pre}}(Z)$  ( $\text{left}_d(\pi_S(U))=Z$ ), and the second-preimage relation  $R_{\text{spr}}(X^*, \cdot)$  (the core output internally hits the target  $X^*$  with distinct inputs). Verifying  $R_{\text{co}}$  requires two core evaluations (relation arity  $k=2$ ),  $R_{\text{pre}}$  and  $R_{\text{spr}}$  each require one ( $k=1$ ), and all three relation arities satisfy  $k \leq 2$ , providing a relation-level basis for the classical keyless-CI reduction (for the quantum baseline see §3.3.6 and Appendix E.14).

Theorem 1 (collision resistance). For any collision adversary making  $\sigma$  core queries, there exists an adversary attacking  $R_{\text{co}}$  and  $R_{\text{pre}}(\text{IV})$  such that  $\text{Adv}_H^{\text{coll}} \leq \text{Adv}_P^{R_{\text{co}}-\text{ci}} + \text{Adv}_P^{R_{\text{pre}}(\text{IV})-\text{ci}}$ ; instantiated under the random permutation model (RPM, with  $g$  and  $h$  modeled as independent random permutations) as

$$\text{Adv}_H^{\text{coll}} \leq \frac{(\sigma+2\ell_{\text{max}})^2}{2^d} + \frac{2(\sigma+2\ell_{\text{max}})}{2^d} = O(\sigma^2/2^d)$$

reaching  $\min\{c/2, d/2\} = d/2$  ( $c = d + 64 \geq d$ ). The suffix-freeness of the frame encoding in §3.2.8 eliminates only the pure frame-suffix or length-extension branch. It does not make the capacity-window term vanish: capacity-window equality, IV-capacity hits, internal capacity repetitions, and rate-compensated chain-alignment events remain valid  $c$ -bit bad events and are retained in the  $K_{\text{coll}} \cdot \sigma^2/2^c$ -type term. Therefore the collision bound is the digest-birthday term plus the retained capacity-bad-event terms, rather than a reduction only to the digest-collision relation.

Theorem 2 (preimage resistance). For any preimage adversary with challenge image  $Z \in \{0,1\}^d$  and  $\sigma$  queries, there exists an adversary attacking  $R_{\text{pre}}(Z)$  such that  $\text{Adv}_H^{\text{pre}} \leq \text{Adv}_p^{R_{\text{pre}}(Z)-ci}$ ; under RPM

$$\text{Adv}_H^{\text{pre}} \leq \frac{2(\sigma + \ell_{\max})}{2^d} = O(\sigma/2^d)$$

reaching  $\min\{d, c\} = d$ . Taking the digest from the capacity window  $S$  (fully internal squeezing) prevents the preimage strength from degenerating to  $c/2$ .

Theorem 3 (second-preimage resistance). For any second-preimage adversary with a sampleable message distribution and  $\sigma$  queries, there exists an adversary attacking  $R_{\text{co}}$ ,  $R_{\text{pre}}(\text{IV})$ , and the multi-target  $R_{\text{spr}}$  such that  $\text{Adv}_H^{\text{spr}} \leq \text{Adv}_p^{R_{\text{co}}-ci} + \text{Adv}_p^{R_{\text{pre}}(\text{IV})-ci} + \text{Adv}_p^{mt-R_{\text{spr}}-ci}$ ; under RPM

$$\text{Adv}_H^{\text{spr}} \leq \frac{\ell^2}{2^d} + \frac{2\ell}{2^c} + \frac{2\ell(\sigma + 2\ell_{\max})}{2^c} + \frac{2(\sigma + 2\ell_{\max})}{2^d}$$

Substituting  $c = d + 64$  and  $\ell_{\max} < 2^{56}$  (message length  $\leq 2^{64}$  bits, rate  $r \geq 512$ ), we obtain  $c - \log_2 \ell_{\max} > d + 8$ , hence reaching  $\min\{d, c - \log_2 \ell_{\max}\} = d$ . Here window alignment raises the reachability of the third path of the second preimage from  $c/2$  to  $c - \log_2 \ell_{\max}$ , guaranteed by the alignment lemma of §3.2.7.

The common core of the reductions: chain-divergence localization. The common core of the above three reductions is to trace the winning event of the hash adversary back along the absorbing chain, localizing it either to a digest relation hit or to a capacity-window bad event at some core evaluation. Lemma 3-1 (chain-divergence localization, proof in Appendix E) shows that, for any colliding pair, the comparison of the two block chains yields either a final digest collision, an intermediate  $S$ -window equality, an IV-capacity hit, or an internal capacity/chain-alignment event. Suffix-free frame encoding rules out only the pure length-extension suffix branch; it does not rule out capacity alignment obtained through rate-side compensation. The preimage and second-preimage reductions take the relation hit at the last core evaluation and at the multi-target capacity confluence, respectively.

The above results hold under the ideal-permutation model, which is the standard premise for the provable-security analysis of sponge-type hashes (such as SHA-3); the component-level premise that the underlying permutation AFS- $p$ -S6 behaves as a good permutation is supported by the differential, linear, and indistinguishability analyses of §3.3.

### 3.2.7 The Alignment Lemma and Cutting Off the Misalignment Channel

The misalignment channel. In an Encrypted-DM sponge where the feed-forward acts on the capacity but the digest is squeezed from the rate, the feed-forward window and the squeezing window are decoupled: the adversary can manipulate the digest on the rate side while leaving the capacity-side feed-forward free, and when  $c/2 < \min\{r', d\}$  this forms a second-preimage channel better than exhaustive search, which worsens as the number of feed-forward bits decreases<sup>[2]</sup>. AFS-TrEDM takes the number of feed-forward bits  $a = c$  and squeezes the digest from the capacity window  $S$ ; the following alignment lemma proves that this channel is structurally cut off.

Lemma 3-2 (alignment cutoff). Under the aligned baseline ( $a = c$ , feed-forward window = chain-value window = squeezing window =  $S$ , Digest =  $\text{left}_d(\pi_S(X_{\text{last}}))$ ), the third path of the second-preimage reduction cannot be realized via “manipulating the rate-side digest while keeping the interior free”: any  $M'$  that makes  $\text{Digest}(M') = \text{Digest}(M)$  must make  $\text{left}_d(\pi_S(X'_{\text{last}})) = \text{left}_d(\pi_S(X_{\text{last}}))$ , i.e., it must produce a value internally in the capacity equal to the target, and that value is already locked by the mid-section EDM feed-forward. Therefore the success probability of the third path is at most the multi-target-hit probability of hitting the interior of some target in a target chain of length  $\leq \ell$

$$\text{Adv}_P^{\text{mt-R}_{\text{spr-ci}}} \leq (\ell - 1)\delta_1 + \delta_2, \delta_1 \leq \frac{2(\sigma + 2\ell_{\max})}{2^c}, \delta_2 \leq \frac{2(\sigma + 2\ell_{\max})}{2^d}$$

corresponding to a second-preimage strength of  $\min\{d, c - \log_2 \ell\}$  (rather than  $c/2$ ). See Appendix E for the complete three-step proof (digest is interior, no rate-side action channel; interior locked by feed-forward; multi-target counting).

Corollary 3-1 (misalignment-channel cutoff). Under the aligned baseline, the “rate-manipulates-digest” misalignment attack channel possessed by constructions that only feed forward the capacity but squeeze from the rate does not exist: manipulating the digest first requires manipulating  $S$ , and  $S$  is locked by the feed-forward. This gives the  $c$  (rather than  $c/2$ ) baseline contributed by alignment in Theorem 3, which is the security value of the AFS-TrEDM design of moving the squeezing window into the capacity subset  $S$ .

### 3.2.8 Frame Encoding, Capacity Alignment, and Scope of Applicability

$\text{frame}_d$  of §1.4 binds the 64-bit message length, the digest length  $d$ , the rate  $r$ , the capacity  $c$ , and the version number at the end of the message. For a fixed instance, a given valid frame allows  $|M|$  to be recovered from the tail metadata and the unique 1-bit padding to be located, so  $\text{frame}_d$  is injective. If the block sequence of one valid frame is a proper suffix of another valid frame, then their tail metadata must be equal, whence the original message length equals the total frame length, contradicting the proper-suffix relation; therefore  $\text{frame}_d$  has block-level suffix-freeness. See Proposition E-1 in Appendix E for the rigorous relation-level proof of this property.

Suffix-freeness excludes pure length-extension suffix branches, but does not remove capacity-window IV hits, internal capacity repetitions, or chain-alignment bad events; these events are already retained as  $2^{-c}$ -magnitude terms in the security bounds of §3.2.2 to §3.2.4. Since  $c = d + 64$ , none of these capacity terms changes the dominant security strength given in Table 8.

This section gives a structure-level idealized derivation: it explains how the AFS-TrEDM parameters set the classical collision, preimage, and second-preimage targets to  $d/2$ ,  $d$ , and  $d$ , respectively. Whether a concrete AFS- $p$ -S6 instance has a structural shortcut below the above targets is supported jointly by the wide-trail active-S-box analysis, rotational analysis, and meet-in-the-middle/algebraic/integral analysis of §3.3, and the dedicated verification of the 512-tier 6+6 split in Appendix D.

For the maximum message length permitted by AFS-TrEDM, we have  $\ell \leq \ell_{\max} < 2^{56}$  and  $c = d + 64$ , so  $c - \log_2 \ell_{\max} > d + 8$ . Hence

## 3.3 Concrete Security

This section gives the strength of each instance in resisting the main cryptanalysis under classical and quantum conditions. §3.3.1 covers differential and linear analysis (including wide-trail active-S-box counting and six-phase window verification); §3.3.2 covers length-extension resistance; §3.3.3 covers free-start/semi-free-start/near collisions; §3.3.4 covers meet-in-the-middle, algebraic, and integral analysis; §3.3.5 covers rotational analysis; §3.3.6 covers quantum analysis.

### 3.3.1 Differential and Linear Analysis

The differential/linear security of AFS-TrEDM adopts the wide-trail strategy: the strong diffusion of the AFS-LMDS-1600-S6 linear layer accumulates the number of active S-boxes over multiple rounds, so that the weight of any non-trivial differential trail or linear trail grows linearly with the number of rounds. The differential weight of a single active AFS-64 S-box is denoted by  $w_D$ , and the linear-correlation weight by  $w_L$ ; their values are determined by the maximum expected differential probability  $\text{MEDCP} \leq 2^{-10}$  and the maximum expected linear correlation  $\text{MELCC} \leq 2^{-6}$  of a single AFS-64 box (see §2.2.4)

$$w_D=10, w_L=6 \quad (\text{differential/linear weight per active S-box}) \quad (\text{differential/linear weight per active S-box})$$

The key lower bounds of the active-S-box count (the differential and linear directions are tracked separately and must not be mixed) are:

Table 9. Active-S-box count lower bounds and multi-round wide-trail combinations

| Quantity             | Differential                    | Linear                          | Note                                                                |
|----------------------|---------------------------------|---------------------------------|---------------------------------------------------------------------|
| $N_3$ (3 rounds)     | $\geq 12$                       | $\geq 12$                       | Three-round window lower bound (window verification)                |
| Window combination   | $\lfloor R/3 \rfloor \cdot N_3$ | $\lfloor R/3 \rfloor \cdot N_3$ | R rounds contain $\lfloor R/3 \rfloor$ disjoint three-round windows |
| $N_{24}$ (24 rounds) | $\geq 96$                       | $\geq 96$                       | $8 \times N_3$ , corresponding to the 1024 instance                 |

Directional separation. The three-round directional lower bounds  $N_3^D \geq 12$ ,  $N_3^L \geq 12$ ; the twenty-four-round linear-direction lower bound  $N_{24}^L \geq 96$ . The active-S-box counts of the differential and linear directions are mutually independent, and this specification gives them separately and tracks their lower bounds independently.

Single-trail weights of the tiered instances. By the window combination  $N_R \geq \lfloor R/3 \rfloor \cdot 12$ , multiplied by the per-box weight, the weight lower bounds of the optimal single trail of the three instances are obtained, as in Table 10.

Table 10. Weight lower bounds of the optimal single differential/linear trail of each instance and security thresholds

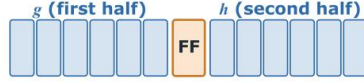
| Instance | Rounds R | Number of three-round windows | Active boxes $N_R$ | Differential weight $w_D$ (bits) | Linear weight $w_L$ (bits) | Threshold $d/2 \cdot c/2$ |
|----------|----------|-------------------------------|--------------------|----------------------------------|----------------------------|---------------------------|
| 512      | 12       | 4                             | $\geq 48$          | $\geq 480$                       | $\geq 288$                 | $256 \cdot 288$           |
| 768      | 20       | 6                             | $\geq 72$          | $\geq 720$                       | $\geq 432$                 | $384 \cdot 416$           |
| 1024     | 24       | 8                             | $\geq 96$          | $\geq 960$                       | $\geq 576$                 | $512 \cdot 544$           |

The differential weight  $w_D$  of all three instances simultaneously exceeds the collision threshold  $d/2$  and the stronger threshold  $c/2$  set by this specification; the linear weight  $w_L$  also reaches or exceeds  $c/2$  (for the 512 instance,  $L = 288$  exactly equals  $c/2 = 288$ , and a separate rate-only-split direct proof  $N_{\text{split}}^L \geq 48$  gives  $L \geq 48 \cdot 6 = 288 = c/2$ ). Since the single-trail weight addresses the collision-type problem of “finding two inputs for a given output difference,” Table 10 directly supports the collision resistance but does not directly give the preimage resistance; the latter is provided by the structural reduction of §3.2.

The upper part of Figure 8 gives the tiered round counts, the  $g/h$  split, and the feed-forward position of the three instances, and the lower part compares, with a bar chart, the linear-direction single-trail weight lower bound of each instance against the  $d/2$  and  $c/2$  thresholds, intuitively confirming that the linear direction is the bottleneck and that the linear bounds of all three tiers meet the threshold.

### Tiered round counts and collision-resistance differential/linear margins

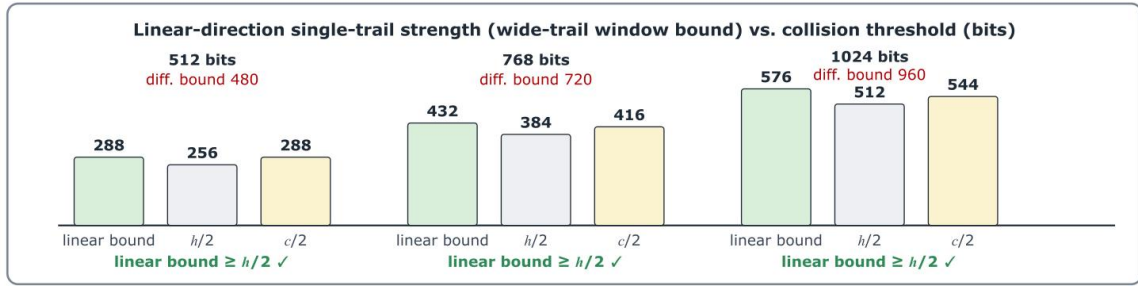
**AFS-TrEDM-512**  
 $r=1024$   $c=576$   
**6 + 6 = 12 rounds**



**AFS-TrEDM-768**  
 $r=768$   $c=832$   
**10 + 10 = 20 rounds**



**AFS-TrEDM-1024**  
 $r=512$   $c=1088$   
**12 + 12 = 24 rounds**



Linear direction is the bottleneck ( $w_L=6 < w_D=10$ ): 512 uses 6+6, 768 uses 10+10, 1024 uses 12+12;  
all three linear bounds  $\geq h/2$  and reach the recommended  $c/2$  threshold

Fig. 8. Tiered round counts and collision-resistance differential/linear margins

Six-phase three-round window verification. The window combination  $N_R \geq \lfloor R/3 \rfloor \cdot N_3$  relies on a premise: every three-round window has at least  $N_3$  active S-boxes, and this lower bound does not degrade with the phase of the window in the round schedule. The direction schedule of AFS-LMDS-1600-S6,  $\text{DIR\_SCHEDULE} = [\infty, 0, 1, 2, 3, 4]$ , has a period of 6, so a three-round window has 6 phases in total. To eliminate the phase-equivalence assumption, this algorithm constructs, for all 6 phases and the two directions (differential and linear), a total of  $6 \times 2 = 12$  targets, a satisfiability instance of “fewer than  $N_3$  active S-boxes within the window” for each, and decides them with CryptoMiniSat 5.14.5 (with Gauss-Jordan elimination enabled); all 12 targets return UNSAT. The DRAT proofs of two of the single-round bit-level UNSAT results are additionally verified independently by drat-trim. Thus the window lower bound holds for all phases, and the wide-trail combination  $N_{24} \geq 8 \times N_3 = 96$  becomes an unconditional conclusion (in the sense of UNSAT decision + reproducible archive). See Appendix B for the verification checklist, solver configuration, and actual running times.

Note on the conservativeness of the active-S-box model. The above MILP/satisfiability model based on active-S-box counting is an “activity-preserving” model and is systematically conservative for multi-round weights: the model only decides whether an S-box is active and does not exploit the concrete probability of each differential transition in the actual AFS-64 differential distribution table, so the true optimal-trail weight can only be higher. Incorporating the actual differential/linear distribution of the S-box into the model to tighten the bound is listed as a subsequent verification item.

### 3.3.2 Length-Extension-Attack Resistance

Length-extension attacks target Merkle–Damgård-type iterated hashes<sup>[9]</sup> (such as SHA-2<sup>[17]</sup>, where knowing  $H(M)$  allows  $H(M \parallel \text{pad} \parallel M')$  to be computed without knowing  $M$ ). AFS-TrEDM is a permutation-based sponge construction, and the digest is given by truncating the capacity window  $S$ : the externally output  $d$  bits are only part of the  $c = d + 64$ -bit capacity state, and an attacker cannot recover the full internal state from the  $d$ -bit digest

and therefore cannot continue the computation. The natural resistance of the sponge construction to length extension is covered by the indistinguishability conclusion of §3.2 under the ideal-permutation model. Therefore AFS-TrEDM directly resists length-extension attacks without an outer wrapper such as HMAC.

### 3.3.3 Free-Start, Semi-Free-Start, and Near Collisions

Free-start collisions (free-start, allowing the attacker to freely choose the initial chain value/IV) and semi-free-start collisions (semi-free-start, allowing one of them to be chosen) are usually easier than standard collisions, but in a permutation-based sponge, the IV is a public constant that does not enter the controllable input of the compression function, and the evolution of the chain value is entirely determined by the public permutation and the absorbed message blocks. The active-S-box lower bound of §3.3.1 applies equally to internal collisions of “any non-trivial input difference mapping to zero difference on the capacity window,” independent of whether the starting chain value is chosen by the attacker, so free-start/semi-free-start collisions are also constrained by the weight bound  $D \geq d/2$  (and  $\geq c/2$ ). For near-collisions (near-collision, requiring the output to differ in only a few bits), the strong-diffusion linear layer makes the input difference corresponding to any low-weight output difference still pass through a sufficient number of active S-boxes, and no near-collision path below the generic bound has been found. The collision/preimage/second-preimage security of any fixed output subset is inherited from the property that the digest is a uniform truncation of the capacity window.

### 3.3.4 Meet-in-the-Middle, Algebraic, and Integral Analysis

Meet-in-the-middle (MITM) and state recovery. The EDM feed-forward closure (Layer 5 of §3.2) breaks the meet-in-the-middle structure of splitting the permutation at its midpoint and solving the two ends independently: the feed-forward coupling constraint makes the two ends no longer independent, so the attacker cannot use MITM to reduce the preimage complexity below the  $c$ -bit capacity barrier.

Algebraic degree and algebraic attacks. AFS-64 is an ARX-type S-box, where modular-addition carries raise the degree bit by bit, and within a few rounds the algebraic degree approaches the upper limit of the state size; together with the global diffusion of AFS-LMDS-1600-S6, the entire permutation reaches a high algebraic degree in far fewer than  $n_r$  rounds, making it difficult to construct a low-degree polynomial approximation or a higher-order differential distinguisher. A precise upper-bound estimate of the algebraic degree of the ARX structure is listed as a subsequent analysis item.

Integral and zero-sum distinguishers. Integral/zero-sum attacks exploit a low algebraic degree to construct a balance structure. Strong diffusion and ARX degree-raising jointly suppress the propagation of a controllable integral property over multiple rounds; the tiered round counts (12 rounds for 512, 24 rounds for 1024) leave a margin relative to the number of rounds needed to reach full diffusion and a high algebraic degree. A quantitative upper bound on the integral/algebraic degree is listed as a subsequent analysis item.

### 3.3.5 Rotational Analysis

The rotational attack is a key analysis surface under the ARX structure of this algorithm. Rotational cryptanalysis exploits the property that modular addition, XOR, and circular shift in an ARX construction approximately commute with “circular shift by the same amount.” The nonlinear layer AFS-64 of AFS-TrEDM uses 64-bit/32-bit ROTR and modular addition, and rotational invariance is deliberately broken in the design; its resistance to rotational analysis is jointly guaranteed by three structural mechanisms. First, each round injects a 32-bit round constant RC32, derived from SplitMix64, distinct and non-all-zero for every round and every lane (§1.7 has verified that the 300/500/600 constants are all nonzero and distinct); since RC32 itself has no rotational symmetry, the rotational relation cannot be preserved after round-constant injection, and the rotational characteristic therefore cannot propagate stably across rounds. Second, the  $\rho_{\text{AFS}}$  of AFS-LMDS-1600-S6 uses distinct circular-shift amounts for each lane (see Table 3), and the lane reordering of  $\pi_{\text{AFS}}$  further breaks up the structure of

rotation by a uniform amount, so a global “uniform rotation” is no longer an approximate symmetric transformation of the state. Third, the IV and frame encoding introduce a fixed structure that does not commute with rotation, eliminating the rotational symmetry on the input side. Combining the above mechanisms, the round constants, the per-lane shift amounts, and the frame structure all lack rotational symmetry, and the rotational invariance on which rotational analysis depends is systematically broken, which constitutes the structural basis for this algorithm’s resistance to rotational analysis.

### 3.3.6 Quantum Security

This section gives the security-strength analysis of AFS-TrEDM under quantum computing conditions. The quantum security strength is reckoned by the known generic quantum-attack complexity; it does not claim a security strength exceeding the generic bound, nor does it reduce the  $g/h$  split TrEDM core to a single call of a single public permutation to apply a tight quantum reduction.

Collision attack. For a  $d$ -bit digest, the generic quantum collision-search complexity is about  $2^{(d/3)}$ . AFS-TrEDM also has a  $c$ -bit capacity window, but  $c=d+64$ , so the quantum collision complexity on the capacity window is  $2^{(c/3)}=2^{(d/3+64/3)}$ , higher than the  $d$ -bit digest collision complexity. Therefore the quantum collision resistance is dominated by the digest length:

$$S_{\text{coll},q} = \min\{d/3, c/3\} = d/3.$$

Corresponding to AFS-TrEDM-512, AFS-TrEDM-768, and AFS-TrEDM-1024, the quantum collision resistance is about 170.7, 256, and 341.3 bits, respectively.

Preimage attack. Given a target digest  $Z \in \{0,1\}^d$ , the generic quantum method is Grover search, which reduces a classical  $2^d$ -level unstructured search to about  $2^{(d/2)}$ . Any path that tries to bypass the final-digest search via capacity-window connection needs to hit the  $c$ -bit  $S$  window, and even with square-root speedup requires about  $2^{(c/2)}=2^{(d/2+32)}$  quantum queries. Therefore the quantum preimage resistance is

$$S_{\text{pre},q} = d/2.$$

namely 256, 384, and 512 bits for the three tiers.

Second-preimage attack. Suppose the target message contains  $\ell \leq \ell_{\max}$  blocks after frame <sub>$d$</sub> . A quantum adversary can directly search for a second preimage of the final  $d$ -bit digest, with complexity about  $2^{(d/2)}$ ; or search for candidates that can connect to an internal capacity window of the target chain, and since the number of target windows is at most  $\ell$ , its complexity is about  $2^{((c-\log_2 \ell)/2)}$ . Hence

$$S_{\text{spr},q} = \min\{d/2, (c - \log_2 \ell_{\max})/2\}.$$

From  $\ell_{\max} < 2^{56}$  and  $c=d+64$ , we get  $(c-\log_2 \ell_{\max})/2 > d/2+4$ , so the capacity multi-target path is no less than the direct digest-search path, and ultimately

$$S_{\text{spr},q} = d/2.$$

The quantum second-preimage resistance of the three tiers is 256, 384, and 512 bits, respectively.

Structural quantum-shortcut screening. AFS- $p$ -S6 injects public, position-dependent round-lane constants every round, the S6 linear layer breaks fixed-direction low-dimensional invariants by six-direction rotation, frame <sub>$d$</sub>  binds the length and instance identity, and the aligned TrEDM locks the feed-forward, chain value, and output to the same capacity window  $S$ . Based on the verification of §3.3.1 to §3.3.5 and Appendix D, no stable structure exploitable by a Simon period, a hidden shift, a rotational period, a linear structure, or a meet-in-the-middle quantum speedup has been found so far. This judgment is a combination of structural screening and the generic-attack baseline, and is not equivalent to a rigorous lower-bound proof against all quantum attacks.

Two technical paths to quantum-provable security. Besides giving the strength claim by the generic quantum-attack complexity, there are currently two main technical paths to a provable security bound under quantum computing conditions. The first is generic lifting, represented by the quantum lifting theorem of Cojocaru et al.<sup>[21]</sup>: it mechanically lifts the relation-level security bound established by the classical keyless-CI reduction (see §3.2.6 and Appendix E) to the quantum random-permutation model via a quantum lifting factor over a random permutation, and is applicable to any keyless-CI reduction. The second is the compressed oracle and quantum indistinguishability, based on Zhandry’s compressed-oracle technique<sup>[22]</sup> and generalized to a compressed permutation oracle: it directly proves that the construction is indistinguishable from a random oracle against a quantum adversary, so that generic bounds such as Grover and BHT hold directly as provable conclusions.

The provable bound of the generic-lifting path (baseline I). The lifting factor of the quantum lifting theorem grows with the number of underlying-permutation query points involved in the lifted relation. A single evaluation of the absorbing core  $C_{g,h}$  of AFS-TrEDM passes through two sub-permutation stages  $g$  and  $h$ ; when counting points correctly by the dual oracle  $(g,h)$ , the preimage relation contains 2 query points and the collision and second-preimage relations contain 4 query points, so the relation arity under the classical core-evaluation baseline doubles accordingly. Substituting this number of query points into the quantum lifting theorem (see Theorem E-7 in Appendix E for the complete derivation), the resulting provable quantum security strength is baseline I: preimage  $d/4$ , collision and second-preimage  $d/8$  bits. This baseline is strictly weaker than the quantum security strength given above by the generic-attack complexity.

The status and obstacles of the compressed-oracle path. The second path is the main route to a tight quantum-provable bound: once the construction is proven quantum-indistinguishable from a random oracle, the above generic bounds hold as provable conclusions, and the conservativeness of baseline I is thereby eliminated. But this path has a significantly higher technical threshold and has long been an open problem — for quite a long time, little was known about the security of sponge constructions against quantum adversaries, including even basic properties such as preimage and collision resistance beyond a single round, the main obstacle being the lack of a satisfactory analogue of quantum lazy sampling for permutations. It was not until 2025 that Alagic, Carolan, Majenz, and Tokat proved that the standard sponge is quantum-indistinguishable from a random oracle in the invertible-permutation case<sup>[23]</sup>, raising the post-quantum security of the underlying sponge of NIST SHA-3 to a level matching its classical security. However, this conclusion is for the standard sponge and does not yet cover the aligned-TrEDM construction with EDM feed-forward and squeezing from the capacity subset  $S$ ; in fact, the feed-forward mechanism by which AFS-TrEDM raises the classical second-preimage resistance from  $c/2$  to  $c$  itself goes beyond the scope covered by standard-sponge indistinguishability. In other words, the feed-forward property that makes this construction classically stronger is the very same structural reason that makes it deviate from clean quantum indistinguishability and loosens the generic lifting factor.

Combining the above two paths, this specification makes the following choices for the quantum security strength. First, the strength is claimed by the known generic quantum-attack complexity (collision  $2^{d/3}$ , preimage and second-preimage  $2^{d/2}$ ), with no claim of strength exceeding the generic bound. Second, baseline I of generic lifting is not used as the claimed bound: baseline I is weaker than the generic bound, and claiming by it would underestimate the actual strength; this baseline, as a quantum corollary of the keyless-CI framework, is listed together with the complete derivation in Appendix E (Theorem E-7) and retained as a weaker secondary provable result. Third, the  $g/h$  split TrEDM core is not reduced to a single call of a single public permutation  $P$  to apply the tight quantum-lifting conclusion of a single-permutation relation — this reduction is inconsistent with the actual structure of the core (a single evaluation contains two permutation queries  $g$  and  $h$ ) and would yield an invalid tight bound. Fourth, the correct route to establishing a tight quantum-provable bound for this construction is the second-class direct analysis of the compressed-permutation oracle, which does not yet cover this construction and is left as a direction for future research.

Applicable baseline. Since a single core evaluation of the absorbing core  $C_{g,h}(X)=h(g(X) \oplus \iota_s(\pi_s(X)))$  of AFS-TrEDM involves two sub-permutation stages  $g$  and  $h$ , this document does not reduce it to a single query of a single public permutation  $P$ . Therefore the quantum security strength of this section is given by the generic quantum-attack complexity and AFS structural screening, and it is not claimed that the existing single-permutation-relation lifting theorem can directly give a tight quantum-provable bound for the  $g/h$  split core.

### 3.3.7 Front-Half-Round and Aligned-Split Risk Verification

The wide-trail window bound of §3.3.1 gives the active-S-box lower bound for a single trail over the full 12 rounds. But the 6+6 split introduces a specific risk that this window bound does not isolate on its own: an attacker may keep a low active-S-box cost in the first 6 rounds ( $g_6$ ), exploit the capacity-window feed-forward of the aligned TrEDM to form a cancellation at the split boundary  $V[S]=g_6(X)[S] \oplus P_S(X)$ , and still satisfy the digest/ $S$  window constraint after passing through  $h_6$ , thereby constructing a preimage-type or relation-type shortcut below the 512 design threshold. To screen for this risk, on top of the three-round window verification of §3.3.1/Appendix B ( $C0, N_3 \geq 12$ ), this specification adds a set of dedicated verifications for the 6+6 tier of 512.

Its core verification item C1, in an arbitrary-starting-state split model, explicitly models the capacity backup  $T=P_S(X)$  and the feed-forward coupling  $V[S]=U[S] \oplus T$ , whose initial perturbation may take any internal state and is not limited to the rate part. This verification gives two sets of lower bounds: the differential direction reaches  $N_D \geq 29$  by direct Gauss-Jordan decision, the linear direction reaches  $N_L \geq 43$  by the same decision, and further reaches  $N_L \geq 48$  with the help of the C0-dependent wide-trail window bound. Thus both the differential and linear directions reach the  $c/2$  threshold (288) of the 512 instance. This generalizes the split bound of §3.3.1, which was only for the rate split, to any internal state, thereby directly covering the low-activity channel of  $g_6 + \text{feed-forward} + h_6$ . On top of C1, within the claimed scope, the low-activity semi-free/free-start and keyless-CI two-path relations (C2/C3), the low-activity RX/RDL skeleton and the universal rotational equivariance of  $s = 1 \dots 63$  (C4), and bounded cube/integral and subthreshold hull (C5/C6) are each handled by subset reduction or counterexample, corroborating the analyses of §3.3.3, §3.3.5, and §3.3.4. See Appendix D (Table D-1) for each verification, conclusion, and proof type.

Each verification item uses the conservative single-box weights of §3.3.1 ( $w_D=10, w_L=6$ ) and decides with CryptoMiniSat 5.14.5 (with Gauss-Jordan enabled) under a fail-closed rule: solver timeout/unknown, placeholder models, and witnesses not independently reviewed are never promoted to PASS. Therefore, for the 6+6 tier of 512, under the claimed active-S-box model, no low-activity structural channel that could form an effective preimage-type shortcut in the first 6 rounds or at the aligned-split/feed-forward boundary has been found.

According to the statement grading of §3.0, the conclusion of this section belongs to class (ii) (solver UNSAT decision + reproducible archive), not class (i) mathematical proof, and its precise statement is scoped machine-checked evidence rather than a global security proof. The following are explicitly not claimed: a universal bit-based division-property/integral proof, the aggregation of full near-threshold differential hulls and linear trilinear-correlation hulls, and the extension of the arbitrary-starting-state split differential  $c/2$  bound to 768/1024 (the wide-trail window bounds of the latter two tiers are still given by §3.3.1). See Appendix D for the complete model, thresholds, stage-by-stage verification, and claim ledger.

## 3.4 Statistical-Randomness Test Results

AFS-TrEDM-S6 has been evaluated on three standard statistical-test families, covering the three instances and multiple input families; see Appendix C, the “AFS-TrEDM-S6 Randomness Test Report,” of this specification for the complete test report. The results are summarized in Table 11.

Table 11. Summary of statistical-randomness test results

| Test family               | Scale                                                                                          | Conclusion        | Note                                           |
|---------------------------|------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------|
| <b>NIST SP 800-22 STS</b> | 6 groups (instance $\times$ input family), 1000 sequences $\times$ 1M bits each, 188 sub-tests | <b>Pass</b>       | /                                              |
| <b>TestU01 SmallCrush</b> | 6-group combination                                                                            | <b>All passed</b> | 0 rewinds                                      |
| <b>SAC / BIC</b>          | 27 configurations                                                                              | <b>All passed</b> | flip rate $\approx 0.5$ , no out-of-range bits |

Combining the results of the three test families, no distinguishable statistical bias between the AFS-TrEDM-S6 output and a random sequence has been found; the flip rates of the strict avalanche criterion (SAC) and the bit-independence criterion (BIC) are concentrated at 0.5, indicating that a single-bit input perturbation can diffuse sufficiently over the output.

## 4 Performance Evaluation

This chapter is organized into six parts: §4.1 gives a performance analysis from the design-rationale level; §4.2 gives the detailed configuration of the test platform; §4.3 gives the computational efficiency of the three instances under four software implementations (four tables: cycles per byte cpb, cycles per hash, throughput, and latency); §4.4 gives a comparison with SHA3-512 on the same device; §4.5 summarizes the speedup ratios; §4.6 explains reproducibility.

### 4.1 Design-Level Performance Analysis

AFS-TrEDM is designed for both software throughput and hardware parallelism; its main performance characteristics stem from the following three points.

(1) A lane-friendly linear layer. AFS-LMDS-1600-S6 takes the 64-bit lane as its basic operation unit; the column mixing  $M_{\text{col}}$ , the  $\rho_{\text{AFS}}$  circular shift, the  $\pi_{\text{AFS}}$  lane reordering, and the  $\mu$  linear mixing are all lane-level ROTR64 and XOR; the six-direction rotation  $\tau$  is merely a rewiring of lanes with no arithmetic overhead. This makes the state fit exactly into general-purpose registers or SIMD lanes, avoiding bit-by-bit operations.

(2) An ARX nonlinear layer. The AFS-64 S-box consists only of 32-bit modular addition, XOR, and circular shift, with no table lookups and no S-box storage; it is naturally resistant to cache-timing side channels and can be directly vectorized on scalar, AVX2, and AVX512.

(3) Batch-friendly and tiered round counts. The permutation has no data-dependent branches and is suitable for multi-buffer parallelism: on AVX512, 16 messages can be processed simultaneously to fill the pipeline and eliminate bubbles. The tiered round counts (12 rounds for 512, 20 rounds for 768, 24 rounds for 1024) allow shorter-digest instances to obtain higher throughput while keeping a security margin (§3.3.1) — the round count of the 512 instance is only half that of the 1024 instance, with throughput increasing accordingly.

Figure 9 shows the organization of batch multi-buffer parallelism: 16 independent messages share the AVX-512 zmm register file in a lane-major manner, the AFS- $p$ -S6 round function executes on them in SIMD lockstep, and the high rate and block-level parallelism jointly amortize the per-unit-data permutation-call cost.

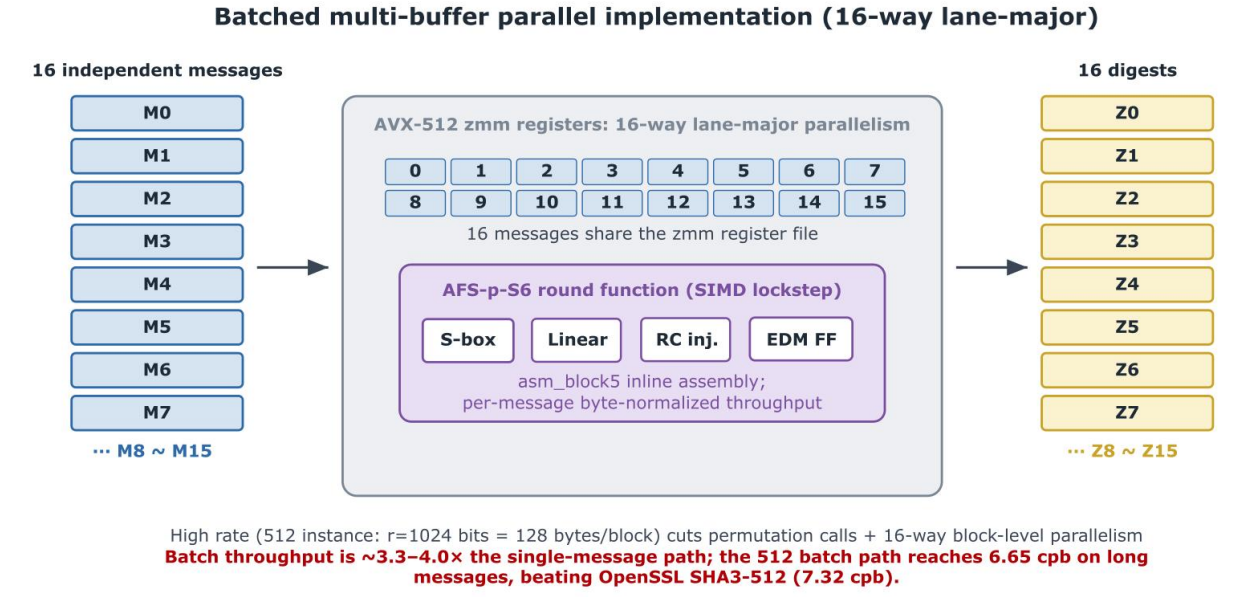


Fig. 9. Batch multi-buffer (16-way lane-major) parallel implementation

## 4.2 Test Platform

The software performance is measured on the following 64-bit PC platform.

Table 12. Software-implementation performance test platform configuration

|                                          |                                                                 |
|------------------------------------------|-----------------------------------------------------------------|
| <b>Processor</b>                         | Intel Xeon w7-2495X (24 physical cores / 48 logical cores)      |
| <b>Instruction set</b>                   | x86-64, supporting AVX2 and AVX-512                             |
| <b>Operating system</b>                  | Ubuntu 22.04 LTS                                                |
| <b>Reference-implementation language</b> | ISO C (no dependence on a specific platform instruction set)    |
| <b>Optimized implementation</b>          | opt64 (scalar 64-bit), AVX2 hybrid, Batch16 AVX512 multi-buffer |
| <b>Comparison baseline</b>               | SHA3-512: XKCP readable、XKCP CompactFIPS202、OpenSSL EVP         |

Note: cpb, cycles per hash, throughput, and latency are measured at multiple message lengths from 64B to 1MB; throughput is in decimal bytes per second; the 1KB/64KB/1MB labels correspond to the binary byte counts 1024 / 65536 / 1048576.

## 4.3 Software-Implementation Computational Efficiency

Table 13 gives the cycles per byte (cycles per byte, cpb, smaller is faster), Table 14 gives the cycles per hash (cycles per hash, smaller is faster), Table 15 gives the throughput (MB/s, larger is faster), and Table 16 gives the latency (milliseconds, smaller is faster). All four tables cover six message lengths: 64B, 192B, 1KB, 1536B, 64KB, and 1MB. AFS-TrEDM-S6 has four implementations in total: the reference implementation (Reference), the scalar optimization (opt64), the AVX2 hybrid (AVX2 hybrid, i.e., AVX2 AFS-64 S-box + opt64 S6 linear layer), and the AVX512 multi-buffer (Batch16, true 16-way multi-buffer, normalized per message over a batch of 16 messages).

Table 13. Cycles per byte cpb (smaller is faster)

| Implementation                    | 64B    | 192B   | 1KB    | 1536B  | 64KB   | 1MB    |
|-----------------------------------|--------|--------|--------|--------|--------|--------|
| AFS-TrEDM-S6 Reference 512        | 195.76 | 126.31 | 112.04 | 107.88 | 99.19  | 102.94 |
| AFS-TrEDM-S6 Reference 768        | 346.45 | 354.49 | 237.30 | 251.84 | 235.51 | 224.46 |
| AFS-TrEDM-S6 Reference 1024       | 829.89 | 520.19 | 428.32 | 418.14 | 399.17 | 401.47 |
| AFS-TrEDM-S6 Optimized opt64 512  | 25.14  | 16.27  | 14.00  | 12.83  | 12.12  | 12.11  |
| AFS-TrEDM-S6 Optimized opt64 768  | 42.81  | 40.06  | 27.15  | 27.25  | 26.34  | 25.94  |
| AFS-TrEDM-S6 Optimized opt64 1024 | 97.06  | 63.42  | 53.26  | 50.86  | 48.13  | 48.66  |
| AFS-TrEDM-S6 AVX2 hybrid 512      | 18.81  | 12.50  | 10.21  | 9.79   | 8.92   | 9.00   |
| AFS-TrEDM-S6 AVX2 hybrid 768      | 30.37  | 32.00  | 21.24  | 22.01  | 20.61  | 20.54  |
| AFS-TrEDM-S6 AVX2 hybrid 1024     | 74.99  | 48.32  | 37.27  | 37.53  | 35.56  | 36.08  |
| AFS-TrEDM-S6 Batch16 512          | 7.08   | 4.55   | 3.69   | 3.53   | 3.40   | 3.33   |
| AFS-TrEDM-S6 Batch16 768          | 11.11  | 10.68  | 7.42   | 7.48   | 7.04   | 7.04   |
| AFS-TrEDM-S6 Batch16 1024         | 25.91  | 16.99  | 13.67  | 13.21  | 12.61  | 12.88  |
| SHA3-512 XKCP readable            | 182.22 | 178.89 | 168.69 | 161.09 | 159.79 | 157.26 |
| SHA3-512 XKCP CompactFIPS202      | 193.22 | 200.58 | 184.94 | 183.40 | 176.37 | 175.67 |
| SHA3-512 OpenSSL EVP              | 18.77  | 11.61  | 8.39   | 7.83   | 7.34   | 7.28   |

Table 14. Cycles per hash (cycles per hash, smaller is faster)

| Implementation                    | 64B      | 192B     | 1KB       | 1536B     | 64KB        | 1MB          |
|-----------------------------------|----------|----------|-----------|-----------|-------------|--------------|
| AFS-TrEDM-S6 Reference 512        | 12528.46 | 24250.89 | 114733.35 | 165696.41 | 6500533.70  | 107941505.67 |
| AFS-TrEDM-S6 Reference 768        | 22173.11 | 68061.60 | 242997.46 | 386820.05 | 15434166.75 | 235361258.83 |
| AFS-TrEDM-S6 Reference 1024       | 53112.78 | 99876.47 | 438601.61 | 642260.10 | 26159894.18 | 420971036.33 |
| AFS-TrEDM-S6 Optimized opt64 512  | 1608.64  | 3123.14  | 14334.90  | 19713.58  | 794493.84   | 12700741.67  |
| AFS-TrEDM-S6 Optimized opt64 768  | 2739.52  | 7691.18  | 27800.11  | 41857.39  | 1726377.28  | 27197905.67  |
| AFS-TrEDM-S6 Optimized opt64 1024 | 6211.99  | 12176.62 | 54534.90  | 78117.25  | 3154093.24  | 51027719.00  |
| AFS-TrEDM-S6 AVX2 hybrid 512      | 1203.90  | 2399.67  | 10455.99  | 15032.81  | 584297.45   | 9441956.81   |
| AFS-TrEDM-S6 AVX2 hybrid 768      | 1943.82  | 6143.72  | 21751.73  | 33803.83  | 1350478.28  | 21536845.50  |

| Implementation                | 64B      | 192B     | 1KB       | 1536B     | 64KB        | 1MB          |
|-------------------------------|----------|----------|-----------|-----------|-------------|--------------|
| AFS-TrEDM-S6 AVX2 hybrid 1024 | 4799.31  | 9278.16  | 38164.02  | 57642.24  | 2330357.10  | 37828244.58  |
| AFS-TrEDM-S6 Batch16 512      | 453.04   | 874.39   | 3777.06   | 5429.12   | 223030.65   | 3486705.18   |
| AFS-TrEDM-S6 Batch16 768      | 711.29   | 2050.45  | 7601.22   | 11494.35  | 461231.28   | 7385976.26   |
| AFS-TrEDM-S6 Batch16 1024     | 1658.20  | 3261.76  | 13997.24  | 20287.75  | 826333.20   | 13503134.70  |
| SHA3-512 XKCP readable        | 11662.25 | 34347.12 | 172742.24 | 247432.70 | 10471952.49 | 164894304.72 |
| SHA3-512 XKCP CompactFIPS202  | 12366.29 | 38510.84 | 189376.85 | 281698.44 | 11558597.10 | 184206205.35 |
| SHA3-512 OpenSSL EVP          | 1201.57  | 2228.70  | 8592.37   | 12021.10  | 480900.38   | 7634838.05   |

Table 15. Throughput (MB/s, decimal bytes per second, larger is faster)

| Implementation                    | 64B    | 192B   | 1KB    | 1536B  | 64KB   | 1MB    |
|-----------------------------------|--------|--------|--------|--------|--------|--------|
| AFS-TrEDM-S6 Reference 512        | 12.75  | 19.76  | 22.28  | 23.14  | 25.17  | 24.25  |
| AFS-TrEDM-S6 Reference 768        | 7.21   | 7.04   | 10.52  | 9.91   | 10.60  | 11.12  |
| AFS-TrEDM-S6 Reference 1024       | 3.01   | 4.80   | 5.83   | 5.97   | 6.25   | 6.22   |
| AFS-TrEDM-S6 Optimized opt64 512  | 99.31  | 153.46 | 178.31 | 194.49 | 205.91 | 206.09 |
| AFS-TrEDM-S6 Optimized opt64 768  | 58.32  | 62.31  | 91.95  | 91.60  | 94.76  | 96.24  |
| AFS-TrEDM-S6 Optimized opt64 1024 | 25.72  | 39.36  | 46.87  | 49.08  | 51.87  | 51.30  |
| AFS-TrEDM-S6 AVX2 hybrid 512      | 132.70 | 199.72 | 244.46 | 255.05 | 279.98 | 277.22 |
| AFS-TrEDM-S6 AVX2 hybrid 768      | 82.19  | 78.01  | 117.51 | 113.42 | 121.14 | 121.50 |
| AFS-TrEDM-S6 AVX2 hybrid 1024     | 33.20  | 51.52  | 66.80  | 66.34  | 70.02  | 69.01  |
| AFS-TrEDM-S6 Batch16 512          | 351.75 | 546.74 | 675.05 | 704.47 | 731.67 | 748.86 |
| AFS-TrEDM-S6 Batch16 768          | 224.07 | 233.18 | 335.48 | 332.79 | 353.85 | 353.56 |
| AFS-TrEDM-S6 Batch16 1024         | 96.13  | 146.60 | 182.21 | 188.57 | 197.54 | 193.42 |
| SHA3-512 XKCP readable            | 13.67  | 13.92  | 14.77  | 15.46  | 15.59  | 15.84  |
| SHA3-512 XKCP CompactFIPS202      | 12.89  | 12.42  | 13.47  | 13.59  | 14.13  | 14.19  |
| SHA3-512 OpenSSL EVP              | 132.74 | 214.71 | 297.02 | 318.45 | 339.65 | 342.30 |

Table 16. Latency (milliseconds, smaller is faster)

| Implementation                    | 64B        | 192B       | 1KB        | 1536B      | 64KB       | 1MB        |
|-----------------------------------|------------|------------|------------|------------|------------|------------|
| AFS-TrEDM-S6 Reference 512        | 5.0190e-03 | 9.7151e-03 | 4.5963e-02 | 6.6380e-02 | 2.6042e+00 | 4.3242e+01 |
| AFS-TrEDM-S6 Reference 768        | 8.8828e-03 | 2.7266e-02 | 9.7347e-02 | 1.5496e-01 | 6.1830e+00 | 9.4288e+01 |
| AFS-TrEDM-S6 Reference 1024       | 2.1278e-02 | 4.0011e-02 | 1.7571e-01 | 2.5730e-01 | 1.0480e+01 | 1.6864e+02 |
| AFS-TrEDM-S6 Optimized opt64 512  | 6.4444e-04 | 1.2512e-03 | 5.7427e-03 | 7.8974e-03 | 3.1828e-01 | 5.0880e+00 |
| AFS-TrEDM-S6 Optimized opt64 768  | 1.0975e-03 | 3.0812e-03 | 1.1137e-02 | 1.6768e-02 | 6.9160e-01 | 1.0896e+01 |
| AFS-TrEDM-S6 Optimized opt64 1024 | 2.4886e-03 | 4.8781e-03 | 2.1847e-02 | 3.1294e-02 | 1.2636e+00 | 2.0442e+01 |
| AFS-TrEDM-S6 AVX2 hybrid 512      | 4.8229e-04 | 9.6133e-04 | 4.1888e-03 | 6.0223e-03 | 2.3407e-01 | 3.7825e+00 |
| AFS-TrEDM-S6 AVX2 hybrid 768      | 7.7871e-04 | 2.4612e-03 | 8.7139e-03 | 1.3542e-02 | 5.4101e-01 | 8.6305e+00 |
| AFS-TrEDM-S6 AVX2 hybrid 1024     | 1.9279e-03 | 3.7269e-03 | 1.5330e-02 | 2.3153e-02 | 9.3602e-01 | 1.5194e+01 |
| AFS-TrEDM-S6 Batch16 512          | 1.8195e-04 | 3.5118e-04 | 1.5169e-03 | 2.1804e-03 | 8.9570e-02 | 1.4002e+00 |
| AFS-TrEDM-S6 Batch16 768          | 2.8563e-04 | 8.2339e-04 | 3.0523e-03 | 4.6156e-03 | 1.8521e-01 | 2.9658e+00 |
| AFS-TrEDM-S6 Batch16 1024         | 6.6579e-04 | 1.3097e-03 | 5.6199e-03 | 8.1456e-03 | 3.3177e-01 | 5.4213e+00 |
| SHA3-512 XKCP readable            | 4.6820e-03 | 1.3789e-02 | 6.9349e-02 | 9.9333e-02 | 4.2038e+00 | 6.6186e+01 |
| SHA3-512 XKCP CompactFIPS202      | 4.9633e-03 | 1.5456e-02 | 7.6006e-02 | 1.1306e-01 | 4.6387e+00 | 7.3918e+01 |
| SHA3-512 OpenSSL EVP              | 4.8213e-04 | 8.9425e-04 | 3.4476e-03 | 4.8233e-03 | 1.9295e-01 | 3.0633e+00 |

Note: Batch16 is a multi-buffer normalized-throughput backend; its cpb, cycles/hash, and latency are all per-message normalized values within a batch of 16 messages and should not be interpreted as single-message latency. The three SHA3-512 baselines are external comparisons and are not the reference or optimized implementations of AFS-TrEDM.

## 4.4 Same-Device Comparison with SHA3-512

To ensure comparability, the comparison is done on the same test machine, distinguishing three deployment contexts (to avoid conflating numbers from different contexts). The following comparison takes a 1MB long message as representative (cpb, smaller is faster).

(1) Reference implementation vs. reference implementation. AFS-TrEDM-S6 Reference 512 (102.94) vs. SHA3-512 XKCP readable (157.26): the AFS-TrEDM-S6 Reference 512 implementation is about 1.53 times faster. This shows that at the ISO C reference level, independent of the platform instruction set, AFS-TrEDM-S6 Reference 512 already outperforms the SHA3-512 XKCP readable reference implementation.

(2) Deployment-level single message vs. deployment-level single message. AFS-TrEDM-S6 AVX2 hybrid 512 (9.00) vs. SHA3-512 OpenSSL EVP (7.28): in the single-message deployment context, the two are of the same magnitude, with SHA3-512 OpenSSL EVP about 1.24 times faster. The AFS-TrEDM-S6 AVX2 hybrid 512 single-message optimized implementation has entered a range comparable to the highly optimized OpenSSL SHA3.

(3) High-throughput multi-buffer. AFS-TrEDM-S6 Batch16 512 (3.33 cpb, 748.86 MB/s) vs. SHA3-512 OpenSSL EVP (7.28 cpb, 342.30 MB/s): in the multi-buffer throughput context, AFS-TrEDM-S6 Batch16 512 has about 2.19 times the throughput of SHA3-512 OpenSSL EVP. In large-batch parallel hashing scenarios, AFS-512 leads significantly.

Table 17. Comparison with SHA3-512 under a 1MB message (cpb / throughput)

| Implementation                      | cpb@1MB | Throughput@1MB<br>(MB/s) | Comparison      |
|-------------------------------------|---------|--------------------------|-----------------|
| <b>AFS-TrEDM-S6 Reference 512</b>   | 102.94  | 24.25                    | Reference level |
| SHA3-512 XKCP readable              | 157.26  | 15.84                    | Reference level |
| <b>AFS-TrEDM-S6 AVX2 hybrid 512</b> | 9.00    | 277.22                   | Single message  |
| SHA3-512 OpenSSL EVP                | 7.28    | 342.30                   | Single message  |
| <b>AFS-TrEDM-S6 Batch16 512</b>     | 3.33    | 748.86                   | Multi-buffer    |

## 4.5 Reproducibility

All the above numbers are produced in one click by the performance-test script accompanying the implementation, covering the three instances, the four implementations, and the three SHA3-512 baselines, and can be reproduced on the §4.2 platform. The script simultaneously outputs the four tables of cpb, cycles/hash, throughput, and latency, together with the metric-baseline explanation, for the evaluator to check. Both the reference and optimized implementations come with automated build scripts that can generate the KATs in §1.12 and the test-vector directory.

## 5 Statement of Features

This chapter states the features of AFS-TrEDM in four aspects: innovativeness, simplicity, flexibility, and cross-platform performance advantages.

### 5.1 Innovativeness

The innovation of AFS-TrEDM is concentrated at two levels: the mode and the components.

- The aligned truncated-EDM sponge mode. It locks the feed-forward window, the chain-value window, and the digest-output window onto the same segment of capacity bits  $S$ , combining the capacity security of the sponge with the Encrypted Davies–Meyer feed-forward, providing a structural source for preimage and second-preimage resistance while preserving the capacity bound (§3.2).
- The AFS-LMDS-1600-S6 six-direction rotating single-MDS linear layer. Centered on a single Cauchy MDS column mixing over GF(32), combined with the  $\rho_{\text{AFS}}$  differentiated circular shift, the  $\pi_{\text{AFS}}$  lane reordering, and the  $\mu$  mixing, and using the six-direction rotation  $\tau$  to inject different column directions in different rounds; the rotation  $\tau$  is merely a rewiring of lanes with no arithmetic overhead, yet significantly enhances cross-lane diffusion.
- The AFS-64 ARX large S-box. Replaces bit-by-bit  $\chi$ -type nonlinearity with a 64-bit ARX-Feistel, with no table lookups, resistant to cache-timing side channels, and naturally vectorizable.
- Tiered round counts. Uses 12 / 20 / 24 rounds for the underlying permutation according to the digest length, striking a per-instance balance between security margin and throughput, reflecting the engineering trade-off of “stop when security is sufficient.”

## 5.2 Simplicity

The algorithm is easy to understand and implement, facilitating a thorough security evaluation: (1) both the nonlinear and linear layers consist only of modular addition, XOR, and circular shift, with no S-box lookup tables and no large constant tables, and the reference implementation is short ISO C; (2) the round constants and IV are all deterministically derived from the public, documented SplitMix64 / FNV-1a64, can be regenerated and verified by short code, and contain no unexplainable “magic numbers”; (3) the state is represented as 25 64-bit lanes, naturally aligned with 64-bit general-purpose registers and SIMD lanes, with a direct implementation path. The simple ARX structure also makes formal analysis (such as the satisfiability window verification of this specification) easy to model.

## 5.3 Flexibility

The sponge skeleton based on a public permutation easily derives other cryptographic functions: a message authentication code (MAC, such as a keyed-sponge / feed-forward MAC), a key-derivation function (KDF), and an extendable-output function (XOF, which can output any length by adjusting the squeezing phase) can be constructed on the same permutation. The configurability of the tiered round counts and the rate/capacity provides a balanced trade-off between computational efficiency and resource consumption: in resource-constrained or high-throughput scenarios, a shorter-digest instance with fewer rounds can be chosen, and in high-security-demand scenarios, the 1024 instance can be chosen.

## 5.4 Implementation Performance Advantages on Different Platforms

AFS-TrEDM is a hash algorithm oriented toward efficient software implementation. Its nonlinear layer AFS-64 adopts a pure ARX-Feistel structure, consisting only of modular addition, circular shift, and XOR, which are all single-cycle native instructions on modern general-purpose processors and can be mapped directly to scalar registers and SIMD lanes; the lane-friendly three-shift structure of the linear layer AFS-LMDS-1600-S6 is likewise implemented with in-register shift-XOR, with no table lookups and no branches. For this reason, the performance advantage of this algorithm is concentrated in software implementation: the measurements of §4.3–§4.5 show that AFS-TrEDM-512 can reach about 2.19 times the throughput of the highly optimized SHA3-512 implementation (OpenSSL EVP) in the multi-buffer throughput context, and is also faster than the SHA3-512 readable reference implementation at the ISO C reference level; this speed advantage stems precisely from the natural fit between ARX operations and the processor instruction set.

The algorithm can be implemented securely and efficiently on a variety of platforms from high-performance servers to resource-constrained embedded devices; the concrete manifestations of the above software advantages on different target platforms include: (1) parallel processing — the permutation with no data-dependent branches is suitable for multi-buffer parallelism, and 16-way multi-buffering on AVX-512 can fill the pipeline (§4); (2) instruction-set acceleration — ARX operations can be mapped directly to SIMD vector instructions such as x86's AVX2/AVX-512 and ARM's NEON, and the AVX2 hybrid and Batch16 implementations of §4 give significant speedups; (3) ARM and embedded platforms — the 64-bit ARX structure of AFS-64 can be mapped directly to the native 64-bit registers and single-cycle circular-shift and modular-addition instructions of AArch64, and 128-bit NEON can further accelerate multi-way permutations in parallel; the absence of table lookups eliminates data-dependent memory access, which both avoids cache-timing side channels and relieves the memory-access and cache pressure of embedded SoCs, and the branch-free structure is especially friendly to in-order-execution cores such as Cortex-M / Cortex-A; (4) resource-constrained environments — with no large state lookup tables and no large constant tables, the storage footprint is small, all public constants can be regenerated on the fly by short code such as SplitMix64, and the code and data size is small, facilitating deployment on RAM/Flash-constrained embedded and IoT devices.

## 6 Implementation Status and Code Structure

This chapter describes the structure and status of the accompanying implementation code (the Implementations folder). All implementations use the unified CryptHash programming interface (§1.11) and cover the three algorithm instances; the implementation code of each instance is placed in a separate folder, named by [AlgorithmInstance] (AFS-TrEDM-512 / AFS-TrEDM-768 / AFS-TrEDM-1024).

### 6.1 Reference Implementation (Reference\_Implementation)

Written in ISO C with no dependence on a specific platform instruction set, serving as the authoritative readable definition of the algorithm. Each instance directory contains: the AFS-64 S-box (`afs_sbox64.c/h`), the AFS-LMDS-1600-S6 linear layer (`afs_lmlds1600_s6.c/h`), the AFS-*p*-S6 permutation (`afs_p1600.c/h`, with the tier derived from DIGEST\_BIT\_LENGTH: 512→6, 768→10, 1024→12), the aligned-TrEDM sponge (`afs_tredm.c/h`), and the CryptHash interface (`CryptHash_AlgorithmInstance.c/h`). The source code of the three instances differs only in DIGEST\_BIT\_LENGTH in `CryptHash_AlgorithmInstance.h`. The outputs of the reference and optimized implementations are bit-exact.

### 6.2 Optimized Implementation (Optimized\_Implementation)

Targeting mainstream 64-bit PC processors, it includes two types of backends: opt64 (a generated scalar 64-bit implementation) and AVX2 hybrid (an AVX2-vectorized AFS-64 S-box + the opt64 S6 linear layer). The common code is organized in `core / dispatch / low / s6` layers for distribution across different instruction sets.

### 6.3 Additional Implementation (Additional\_Implementation)

Batch\_Multibuffer: a true 16-way AVX512 multi-buffer implementation that parallelizes a batch of 16 messages to maximize throughput, with performance reported per-message normalized (see the §4.3 table note).

### 6.4 Tools and Automated Build

The tools directory provides regeneration and verification scripts for constants and algebraic properties: `gen_s6_iv.py` (regenerates the IVs of the three instances), `check_round_constants.py` (verifies that all RC32 are nonzero and distinct), and `check_s6_algebra.py` (verifies the GF(32) and Cauchy MDS algebraic properties).

Both the reference and optimized implementations come with automated build scripts that can generate the KATs required by the Test\_Vectors directory (KAT\_2\_12 / KAT\_2\_23 / KAT\_2\_33 / KAT\_Loop). The README gives the file directory and a brief description of each file.

## 7 References

- [1] GUO C, HU K, JIANG S, et al. Permutation-Based Hash from Non-Idealized Assumptions: Adding Feed-Forward to Sponge[C/EB/OL]. Cryptology ePrint Archive, Paper 2025/1006. 2025. <https://eprint.iacr.org/2025/1006>.
- [2] SUN S, LI S, ZHANG Z, et al. Permutation-Based Hashing With Stronger (Second) Preimage Resistance[C/EB/OL]. Cryptology ePrint Archive, Paper 2025/963. 2025. <https://eprint.iacr.org/2025/963>.
- [3] YAN Z, WEI Y, RODRIGUEZ-ALDAMA R, et al. AFS: A Family of ARX-Based Large-State S-boxes with Exceptional Properties[EB/OL]. 2026. <https://github.com/Crypto2026Code/AFS>.
- [4] National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions: FIPS PUB 202[S/OL]. Gaithersburg, MD: NIST, August 2015. <https://doi.org/10.6028/NIST.FIPS.202>.
- [5] TURAN M S, MCKAY K A, KANG J, et al. Ascon-Based Lightweight Cryptography Standards for Constrained Devices: NIST SP 800-232[S/OL]. Gaithersburg, MD: NIST, August 2025. <https://doi.org/10.6028/NIST.SP.800-232>.
- [6] BERTONI G, DAEMEN J, PEETERS M, et al. Sponge Functions[C]//Ecrypt Hash Workshop. Barcelona, Spain, 2007.
- [7] BERTONI G, DAEMEN J, PEETERS M, et al. On the Indifferentiability of the Sponge Construction[C]//SMART N P, ed. Advances in Cryptology — EUROCRYPT 2008. LNCS, vol. 4965. Berlin: Springer, 2008: 181-197. [https://doi.org/10.1007/978-3-540-78967-3\\_11](https://doi.org/10.1007/978-3-540-78967-3_11).
- [8] BERTONI G, DAEMEN J, HOFFERT S, et al. The eXtended Keccak Code Package (XKCP)[EB/OL]. GitHub Repository. <https://github.com/XKCP/XKCP>.
- [9] CORON J S, DODIS Y, MALINAUD C, et al. Merkle-Damgard Revisited: How to Construct a Hash Function[C]//SHOUP V, ed. Advances in Cryptology — CRYPTO 2005. LNCS, vol. 3621. Berlin: Springer, 2005: 430-448. [https://doi.org/10.1007/11535218\\_26](https://doi.org/10.1007/11535218_26).
- [10] CHOI W, LEE B, LEE J. Indifferentiability of Truncated Random Permutations[C]//GALBRAITH S D, MORIAI S, eds. Advances in Cryptology — ASIACRYPT 2019, Part I. LNCS, vol. 11921. Cham: Springer, 2019: 175-195. [https://doi.org/10.1007/978-3-030-34578-5\\_7](https://doi.org/10.1007/978-3-030-34578-5_7).
- [11] COGLIATI B, SEURIN Y. EWCDM: an Efficient, Beyond-Birthday Secure, Nonce-Misuse Resistant MAC[C]//ROBshaw M, KATZ J, eds. Advances in Cryptology — CRYPTO 2016, Part I. LNCS, vol. 9814. Berlin: Springer, 2016: 121-149. [https://doi.org/10.1007/978-3-662-53018-4\\_5](https://doi.org/10.1007/978-3-662-53018-4_5).
- [12] BELLARE M, ROGAWAY P. Random Oracles are Practical: A Paradigm for Designing Efficient Protocols[C]//Proceedings of the 1st ACM CCS 1993. New York, NY: ACM, 1993: 62-73. <https://doi.org/10.1145/168588.168596>.
- [13] BEIERLE C, BIRYUKOV A, DOS SANTOS L C, et al. Alzette: A 64-Bit ARX-Box (Feat. CRAX and TRAX)[C]//MICCIANCIO D, RISTENPART T, eds. Advances in Cryptology — CRYPTO 2020, Part III. LNCS, vol. 12172. Cham: Springer, 2020: 419-448. [https://doi.org/10.1007/978-3-030-56877-1\\_15](https://doi.org/10.1007/978-3-030-56877-1_15).
- [14] BERTONI G, DAEMEN J, PEETERS M, et al. KangarooTwelve: Fast Hashing Based on Keccak-p[C]//PRENEEL B, VERCAUTEREN F, eds. ACNS 2018. LNCS, vol. 10892. Cham: Springer, 2018: 400-418. [https://doi.org/10.1007/978-3-319-93387-0\\_21](https://doi.org/10.1007/978-3-319-93387-0_21).
- [15] BERTONI G, DAEMEN J, HOFFERT S, et al. TurboSHAKE[EB/OL]. Cryptology ePrint Archive, Paper 2023/342. 2023. <https://eprint.iacr.org/2023/342>.
- [16] LEFEVRE C, MENNINK B. Tight Preimage Resistance of the Sponge Construction[C]//DODIS Y, SHRIMPTON T, eds. Advances in Cryptology — CRYPTO 2022, Part IV. LNCS, vol. 13510. Cham: Springer, 2022: 185-204. [https://doi.org/10.1007/978-3-031-15985-5\\_7](https://doi.org/10.1007/978-3-031-15985-5_7).
- [17] National Institute of Standards and Technology. Secure Hash Standard (SHS): FIPS PUB 180-4[S/OL]. Gaithersburg, MD: NIST, August 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>.
- [18] State Cryptography Administration. SM3 Cryptographic Hash Algorithm: GM/T 0004-2012[S]. Beijing: Standards Press of China, 2012.

- [19] Institute of Commercial Cryptography Standards. Submission Requirements for Cryptographic Hash Algorithms[S/OL]. Beijing: Institute of Commercial Cryptography Standards, 2025.
- [20] Institute of Commercial Cryptography Standards. Evaluation Criteria for Cryptographic Hash Algorithms[S/OL]. Beijing: Institute of Commercial Cryptography Standards, 2025.
- [21] COJOCARU A, HHAN M, LIU Q, et al. Quantum Lifting for Invertible Permutations and Ideal Ciphers[C]//TAUMAN KALAI Y, KAMARA S F, eds. Advances in Cryptology — CRYPTO 2025. LNCS, vol. 16001. Cham: Springer, 2025: 481-512. [https://doi.org/10.1007/978-3-032-01878-6\\_16](https://doi.org/10.1007/978-3-032-01878-6_16).
- [22] ZHANDRY M. How to Record Quantum Queries, and Applications to Quantum Indifferentiability[C]//BOLDYREVA A, MICCIANCIO D, eds. Advances in Cryptology — CRYPTO 2019, Part II. LNCS, vol. 11693. Cham: Springer, 2019: 239-268. [https://doi.org/10.1007/978-3-030-26951-7\\_9](https://doi.org/10.1007/978-3-030-26951-7_9).
- [23] ALAGIC G, CAROLAN J, MAJENZ C, et al. The Sponge is Quantum Indifferentiable[EB/OL]. Cryptology ePrint Archive, Paper 2025/731. 2025. <https://eprint.iacr.org/2025/731>.

*Reference [3] is work that has not yet been formally published; once the paper is formally published, the submitter will supplement its venue / DOI / page-number fields.*

## Appendix A Initial Values (IV)

The IV of each instance is a public 1600-bit constant, deterministically derived from the domain string by the FNV-1a64 + SplitMix64 chain described in §1.9, and is verbatim identical to the hardcoded constants in the reference/optimized implementations. The complete IVs of the three instances are given below; each IV is 25 64-bit words, listed in big-endian hexadecimal with 5 words per row ( $A[0]$  is at the leftmost of the first row,  $A[24]$  is at the rightmost of the last row). Modifying any parameter, the linear layer, or the round count changes the domain string and thereby changes all IVs and KATs.

### A.1 The IV of AFS-TrEDM-512

Domain string: AFS-TrEDM-v2|d=512|r=1024|c=576|b=1600| $n_r$ =12|sbox=AFS64\_t5\_k2|linear=AFS-LMDS-1600-S6

|                  |                  |                  |                  |                  |
|------------------|------------------|------------------|------------------|------------------|
| 53994199EC2747FF | BFD3CD9EBA914A21 | E23244DD302409B  | B82F2C0D67F11A56 | 2D61D2E164DC0C64 |
| F604258A090B4F87 | E583B2DE7BCC0934 | 9C1960759D1FD367 | CDE041B956651F19 | 32D52A1048F20B3F |
| DA10FA3BE09F1AEE | C22C74696B68742D | 81A2E4EC00649D7F | 0057434538916AEA | 4FEA9520E57D8E11 |
| 3121586C1336D4CB | D652524E1FB3F916 | 27804CBD8F09D5F2 | 9F7DC67670C3DF84 | EB9CDC94C6B7B81B |
| 1388619624CA2963 | 09B71D9B01191FED | 14CFDB61E20D4FDC | 3DFA768C46B23134 | E50AEFF14CB6DA44 |

### A.2 The IV of AFS-TrEDM-768

Domain string: AFS-TrEDM-v2|d=768|r=768|c=832|b=1600| $n_r$ =20|sbox=AFS64\_t5\_k2|linear=AFS-LMDS-1600-S6

|                  |                  |                  |                  |                  |
|------------------|------------------|------------------|------------------|------------------|
| B0F2BB7C26FDCB63 | 41D198EC6D5B2B7B | 751BCA414F8B3A3D | 9B4D5ECA37E5DCA2 | D5451D0D87B13E70 |
| 29BDA85FB12825C0 | EFAE039D229BB856 | 5533F9678259C50F | 57F530B2442DEC6C | ED048C2F7D498654 |
| 0A7306A3F12442A3 | CCE25A5F87D775C1 | 079D4E0253B563AA | AC33774D15DBC6FD | 81EC4A8254BFD641 |
| C3B964F0008423A0 | 06B3661AA4B14E44 | C47AD03E50312E31 | 6AD795F957D10241 | EF8D172A503F8F62 |
| EB3CA46C3DD3FB80 | ECEE8411633651CD | D7139EC73D704F63 | E307189922FE9F8E | 73DD2748F22D7F1F |

### A.3 The IV of AFS-TrEDM-1024

Domain string: AFS-TrEDM-v2|d=1024|r=512|c=1088|b=1600| $n_r$ =24|sbox=AFS64\_t5\_k2|linear=AFS-LMDS-1600-S6

|                  |                  |                  |                  |                  |
|------------------|------------------|------------------|------------------|------------------|
| A4F69C995CEFC6D9 | 06991C0A3E9FB307 | 6A07F13502426450 | 60F0251F0F6A8899 | 47F583E911141969 |
| C5B7C6E311543B3F | 2C0515877EC5BDAF | 14A57976513CCA55 | EA295865D33E9D7B | 3F17E94697F82F64 |
| F0D601977DB63BA5 | 6C2026D3858676EE | 7D0C3960480FD237 | 2D8ED3ED88F475D7 | 9332B33412A596F6 |
| 9DF09750C9D639B4 | 52D73E916617792C | 6493DF5CE7DF5A21 | 20384DA1E6C497FE | 61C456C0442053A2 |
| E9CDFD19AAEC53A1 | EAEA95C4CEA0E06E | 19F1B0C69940D2BE | 4F4A68E0F0FB52F1 | A59BEAD309B75F6B |

## Appendix B Six-Phase Three-Round Window Verification

This appendix lists the six-phase three-round window verification checklist cited in §3.3.1. The direction schedule of AFS-LMDS-1600-S6,  $\text{DIR\_SCHEDULE} = [\infty, 0, 1, 2, 3, 4]$ , has a period of 6, so a three-round window has 6 phases in total; for each phase and the two directions (differential and linear), a satisfiability instance of fewer than  $N_3$  active S-boxes within the window is constructed and decided by the solver. Table B-1 gives the 12 targets and their decision results.

Table B-1. Six-phase  $\times$  differential/linear window verification (12 targets in total)

| Phase ( $i, \text{mod } 6$ ) | Three-round direction sequence       | Direction           | Solver          | Result        |
|------------------------------|--------------------------------------|---------------------|-----------------|---------------|
| 0                            | $\infty \rightarrow 0 \rightarrow 1$ | Differential/Linear | CMS 5.14.5 + GJ | UNSAT / UNSAT |

| Phase ( $i_r \bmod 6$ ) | Three-round direction sequence       | Direction           | Solver          | Result               |
|-------------------------|--------------------------------------|---------------------|-----------------|----------------------|
| 1                       | $0 \rightarrow 1 \rightarrow 2$      | Differential/Linear | CMS 5.14.5 + GJ | <b>UNSAT / UNSAT</b> |
| 2                       | $1 \rightarrow 2 \rightarrow 3$      | Differential/Linear | CMS 5.14.5 + GJ | <b>UNSAT / UNSAT</b> |
| 3                       | $2 \rightarrow 3 \rightarrow 4$      | Differential/Linear | CMS 5.14.5 + GJ | <b>UNSAT / UNSAT</b> |
| 4                       | $3 \rightarrow 4 \rightarrow \infty$ | Differential/Linear | CMS 5.14.5 + GJ | <b>UNSAT / UNSAT</b> |
| 5                       | $4 \rightarrow \infty \rightarrow 0$ | Differential/Linear | CMS 5.14.5 + GJ | <b>UNSAT / UNSAT</b> |

All 12 targets return UNSAT, indicating that the number of active S-boxes in every three-round window is at least  $N_3$  (both differential and linear  $\geq 12$ ), and this lower bound does not degrade with the window phase. The DRAT proofs of two of the single-round bit-level UNSAT results are additionally verified independently by drat-trim. Key points of the solver configuration: for XOR-dense UNSAT instances, Gauss-Jordan elimination of CryptoMiniSat must be enabled (explicitly set `--maxmatrixcols`, `--maxxormat`, `--autodisablegauss 0`; CMS silently disables GJ when the default thresholds are exceeded).

*Note: what this appendix describes is solver UNSAT decision plus reproducible archive, used to support the phase-equivalence premise of the wide-trail combination  $N_{24} \geq 8 \times N_3 = 96$ ; its nature differs from a rigorous mathematical proof (see the distinction of the three classes of statements in the introduction of this chapter).*

## Appendix C AFS-TrEDM-S6 Randomness Test Report

This appendix gives the main text of the “AFS-TrEDM-S6 Randomness Test Report” cited in §3.4; the report records the complete evaluation methodology, sample generation, and results of the three instances (512 / 768 / 1024) on three classes of standard statistical-randomness tests: NIST SP 800-22 STS, TestU01 SmallCrush, and avalanche/bit-independence (SAC/BIC).

### C.1 Report Scope and Test Objects

#### C.1.1 Test Objects

The objects under test are the CryptHash digest outputs of the AFS-TrEDM-S6 algorithm, covering the three digest-length configurations 512, 768, and 1024. Each bit stream is taken from the direct output of CryptHash, with no whitening, shuffle, bit reverse, compression, or other post-processing applied, to ensure that the test reflects the output statistical characteristics of the primitive itself, rather than a result masked by post-processing.

#### C.1.2 Report Scope

The main decision items of the main text of this report are three classes of tests: NIST SP 800-22 STS, TestU01 SmallCrush, and avalanche/bit-independence (SAC/BIC). TestU01 executes only the SmallCrush battery, not Crush or BigCrush, so no conclusion is drawn about the latter two. The report does not include historical test records.

#### C.1.3 Evaluation Criteria and Decision Baseline

The acceptance criteria of each test are as follows:

- NIST STS (significance level  $\alpha = 0.01$ ): for each sub-item, simultaneously test (a) whether the proportion of passing sequences falls within the acceptable interval, and (b) the uniformity of the P-values on  $[0,1)$ . For  $m = 1000$  sequences, the proportion lower bound is  $0.99 - 3 \cdot \sqrt{(0.99 \cdot 0.01 / 1000)} \approx 0.9806$ , i.e.,  $\geq 981/1000$  is required; the uniformity P-value must be  $\geq 1 \times 10^{-4}$ .
- TestU01 SmallCrush: the p-values of all statistics in the battery fall within the acceptable range (typically requiring no significant anomaly of  $p < 10^{-3}$  or  $p > 1 - 10^{-3}$ ), and no rewind occurs during stream reading.
- Avalanche/bit-independence: the formal decision is based on whether the output bit-flip rate falls within the coarse threshold band of 45%–55% (and the wider 40%–60%); the finer per-bit 99% confidence-interval outlier count is given as an auxiliary diagnosis.

For ease of review, the meanings of the internal status fields in the report are as follows:

| Internal field                  | Review-readable meaning                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PARSED_NONZERO_EXIT_1           | The report has been successfully parsed; the NIST STS tool returns a nonzero exit code (its usual behavior), and the original warning/confirmation table is retained. |
| PASS_WITH_TARGETED_CONFIRMATION | A boundary warning occurred in the main test and has passed a pre-registered targeted confirmation.                                                                   |

## C.2 Introduction to the Test Software

### C.2.1 NIST SP 800-22 STS

The NIST Statistical Test Suite (STS) is a randomness-evaluation standard for random and pseudorandom number generators in cryptographic applications, published by the U.S. National Institute of Standards and Technology (NIST Special Publication 800-22 Revision 1a, April 2010). The suite consists of 15 classes of statistical hypothesis tests, examining whether a sequence deviates from the ideal random distribution from multiple angles such as frequency, runs, spectrum, template matching, random walk, and linear complexity. This test uses the official NIST reference implementation (assess), computing the pass proportion and P-value uniformity of each sub-item separately for the multi-sequence sample set. It may also return a nonzero exit code under normal operation; this report records this case as PARSED\_NONZERO\_EXIT\_1 and takes the successfully parsed finalAnalysisReport as authoritative.

### C.2.2 TestU01

TestU01 is a C-language library for empirical testing of random number generators developed by P. L'Ecuyer and R. Simard (ACM Transactions on Mathematical Software, 33(4), 2007), and is one of the most rigorous and most commonly used tools in academia for testing PRNG quality. It encapsulates a set of statistical tests into predefined “batteries,” divided from small to large by scale into SmallCrush, Crush, BigCrush, etc. This test uses the smallest-scale SmallCrush battery, feeding the output bytes of AFS-TrEDM-S6 into the battery as 32-bit words through a custom stream reader (harness). The harness records the number of words read and the number of rewinds; a rewind count of 0 indicates that the battery completed all tests without reusing data.

### C.2.3 Avalanche and Bit-Independence Test Program

The avalanche and bit-independence test is a dedicated program implemented by this project according to the Strict Avalanche Criterion (SAC) and the Bit Independence Criterion (BIC); both criteria originate from the S-box design criteria proposed by Webster and Tavares at CRYPTO '85. For each configuration, the program generates a large number of input pairs differing by 1 input bit, counts the flip distribution of the output bits, computes the average Hamming distance, the per-pair flip rate, the per-bit flip rate and their outliers, and gives a formal PASS/FAIL based on the coarse threshold band, while also outputting fine-grained diagnostics such as the per-bit 99% confidence-interval outlier count.

## C.3 Description of Test Items

### C.3.1 NIST SP 800-22 STS Test Items

This test uses all 15 classes of tests (all mode). Since tests such as Non-overlapping Template and Random Excursions contain multiple sub-statistics, the 15 classes expand to 188 sub-items per sequence set. The classes of tests and their sensitive targets are shown in the following table.

| # | Test type               | Number of sub-items | Sensitive target / source of deviation                       |
|---|-------------------------|---------------------|--------------------------------------------------------------|
| 1 | Frequency (Monobit)     | 1                   | Overall 0/1 proportion of the entire sequence                |
| 2 | Block Frequency         | 1                   | Proportion of 1s within fixed blocks                         |
| 3 | Cumulative Sums (Cusum) | 2                   | Maximum offset of the $\pm 1$ random walk (forward/backward) |
| 4 | Runs                    | 1                   | Total number of 0/1 runs and oscillation frequency           |
| 5 | Longest Run of Ones     | 1                   | Longest run of consecutive 1s within a block                 |

| #  | Test type                 | Number of sub-items | Sensitive target / source of deviation                      |
|----|---------------------------|---------------------|-------------------------------------------------------------|
| 6  | Binary Matrix Rank        | 1                   | Submatrix rank distribution (linear correlation)            |
| 7  | DFT (Spectral)            | 1                   | Periodic peaks in the spectrum                              |
| 8  | Non-overlapping Template  | 148                 | Occurrence count of each non-overlapping $m$ -bit template  |
| 9  | Overlapping Template      | 1                   | Occurrence count of the overlapping $m$ -bit template       |
| 10 | Maurer Universal          | 1                   | Sequence compressibility (deviation from ideal entropy)     |
| 11 | Approximate Entropy       | 1                   | Frequency of adjacent $m / m+1$ -bit patterns               |
| 12 | Random Excursions         | 8                   | Number of visits to each state in the random walk           |
| 13 | Random Excursions Variant | 18                  | Number of visits to each state in the random walk (variant) |
| 14 | Serial                    | 2                   | Uniformity of all $m$ -bit patterns                         |
| 15 | Linear Complexity         | 1                   | LFSR linear-complexity distribution                         |

Total: 15 classes of tests  $\rightarrow$  188 statistical sub-items (per “instance  $\times$  input family” combination).

### C.3.2 TestU01 SmallCrush Test Items

SmallCrush is the smallest-scale predefined battery in TestU01, consisting of several basic statistical tests, mainly including:

- Birthday Spacings
- Collision (collision count)
- Gap (gap length)
- Simplified Poker
- Coupon Collector
- Maximum-of- $t$  (maximum of  $t$ )
- Weight Distribution
- Matrix Rank
- Hamming Independence
- Random Walk

The battery computes a p-value for each test; as long as none of the p-values shows a significant deviation (typical threshold  $p \notin (10^{-3}, 1 - 10^{-3})$ ), the whole is judged to pass.

### C.3.3 The Avalanche Criterion (SAC) and the Bit-Independence Criterion (BIC)

The strict avalanche criterion requires that, when the input undergoes a single-bit flip, each output bit flips with a probability of about 0.5 — i.e., a small perturbation of the input should be sufficiently diffused at the output. The bit-independence criterion further requires that the responses of any two output bits to an input perturbation be mutually independent, with no significant correlation.

Based on this, the following quantities are computed in this test:

- Average Hamming distance: the mean number of flipped bits between the two outputs of an input pair, with an ideal value of half the number of output bits.
- Average flip rate and standard deviation: the mean and dispersion of the output flip ratio over all input pairs, with an ideal mean of 0.5.

- Per-bit flip-rate interval: the minimum and maximum of the flip rate at each output bit position, used to detect whether there are biased “dead bits” or “hot bits.”
- Out-of-range count: the number of output bits falling outside the 45%–55% and 40%–60% bands (formal criterion), and the number of bits falling outside the per-bit 99% confidence interval (fine-grained diagnosis).

## C.4 Test-Sample Generation Method

### C.4.1 Test Instances and Input Families

The test covers the three instances AFS-TrEDM-S6-512, AFS-TrEDM-S6-768, and AFS-TrEDM-S6-1024. Each tier is evaluated independently on two classes of input message families: counter-v5-primary (a counter-based deterministic message sequence) and random-v5-primary (a message sequence based on random input). Each sequence uses independent domain separation, with a fixed message length of 256 bytes; the digest stream is the direct output of CryptHash, with no post-processing.

### C.4.2 NIST STS Samples

primary profile: each “instance  $\times$  input family” combination generates 1000 sequences of 1,000,000 bits each, covering all 188 sub-items. For sub-items with a boundary warning, an additional 5000 sequences are generated by the pre-registered seed for targeted confirmation. All bit streams are registered with a SHA-256 digest for reproducibility checking.

### C.4.3 TestU01 Samples

Each combination generates a continuous digest stream of 1 GiB (1,073,741,824 bytes), fed into SmallCrush as 32-bit words by the harness. SmallCrush actually consumes about  $2.27 \times 10^8$  32-bit words, all less than the stream capacity, so the stream rewind count of all 6 combinations is 0.

### C.4.4 Avalanche/Bit-Independence Samples

For each tier instance, 10,000 input pairs differing by 1 input bit are generated at each of 9 input lengths. The input lengths include the fixed lengths 64B, 128B, 1KB, as well as  $r-1$ ,  $r$ ,  $r+1$ ,  $2r-1$ ,  $2r$ ,  $2r+1$  set around the single-block rate boundary  $r$  (where  $r$  is the input block/rate boundary of the instance, namely 1024, 768, and 512 bits for the respective instances). Sampling around the block boundary is intended to test whether the diffusion across the absorbing/block boundary is sufficient.

### C.4.5 Seed Pre-Registration and Anti-Cherry-Picking Strategy

All formal seeds are fixedly registered in the seed registry before the NIST primary test is executed; the confirmation seeds for targeted confirmation are registered separately before the confirmation is executed. This “register first, execute later” process ensures that the review of any boundary warning is not the result of cherry-picking favorable seeds afterward. A representative seed identifier is of the form AFS-RND-NISTPRIMARY-RND61-768-counter-v5-primary-20260621.

## C.5 Test Results

### C.5.1 Summary of NIST SP 800-22 STS Results

The primary profile covers all 6 “instance  $\times$  input family” combinations, reporting the pass proportion and P-value uniformity item by item. The summary is in the following table.

| Instance | Input family       | Number of sequences | Bits per sequence | Number of sub-items | Number of warnings | State                        |
|----------|--------------------|---------------------|-------------------|---------------------|--------------------|------------------------------|
| 512      | counter-v5-primary | 1000                | 1 000 000         | 188                 | 0                  | Pass                         |
| 512      | random-v5-primary  | 1000                | 1 000 000         | 188                 | 0                  | Pass                         |
| 768      | counter-v5-primary | 1000                | 1 000 000         | 188                 | <b>1</b>           | Boundary warning → confirmed |
| 768      | random-v5-primary  | 1000                | 1 000 000         | 188                 | <b>1</b>           | Boundary warning → confirmed |
| 1024     | counter-v5-primary | 1000                | 1 000 000         | 188                 | 0                  | Pass                         |
| 1024     | random-v5-primary  | 1000                | 1 000 000         | 188                 | 0                  | Pass                         |

### C.5.1.1 Boundary-Warning Details

The primary run produced a total of 2 proportion-item boundary warnings, both in the 768 configuration. The uniformity P-values of both (0.1389 and 0.0273) are above the  $1 \times 10^{-4}$  lower bound; the warning comes only from the pass proportion 979/1000 being 2 counts below the 981/1000 threshold, which is a boundary fluctuation.

| Instance | Input family       | Test item              | uniformity P | Pass proportion | Note                                              |
|----------|--------------------|------------------------|--------------|-----------------|---------------------------------------------------|
| 768      | counter-v5-primary | FFT                    | 0.138860     | <b>979/1000</b> | Proportion-item boundary; uniformity did not fail |
| 768      | random-v5-primary  | NonOverlappingTemplate | 0.027313     | <b>979/1000</b> | Proportion-item boundary; uniformity did not fail |

### C.5.1.2 Targeted Confirmation Results

Running 5000 sequences each for the above 2 items by the pre-registered seed, neither reproduced the warning, confirming that the boundary fluctuation in the primary phase is not a systematic defect.

| Instance | Input family       | Test item              | Number of sequences | Number of warnings | State                  |
|----------|--------------------|------------------------|---------------------|--------------------|------------------------|
| 768      | counter-v5-primary | FFT                    | 5000                | <b>0</b>           | Warning not reproduced |
| 768      | random-v5-primary  | NonOverlappingTemplate | 5000                | <b>0</b>           | Warning not reproduced |

**NIST STS decision: all 6 combinations are covered; the 2 proportion-item boundary warnings have passed targeted confirmation with the pre-registered 5000 sequences and were not reproduced. The conclusion is “the main test passes and targeted confirmation is completed,” not a first-round strict-clean claim.**

### C.5.2 Summary of TestU01 SmallCrush Results

All 6 “instance × input family” combinations pass, with exit codes all 0 and stream rewind counts all 0 (no data reuse occurred).

| Instance | Input family       | Battery    | Number of 32-bit words read | Number of rewinds | State       |
|----------|--------------------|------------|-----------------------------|-------------------|-------------|
| 512      | counter-v5-primary | SmallCrush | 226 972 494                 | 0                 | Pass (PASS) |
| 512      | random-v5-primary  | SmallCrush | 226 896 480                 | 0                 | Pass (PASS) |
| 768      | counter-v5-primary | SmallCrush | 226 723 836                 | 0                 | Pass (PASS) |
| 768      | random-v5-primary  | SmallCrush | 226 944 862                 | 0                 | Pass (PASS) |
| 1024     | counter-v5-primary | SmallCrush | 226 951 680                 | 0                 | Pass (PASS) |
| 1024     | random-v5-primary  | SmallCrush | 227 060 775                 | 0                 | Pass (PASS) |

### C.5.3 Summary of Avalanche and Bit-Independence Results

The per-configuration results of the three instances are all judged PASS. The following table first gives the summary by instance, then the complete details of the 27 configurations.

#### C.5.3.1 Summary by Instance

| Instance | Ideal Hamming distance (bits) | Range of average flip rate | Range of average Hamming distance (bits) | Per-bit 99% CI outlier count | Theoretical expectation |
|----------|-------------------------------|----------------------------|------------------------------------------|------------------------------|-------------------------|
| 512      | 256                           | 0.499665 – 0.500457        | 255.828 – 256.234                        | 2 – 9                        | $\approx 5.1$           |
| 768      | 384                           | 0.499683 – 0.500082        | 383.756 – 384.063                        | 4 – 11                       | $\approx 7.7$           |
| 1024     | 512                           | 0.499738 – 0.500195        | 511.732 – 512.200                        | 8 – 13                       | $\approx 10.2$          |

*Note: each tier instance has 9 configurations, each with 10,000 input pairs, and no output bit falls outside the 45%–55% or 40%–60% bands (all formal criteria are 0). The per-bit 99% confidence-interval outlier count is consistent with the theoretical expectation ( $\approx 1\% \times$  the number of output bits), indicating no systematic per-bit bias.*

#### C.5.3.2 Complete Details

| Instance | Input length | Input bits | Average Hamming distance (bits) | Average flip rate | Flip-rate standard deviation | Per-bit flip-rate interval | Out-of-range 45–55 / 40–60 |
|----------|--------------|------------|---------------------------------|-------------------|------------------------------|----------------------------|----------------------------|
| 512      | 64B          | 512        | 256.0113                        | 0.500022          | 0.021767                     | 0.4837 – 0.5150            | 0 / 0                      |
| 512      | 128B         | 1024       | 256.0134                        | 0.500026          | 0.022243                     | 0.4842 – 0.5149            | 0 / 0                      |
| 512      | $r-1$        | 1023       | 255.8461                        | 0.499699          | 0.022084                     | 0.4850 – 0.5152            | 0 / 0                      |
| 512      | $r$          | 1024       | 255.8284                        | 0.499665          | 0.022106                     | 0.4849 – 0.5150            | 0 / 0                      |
| 512      | $r+1$        | 1025       | 256.2341                        | 0.500457          | 0.021963                     | 0.4857 – 0.5152            | 0 / 0                      |
| 512      | $2r-1$       | 2047       | 255.9740                        | 0.499949          | 0.022249                     | 0.4826 – 0.5152            | 0 / 0                      |
| 512      | $2r$         | 2048       | 255.9350                        | 0.499873          | 0.021732                     | 0.4866 – 0.5125            | 0 / 0                      |
| 512      | $2r+1$       | 2049       | 256.1455                        | 0.500284          | 0.022068                     | 0.4856 – 0.5187            | 0 / 0                      |
| 512      | 1KB          | 8192       | 256.1161                        | 0.500227          | 0.021991                     | 0.4841 – 0.5158            | 0 / 0                      |
| 768      | 64B          | 512        | 384.0392                        | 0.500051          | 0.017800                     | 0.4803 – 0.5175            | 0 / 0                      |
| 768      | 128B         | 1024       | 384.0632                        | 0.500082          | 0.018112                     | 0.4875 – 0.5148            | 0 / 0                      |
| 768      | $r-1$        | 767        | 384.0004                        | 0.500001          | 0.018158                     | 0.4837 – 0.5173            | 0 / 0                      |
| 768      | $r$          | 768        | 383.7563                        | 0.499683          | 0.018169                     | 0.4840 – 0.5155            | 0 / 0                      |

| Instance | Input length | Input bits | Average Hamming distance (bits) | Average flip rate | Flip-rate standard deviation | Per-bit flip-rate interval | Out-of-range 45–55 / 40–60 |
|----------|--------------|------------|---------------------------------|-------------------|------------------------------|----------------------------|----------------------------|
| 768      | $r+1$        | 769        | 383.9156                        | 0.499890          | 0.018191                     | 0.4858 – 0.5190            | 0 / 0                      |
| 768      | $2r-1$       | 1535       | 383.8932                        | 0.499861          | 0.017857                     | 0.4792 – 0.5147            | 0 / 0                      |
| 768      | $2r$         | 1536       | 383.9537                        | 0.499940          | 0.017996                     | 0.4817 – 0.5164            | 0 / 0                      |
| 768      | $2r+1$       | 1537       | 384.0218                        | 0.500028          | 0.018175                     | 0.4847 – 0.5158            | 0 / 0                      |
| 768      | 1KB          | 8192       | 383.9479                        | 0.499932          | 0.018125                     | 0.4813 – 0.5164            | 0 / 0                      |
| 1024     | 64B          | 512        | 511.7918                        | 0.499797          | 0.015768                     | 0.4856 – 0.5160            | 0 / 0                      |
| 1024     | 128B         | 1024       | 512.0815                        | 0.500080          | 0.015684                     | 0.4823 – 0.5142            | 0 / 0                      |
| 1024     | $r-1$        | 511        | 511.9075                        | 0.499910          | 0.015560                     | 0.4843 – 0.5193            | 0 / 0                      |
| 1024     | $r$          | 512        | 512.1995                        | 0.500195          | 0.015765                     | 0.4855 – 0.5180            | 0 / 0                      |
| 1024     | $r+1$        | 513        | 511.7374                        | 0.499744          | 0.015823                     | 0.4835 – 0.5155            | 0 / 0                      |
| 1024     | $2r-1$       | 1023       | 512.0659                        | 0.500064          | 0.015653                     | 0.4851 – 0.5165            | 0 / 0                      |
| 1024     | $2r$         | 1024       | 512.0528                        | 0.500052          | 0.015647                     | 0.4828 – 0.5209            | 0 / 0                      |
| 1024     | $2r+1$       | 1025       | 512.0237                        | 0.500023          | 0.015602                     | 0.4844 – 0.5194            | 0 / 0                      |
| 1024     | 1KB          | 8192       | 511.7316                        | 0.499738          | 0.015615                     | 0.4820 – 0.5175            | 0 / 0                      |

*Low-diversity boundary samples serve only as a boundary diagnostic and do not enter the formal bit-frequency decision.*

## C.6 Conclusion

Under this round’s pre-registered test profile, the conclusions of the three classes of main statistical-randomness tests of AFS-TrEDM-S6 are as follows:

- NIST SP 800-22 STS: all 6 “instance  $\times$  input family” combinations are covered; only 2 proportion-item boundary warnings occurred (768/FFT and 768/Non-overlapping Template), neither reproduced under pre-registered 5000-sequence targeted confirmation, judged as “the main test passes and targeted confirmation is completed.”
- TestU01 SmallCrush: all 6 combinations pass, with zero rewinds.
- Avalanche/bit-independence (SAC/BIC): all 27 formal configurations pass, with an average flip rate close to 0.5, an average Hamming distance about half the number of output bits, and no output bit outside the coarse threshold band.

In summary, the current statistical-randomness tests observe no systematic output-randomness defect in AFS-TrEDM-S6, and its output is consistent with ideal random behavior across multiple dimensions such as frequency, spectrum, template, random walk, diffusion, and avalanche.

## C.7 Security Boundary and Limitations

Statistical-randomness tests can serve only as experimental evidence of output randomness and diffusion behavior, and cannot replace cryptographic security analyses such as collision, preimage, second-preimage, differential/linear, integral, rotational, and TrEDM/keyless-CI. The specific boundaries of this report include:

- The conclusions are limited to the above pre-registered test profile and sample scale, and do not constitute a complete cryptographic security proof of the algorithm.

- TestU01 executes only SmallCrush, not Crush or BigCrush, so no judgment is made about larger-scale batteries.
- The NIST STS conclusion is “passed and targeted confirmation completed,” not a first-round strict-clean all-clear claim.
- Passing statistical-randomness tests does not imply the ability to resist structured cryptanalysis; the latter must be given independently by dedicated differential/linear/integral/algebraic and other analyses.

## C.8 Reproducibility Information

The key parameters and evidence index of this test are as follows; together with the pre-registered seed registry and the SHA-256 manifest, they enable full reproduction.

| Item                                | Content                                                                                                                                                                        |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NIST primary parameters</b>      | 1000 sequences $\times$ 1,000,000 bits per combination, all (188 sub-items), $\alpha = 0.01$                                                                                   |
| <b>NIST confirmation parameters</b> | 5000 sequences each for boundary items (TARGETED_CONFIRM5000)                                                                                                                  |
| <b>TestU01 parameters</b>           | SmallCrush, 1 GiB stream per combination, zero rewinds                                                                                                                         |
| <b>Avalanche/BIC parameters</b>     | 10,000 input pairs per configuration, 9 input lengths $\times$ 3 tier instances = 27 configurations                                                                            |
| <b>Seed registry</b>                | RND-60_SEED_REGISTRY.yaml (formal), RND-62_CONFIRMATION_SEED_REGISTRY.yaml (confirmation)                                                                                      |
| <b>Main decision evidence</b>       | RND-61_NIST_PRIMARY_SUMMARY.csv、RND-61_NIST_PRIMARY_WARNINGS.csv、RND-62_TARGETED_CONFIRMATION_MATRIX.csv、RND-64_TESTU01_SUMMARY.csv、RND-65_BIT_INDEPENDENCE_FORMAL_SUMMARY.csv |
| <b>Integrity manifest</b>           | RND-67A_STABLE_EVIDENCE_MANIFEST.sha256 (per-file SHA-256, verification passed)                                                                                                |

## Appendix D Dedicated Verification of the 512 6+6 Front-Half-Round and Aligned-Split Risk

### D.1 Verification Scope and Evidence-Chain Overview

This appendix gives the dedicated verification evidence for the 6+6-tier front-half-round and aligned-split risk of AFS-TrEDM-512. The object of verification is AFS- $p$ -S6[1600,12] with  $g=6$  and  $h=6$ , the linear layer being AFS-LMDS-1600-S6 and the feed-forward being the aligned-TrEDM capacity-window split. Its final status is a scoped claim ledger (PASS\_SCOPED\_CLAIM\_LEDGER), i.e., checkable evidence for the claimed scope, rather than a global security proof (not PASS\_GLOBAL). The evidence-chain overview of each stage is given in Table D-1.

Table D-1. Evidence-chain overview of the dedicated aligned-split risk verification

| Stage | Target       | Conclusion            | Proof type     | Effect on the 512 first-6-rounds / split risk |
|-------|--------------|-----------------------|----------------|-----------------------------------------------|
| C0    | Active-S-box | $N_3 \geq 12$ , 12/12 | PASS_DIRECT_GJ | Excludes                                      |

| Stage | Target                                                              | Conclusion                                                                                    | Proof type                                                         | Effect on the 512 first-6-rounds / split risk                                                                       |
|-------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
|       | lower bound of any 3-round window of S6                             | phase/direction closed                                                                        |                                                                    | narrow-diffusion windows of the linear layer; supports the wide-trail window bound (see also §3.3.1 and Appendix B) |
| C1    | 512 arbitrary-starting-state split differential/linear verification | Differential $N_D \geq 29$ , linear $N_L \geq 48$                                             | Differential PASS_DIRECT_GJ; linear $c/2$ PASS_BY_WIDE_TRAIL_BOUND | Directly covers the low-activity split channel of $g_6$ + feed-forward + $h_6$                                      |
| C2    | Semi-free/free-start low-activity two-path relation                 | The low-activity relation is excluded by the C1 subset bound                                  | PASS_BY_C1_SUBSET_BOUND                                            | Excludes, within the claimed scope, a semi-free/free-start low-activity relation bypassing C1                       |
| C3    | keyless-CI low-activity two-path relation                           | The low-activity KCI relation is excluded by the C1 subset bound                              | PASS_LOW_ACTIVE_RELATION (with a scoped single-state note)         | Covers, within the claimed scope, the KCI/splice low-activity relation                                              |
| C4    | rotational equivariance / RX / RDL                                  | The universal equivariance for $s=1\dots 63$ is refuted; low-activity RX/RDL is covered by C1 | PASS_STRUCTURAL_EQUIVARIANCE_REFUTED / PASS_BY_C1_SUBSET_BOUND     | Excludes global rotational equivariance and, within the claimed scope, the low-activity rotational skeleton         |
| C5    | bounded integral / cube                                             | F1–F5 bounded cube families have no designated zero witness                                   | PASS_SCOPED_BOUNDED_INTEGRAL_ONLY                                  | Provides supplementary experimental evidence for the $S$ /digest window; does not claim universal DP                |

| Stage | Target                      | Conclusion                                        | Proof type             | Effect on the 512 first-6-rounds / split risk                                     |
|-------|-----------------------------|---------------------------------------------------|------------------------|-----------------------------------------------------------------------------------|
| C6    | subthreshold hull emptiness | The hull skeleton below the C1 threshold is empty | PASS_SUBTHRESHOLD_ONLY | Excludes subthreshold-aggregation risk; does not claim a full near-threshold hull |

The verified risk is not “inverting the 6-round permutation in an arbitrary way,” but a more specific split risk: an attacker constructs a low-activity differential/mask/relation that maintains a low active-S-box cost while passing through  $g_6$ , exhibits a feed-forward cancellation at  $V[S]=g_6(X)[S] \oplus P_S(X)$ , and still satisfies the digest/ $S$  window constraint after passing through  $h_6$ , thereby forming a preimage-type or relation-type shortcut below the 512 design threshold. This appendix takes as its main thread the active-S-box lower bound, arbitrary-starting-state splitting, relation-subset reduction, rotational-structure checking, and bounded integral/hull supplementary evidence. The target tier parameters are given in Table D-2.

Table D-2. Dedicated verification parameters for the 6+6 tier of 512

| Item                     | Value                                                                                                   |
|--------------------------|---------------------------------------------------------------------------------------------------------|
| Instance                 | AFS-TrEDM-512                                                                                           |
| State width              | $b = 1600$ bits                                                                                         |
| Message rate             | $r = 1024$ bits                                                                                         |
| Capacity-window size     | $c = 576$ bits                                                                                          |
| Output length            | $d = 512$ bits                                                                                          |
| Capacity-window position | $S = A[16] \dots A[24]$                                                                                 |
| Split parameters         | $g = 6, h = 6$                                                                                          |
| Round function           | AFS- $p$ -S6[1600, 12]                                                                                  |
| Linear layer             | AFS-LMDS-1600-S6                                                                                        |
| Bit-numbering convention | Specification bit $z$ corresponds to C uint64_ $t$ bit $63-z$ , i.e., MSB-first specification numbering |

## D.2 Model, Thresholds, and Fail-Closed Acceptance Rules

The single-block core of the aligned-TrEDM split is abstracted into the following semantics ( $P_S$  denotes the projection onto the capacity window, and  $E_S$  denotes re-embedding the capacity window into the state):

```

X      = current state after rate absorption
T      = P_S(X)
U      = g_6(X)
V[R]   = U[R]
V[S]   = U[S] xor T
Y      = h_6(V)
Digest = left_512(P_S(Y))

```

All split models must explicitly include  $T=P_S(X)$  and  $V[S]=U[S] \oplus T$ ; a model covering only the rate input difference (rate-only) may not be used as an arbitrary-starting-state verification. The active-S-box thresholds use the conservative single-box weights  $w_D=10$  and  $w_L=6$  of §3.3.1. For the 512 tier,  $c/2 = 288$  and  $d/2 = 256$ , from

which the per-direction thresholds are given in Table D-3.

Table D-3. Active-S-box thresholds for the 512 tier

| Target | Differential active threshold | Linear active threshold |
|--------|-------------------------------|-------------------------|
| $d/2$  | $N_D \geq 26$                 | $N_L \geq 43$           |
| $c/2$  | $N_D \geq 29$                 | $N_L \geq 48$           |

The core acceptance target of C1 is: the arbitrary-starting-state split reaches  $N_D \geq 29$  in the differential direction and  $N_L \geq 48$  in the linear direction. All stages of this appendix obey the fail-closed acceptance rules (Table D-4): solver timeout/unknown, placeholder models, supporting-only artifacts, and witnesses not independently reviewed may not be promoted to PASS; a bounded search claims only a bounded scope and may not claim a universal proof; the claim ledger strictly distinguishes claim/evidence/non-claim and does not use PASS\_GLOBAL.

Table D-4. Fail-closed acceptance rules

| Event                          | Allowed status                                         | Disallowed action                          |
|--------------------------------|--------------------------------------------------------|--------------------------------------------|
| Solver timeout                 | BLOCKED_TIMEOUT or residual                            | May not be promoted to PASS                |
| Solver UNKNOWN                 | BLOCKED_UNKNOWN                                        | May not be promoted to PASS                |
| Placeholder model / scope-only | BLOCKED_MODEL                                          | May not be used as verification basis      |
| supporting-only artifact       | PASS_SUPPORTING_ONLY                                   | May not replace a proof                    |
| SAT witness                    | Must be reviewed by an independent checker             | Without review, may not serve as FAIL/PASS |
| subset lemma                   | Must give the logical-inclusion relation and a checker | May not replace an unrelated model         |
| bounded search                 | May only claim a bounded scope                         | May not claim a universal proof            |
| claim ledger                   | Must distinguish claim/evidence/non-claim              | May not use PASS_GLOBAL                    |

## D.3 Individual Verifications

### D.3.1 C1: 512 Arbitrary-Starting-State Split Verification

C1 is the most critical technical verification item in this dedicated evidence package, directly covering the case where an attacker enters the split structure from an arbitrary internal-state perturbation, with the capacity-window feed-forward explicitly modeled. The differential direction is modeled as ( $\Delta X$  is an arbitrary-starting-state nonzero difference, not limited to rate-only):

$$\begin{aligned}
 \Delta T &= P\_S(\Delta X) \\
 \Delta U &= g\_6\_diff(\Delta X) \\
 \Delta V[R] &= \Delta U[R] \\
 \Delta V[S] &= \Delta U[S] \text{ xor } \Delta T \\
 \Delta Y &= h\_6\_diff(\Delta V)
 \end{aligned}$$

The linear direction uses transpose propagation (rather than incorrectly reusing differential propagation), and the transpose contribution of the feed-forward is reflected as  $E_S(\beta V[S])$  fed back into the input capacity window:

$$\begin{aligned}
 \beta V &= h\_6^{^T}(\beta Y) \\
 \beta U &= \beta V \\
 \beta X &= g\_6^{^T}(\beta U) \text{ xor } E\_S(\beta V[S])
 \end{aligned}$$

The goal is to prove that there is no split differential/linear channel with an active-S-box count below the

threshold. The results are given in Table D-5.

Table D-5. C1 arbitrary-starting-state split verification results

| Target                                                    | Proof obligation         | Result        | Proof type                                  |
|-----------------------------------------------------------|--------------------------|---------------|---------------------------------------------|
| Differential $c/2$                                        | Exclude active $\leq 28$ | $N_D \geq 29$ | PASS_DIRECT_GJ                              |
| Linear $d/2$                                              | Exclude active $\leq 42$ | $N_L \geq 43$ | PASS_DIRECT_GJ                              |
| Linear $c/2$                                              | Proof $N_L \geq 48$      | $N_L \geq 48$ | PASS_BY_WIDE_TRAIL_BOUND<br>(depends on C0) |
| Rate-only substituting<br>for arbitrary starting<br>state | Not allowed              | Not used      | fail-closed                                 |

Conclusion: under the claimed active-S-box model, the 512 6+6 split has no differential/linear low-activity path below the  $c/2$  threshold.

### D.3.2 C2/C3: Semi-Free-Start and keyless-CI Low-Activity Relation Verification

C2 focuses on the two-path relation  $F(H, M)$  and  $F(H', M')$ : semi-free-start allows the initial chain value to be freely chosen but with a shared condition for the two paths, and free-start allows a looser initial-state condition. The conclusion is: within the claimed scope, the low-activity semi-free/free-start relation is a subset of the C1 arbitrary-starting-state split low-activity model and is therefore excluded by the C1 lower bound (PASS\_BY\_C1\_SUBSET\_BOUND); this verification gives only a scoped claim for the low-activity two-path relation and does not claim that all semi-free/free-start relations are globally unsatisfiable.

C3 distinguishes two classes of objects: an attack-meaningful low-activity two-path keyless-CI/splice relation; and a single-state preimage consistency equation:

$$\begin{aligned} V &= h_{-6}^{-1}(Y) \\ U &= V \text{ xor } E_S(T) \\ X &= g_{-6}^{-1}(U) \\ P_S(X) &= T \end{aligned}$$

A single-state consistency equation cannot be set as a security target that “should be UNSAT,” so it is marked as a scoped note rather than a global impossibility claim; for the attack-meaningful low-activity two-path relation, the conclusion is still excluded by the C1 subset bound.

### D.3.3 C4: Rotational-Structure Verification

C4 excludes two classes of rotational structures. First, universal rotational equivariance, i.e.,  $F(\text{rot}_s(X)) = \text{rot}_s(F(X))$  holds for all  $X$ : a counterexample witness and checker are given one by one for  $s = 1 \dots 63$ , with status PASS\_STRUCTURAL\_EQUIVARIANCE\_REFUTED. Second, a low-activity rotational-XOR (RX) or rotational differential-linear (RDL) skeleton: via a subset lemma it is brought into the scope of the C1 arbitrary-starting-state split low-activity model and is covered by the C1 lower bound (PASS\_BY\_C1\_SUBSET\_BOUND). This conclusion is bounded/scoped and is not equivalent to excluding all forms of rotational attacks.

### D.3.4 C5/C6: Supplementary Evidence and Explicit Non-Claims

C5 performs a bounded exhaustive search over predefined cube families: it covers the F1–F5 declared families, with 43,840 cubes and 19,326,080 assignments in total, and the number of designated full-vector/digest-prefix/structured-mask zero witnesses is 0, with status PASS\_SCOPED\_BOUNDED\_INTEGRAL\_ONLY. This evidence only indicates that no designated witness was found in the declared bounded families and does not claim

a universal bit-based division-property/integral proof.

C6 supports subthreshold hull emptiness from the low-activity lower bounds of C1/C0: a trail skeleton below the threshold does not exist, so the corresponding subthreshold hull is empty (PASS\_SUBTHRESHOLD\_ONLY); the aggregation effect of near-threshold or above-threshold trails is not globally closed. The items explicitly listed as non-claim/residual are given in Table D-8.

## D.4 Claim Ledger

This appendix strictly distinguishes the claimable machine-checked claims, the limited bounded-search evidence, and the explicit non-claim/residual, given in Table D-6, Table D-7, and Table D-8, respectively.

Table D-6. Machine-checked claims

| Claim                                         | State                                                                                                          |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| C0 S6 3-round windows $N_3 \geq 12$           | R4 PASS_DIRECT_GJ, 12/12 phase/direction (see also §3.3.1 and Appendix B)                                      |
| C1 512 arbitrary-starting-state split         | R4 differential $c/2$ PASS_DIRECT_GJ, linear $d/2$<br>PASS_DIRECT_GJ, linear $c/2$<br>PASS_BY_WIDE_TRAIL_BOUND |
| C2/C3 low-activity relation certificates      | R5 PASS_BY_C1_SUBSET_BOUND / scoped single-state note                                                          |
| C4 rotational equivariance and bounded RX/RDL | R4/R5 scoped PASS                                                                                              |

Table D-7. Scoped bounded-search evidence

| Claim                                                  | State                                              |
|--------------------------------------------------------|----------------------------------------------------|
| C5 bounded F1–F5 no-witness cube families              | PASS_SCOPED_BOUNDED_INTEGRAL_ONLY                  |
| C6 subthreshold hull emptiness                         | PASS_SUBTHRESHOLD_ONLY_WITH_EXPLICIT_HULL_RESIDUAL |
| Reference-implementation selftest and solver preflight | PASS_SUPPORTING_ONLY / PASS_DIRECT_GJ preflight    |

Table D-8. Explicit non-claim / residual

| Non-claimed item                                                    | State                                   |
|---------------------------------------------------------------------|-----------------------------------------|
| C5 universal bit-based division-property/integral proof             | EXPLICIT_NONCLAIM_RESIDUAL              |
| C5 F6 high-dimensional cube exhaustion                              | EXPLICIT_NONCLAIM_RESOURCE              |
| C6 full near-threshold differential hull aggregation                | EXPLICIT_NONCLAIM_RESIDUAL              |
| C6 full near-threshold linear triangle-correlation hull aggregation | EXPLICIT_NONCLAIM_RESIDUAL              |
| 768/1024 differential profile extension                             | EXPLICIT_PROFILE_DIFF_NONCLAIM_RESIDUAL |

## D.5 Evidence Availability and Reproduction

The complete checkable evidence package for each verification item described in this appendix — including the CNF/SMT models of each stage, solver logs, independent witness checkers, status JSON, audit records, and the sha256 integrity manifest, based on CryptoMiniSat 5.14.5 (with Gauss-Jordan enabled) and a documented fail-closed schedule — is maintained according to the reproducible-archive standard for class-(ii) evidence of this specification. Since the electronic submission materials for this solicitation are four fixed classes of materials — the algorithm text, the implementation code, the test vectors, and the related claims — the above raw evidence

package is not included in the submission archive, but is provided during the public-comment phase and is available on request; the model-generation, scheduling, and review commands are all documented, and a third party can independently rebuild and re-verify all UNSAT decisions of this appendix according to the model definitions and thresholds of D.2.

## Appendix E Relation-Level keyless-CI Reduction, Structure-Level Security Bounds, and Constant Tables

### E.1 Scope of This Appendix and Its Relation to Appendix D

This appendix fully develops the relation-level keyless-CI reduction of §3.2.3–§3.2.10 and supplements the generic quantum-lifting proof described in §3.3.6. The contents include: the dual-oracle  $(g, h)$  graph relations, the classical main theorems, chain-divergence localization, alignment cutoff, the suffix-freeness of  $\text{frame}_d$ , the explicit constant tables, the correct point counting of the Cojocaru generic lifting, and the applicability boundary of the compressed-oracle/quantum-indistinguishability path.

Appendix E and Appendix D are at different levels. Appendix E explains how hash collision, preimage, and second preimage reduce to the specified relations if the underlying public permutation pair is intractable on those relations, and further gives the relation-hit probability in the dual-random-permutation model. Appendix D, for the finite-round structure of the actual AFS- $p$ -S6, uses solvers and component cryptanalysis to screen for low-activity relations and the 6+6 split channel. The risk that the actual permutation deviates from the ideal model is uniformly denoted by  $\varepsilon_{\text{str}}$ .

### E.2 Notation, the Dual-Permutation TrEDM Core, and the Relation Families

Let the digest length be  $d \in \{512, 768, 1024\}$ , the state width  $b = 1600$ , the rate  $r = 1600 - c$ , and the capacity and alignment-window length  $c = a = |S| = d + 64$ . Denote by  $\pi_S: \{0, 1\}^b \rightarrow \{0, 1\}^c$  the  $S$  projection, and by  $\iota_S$  the map that embeds  $c$  bits into  $S$  and fills the other positions with zero.

$$C_{g,h}(X) = h(g(X) \oplus \iota_S(\pi_S(X))).$$

For a single core input  $X$ , define the intermediate values

$$G = g(X), \quad U = G \oplus \iota_S(\pi_S(X)), \quad Y = h(U) = C_{g,h}(X).$$

The complete core-graph witness is  $\omega(X) = ((X, G), (U, Y))$ , where the first pair belongs to  $\text{Graph}(g)$  and the second pair belongs to  $\text{Graph}(h)$ . For message blocks  $B_1, \dots, B_\ell$ , the absorbing chain is

$$W_i = X_{i-1} \oplus (B_i \parallel 0^c), \quad X_i = C_{g,h}(W_i), \quad X_0 = \text{IV}.$$

$$H(M) = L_d(\pi_S(X_\ell)).$$

The maximum number of frame blocks satisfies

$$\ell_{\max}(r) = \lceil (2^{64} + 128)/r \rceil < 2^{56}.$$

The original number of core queries of the classical adversary is  $\sigma$ ; the reduction recomputes at most two output message chains, with the total count denoted  $N = \sigma + 2\ell_{\max}$ .

**Relation families.** For  $v \in \{d, c\}$ , define

$$R_{co,v} = \{ (\omega(X), \omega(X')) : X \neq X' \wedge L_v(\pi_S(Y)) = L_v(\pi_S(Y')) \}.$$

$$R_{\text{pre}}(Z) = \{ \omega(X) : L_d(\pi_S(Y)) = Z \}.$$

$$R_{\text{spr}}(X *, v) = \{ \omega(X) : X \neq X * \wedge L_v(\pi_S(Y)) = L_v(\pi_S(Y *)) \}.$$

The second-preimage multi-target relation uses the target chain  $X_j^*, \dots, X_\ell^*$ : the first  $\ell-1$  targets are compared on the  $c$ -bit  $S$  window, and the last target on the  $d$ -bit digest. At the core level,  $R_{\text{pre}}$  requires one candidate core witness,  $R_{\text{co}}$  requires two, and  $R_{\text{spr}}$  requires both the target and the candidate in the conservative verification game; expanded to  $g/h$  graph points, they are 2, 4, and 4 underlying-oracle graph points, respectively.

Bad events. Denote by  $\text{BAD}_{\text{rep}}$  the repetition of a  $g$  or  $h$  input at a position that should be fresh, by  $\text{BAD}_{\text{align}}$  the equality of different chain states on  $S$  or a hit on the IV capacity, and by  $\text{BAD}_{\text{inst}}$  the deviation of the actual AFS- $p$ -S6 from the dual-random-permutation model by exploiting the concrete round structure. Let  $\text{BAD} = \text{BAD}_{\text{rep}} \cup \text{BAD}_{\text{align}} \cup \text{BAD}_{\text{inst}}$ . All uniformity calculations are performed under the condition  $\neg \text{BAD}$ .

### E.3 Classical Main Theorems (with Explicit Coefficients)

Theorem E-1 (collision resistance). In the dual-random-permutation model, any classical adversary inducing  $N$  core candidates satisfies

$$\text{Adv}_H^{\text{coll}} \leq C(N, 2) \cdot 2^{-d} + 2C(N, 2) \cdot 2^{-c} + \varepsilon_{\text{str, coll}}.$$

Therefore, when  $c \geq h$ , the main term is  $N^2/2^d$ , and the classical collision-resistance exponent is  $d/2$ .

Theorem E-2 (preimage resistance). For any fixed target digest  $Z \in \{0, 1\}^d$ ,

$$\text{Adv}_H^{\text{pre}}(Z) \leq N \cdot 2^{-d} + 2N\ell_{\text{max}} \cdot 2^{-c} + \varepsilon_{\text{str, pre}}.$$

The first term is the designated digest hit of the last core, and the second term covers the  $c$ -bit  $S$  window hit required to connect a legitimate chain from any intermediate position.

Theorem E-3 (second-preimage resistance). Suppose the target message contains  $\ell \leq \ell_{\text{max}}$  frame blocks, then

$$\text{Adv}_H^{\text{spr}} \leq N \cdot 2^{-d} + 2N\ell \cdot 2^{-c} + 2C(\ell, 2) \cdot 2^{-c} + \varepsilon_{\text{str, spr}}.$$

The three terms correspond, in order, to the direct digest hit, the multi-target  $S$  window hit of the candidate chain connecting to the target chain, and the internal capacity repetition/IV hit of the target chain itself. From  $c = h + 64$  and  $\ell_{\text{max}} < 2^{56}$ , we have

$$c - \log_2 \ell_{\text{max}} > d + 8.$$

$$(S_{\text{coll, cl}}, S_{\text{pre, cl}}, S_{\text{spr, cl}}) = (d/2, d, d).$$

### E.4 Lemma E-1 (Chain-Divergence Localization) and Its Complete Proof

Lemma E-1. Suppose  $M \neq M'$  and  $H(M)=H(M')$ , and all core evaluations of the two message chains can be recorded or recomputed by the reduction. Then at least one of the following events occurs: (a) two distinct last-core inputs are equal on the  $d$ -bit digest window, giving a  $R_{\text{co}, d}$  witness; (b) two distinct chain states are equal on the full  $S$  window, giving a  $R_{\text{co}, c}$  or  $\text{BAD}_{\text{align}}$  witness; (c) the  $S$  window of some intermediate chain state equals the IV capacity, giving an IV-hit event; (d) the two frames are completely identical, contradicting  $M \neq M'$ .

Proof. Step 1 (terminal core inputs). Let the last core inputs be  $W_\ell, W'_\ell$ , respectively. If the two are distinct but the final digests are equal, then  $L_d(\pi_S(C(W_\ell))) = L_d(\pi_S(C(W'_\ell)))$ , directly giving a  $R_{\text{co}, d}$  witness.

Step 2 (advancing forward with the same core input). If  $W_\ell = W'_\ell$ , then  $\pi_S(X_{\ell-1}) = \pi_S(X'_{\ell-1})$  follows from  $W_i = X_{i-1} \oplus (B_i \parallel 0^c)$ . If  $X_{\ell-1} \neq X'_{\ell-1}$ , then different chain states are already aligned on  $S$ , belonging to (b). If the two full states and the corresponding blocks are both identical, then the previous pair of core inputs can

continue to be compared forward. If the full states are identical but the blocks differ, then the next core input will be different, returning to Step 1 or producing a  $S$  alignment event under rate compensation.

Step 3 (first difference or reaching the chain head). Repeating Step 2, the first encounter of a different core input gives  $R_{co,d}$  or  $R_{co,c}$ ; the first encounter of a different predecessor state but the same  $S$  window gives  $BAD\_align$ . If the longer chain still has a prefix after the shorter chain reaches the IV, then equal subsequent core inputs only require some intermediate state of the longer chain to satisfy  $\pi_S(X_\ell) = \pi_S(IV)$ , and the rate difference can be compensated by the next message block, thus forming (c) which cannot be removed by suffix-freeness. If the two chains reach the IV at the same time and all frame blocks are identical, then the injectivity of  $frame_d$  implies  $M=M'$ , a contradiction. QED.

An important correction. Suffix-freeness excludes the pure length extension where “the frame blocks themselves form a proper suffix,” but does not exclude a capacity-IV hit formed by rate compensation. Therefore the second  $2C(N, 2)2^{-c}$  term in Theorem E-1 must be retained; deleting it to zero would miss legitimate chain-alignment branches.

### E.5 Lemma E-2 (Alignment Cutoff) and Its Complete Proof

Lemma E-2. Under the condition that the feed-forward window, the chain-value window, and the squeezing window are all  $S$ , a second-preimage adversary cannot directly set the digest by manipulating only the rate region. For a target chain containing  $\ell$  blocks, under the  $\neg BAD$  condition, the success probability of  $N$  candidates is at most

$$N \cdot 2^{-d} + N(\ell - 1) \cdot 2^{-c}.$$

Proof. Step 1 (output dependence). The digest is  $L_d(\pi_S(X_\ell))$  and does not read the rate region. Therefore the rate-region degrees of freedom can only affect the digest when they change the  $S$  window after passing through  $g$ , the mid-section feed-forward, and  $h$ ; there is no bypass from the rate region to the digest.

Step 2 (fresh  $h$  input). For a candidate core input  $W$ , the input to  $h$  is  $U = g(W) \oplus \iota_S(\pi_S(W))$ . If  $U$  has not been queried, then the output of the random permutation  $h$  on  $U$  is uniform in the remaining output set, and the probability that its  $S$  projection hits a fixed  $c$ -bit target is at most about  $2^{-c}$ , and the probability of hitting a fixed  $d$ -bit digest is at most about  $2^{-d}$ . If  $U$  has been queried, then the adversary has produced a repeated  $h$  input; this repetition either gives a target-relation witness or belongs to  $BAD\_rep$ .

Step 3 (number of targets). The final state of the target chain provides one  $d$ -bit target, and the first  $\ell-1$  internal states provide at most  $\ell-1$   $c$ -bit targets. Applying the union bound to  $N$  candidates gives  $N2^{-d} + N(\ell - 1)2^{-c}$ . Internal repetition and IV hits of the target chain are counted separately in  $2C(\ell, 2)2^{-c}$ . QED.

Corollary E-1 (misalignment-channel cutoff). Any successful digest manipulation must translate into a  $S$  window hit; therefore the direct misalignment channel that exists when “the feed-forward is on the capacity and the output is on the rate” does not apply to AFS-TrEDM. This corollary gives a structural necessary condition and does not claim that the actual  $g/h$  output is absolutely uniform; the concrete deviation is still covered by  $\epsilon_{str}$ .

### E.6 Proposition E-1 (Frame-Encoding Suffix-Free) and Its Complete Proof

$$frame_d(M) = M // 1 // 0^j // enc64(|M|) // enc16(d) // enc16(r) // enc16(c) // enc16(version).$$

Proposition E-1. For a fixed instance  $(d, r, c, version)$ ,  $frame_d$  satisfies: (i) injective; (ii) block-level suffix-free.

Proof. The last 128 bits of metadata are at fixed positions, where  $enc64(|M|)$  uniquely determines the message length; taking the first  $|M|$  bits of the frame recovers  $M$ , hence injective. If  $frame_d(M')$  is a proper suffix of  $frame_d(M)$ , then their last 128 bits are equal, whence  $|M'|=|M|$ . For a fixed instance, both frame lengths are

$r \cdot \lceil (|M| + l + 128)/r \rceil$  with the same number of blocks, which cannot form a proper suffix, a contradiction. QED.

Applicability boundary. This proposition guarantees that pure frame-suffix extension does not occur, but places no constraint on the chain alignment of “different predecessor states having the same  $S$  window, with the rate difference compensated by the next block”; the latter is already retained in the capacity bad-event terms of E.4 and E.5.

E.1–E.6 give the relation-level deterministic reduction: it answers how a hash success event is localized to a  $g/h$  graph-relation witness. The following E.7–E.13 fully retain and develop the original structure-level idealized probability derivation: it answers how the hit probabilities of these witnesses are counted in the ideal model. The two parts are sequentially connected, not mutually substitutive.

### E.7 Structure-Level Idealization Model, Scope, and Non-Claims

This appendix gives the parameter derivation of the classical security strength and the quantum claimed strength in §3.2. The object of analysis is the  $g/h$  split aligned-TrEDM core defined in §1.10, not a single call of a single public permutation  $P$ . To avoid confusion, this appendix does not reduce  $C_{g,h}$  to  $P(X) \oplus$  feed-forward, nor does it claim that the  $g/h$  split core has obtained a tight quantum-relation reduction under the single-permutation baseline.

The structure-level idealization model assumes: when no repeated input, capacity-window alignment, internal chain repetition, or concrete-permutation structural bad event is triggered, the  $D_d$  output of a new core input is approximately uniform  $d$  bits, and the  $D_c$  output is approximately uniform  $c$  bits. The risk that the actual AFS- $p$ -S6 deviates from this model is denoted  $\varepsilon_{\text{str}}$ , supported by the concrete cryptanalysis of §3.3 and Appendix D.

$$\varepsilon_{\text{str}} = \varepsilon_{\text{str,coll}} + \varepsilon_{\text{str,pre}} + \varepsilon_{\text{str,spr}}.$$

Therefore, what this appendix proves is: under the structure-level idealization model plus the  $\varepsilon_{\text{str}}$  deviation term, the classical strength of the three AFS-TrEDM instances is  $(d/2, d, d)$ , and the quantum claimed strength is  $(d/3, d/2, d/2)$ .

### E.8 Basic Notation and the Block Chain

Let  $S$  be the capacity alignment window,  $\pi_S$  the  $S$  projection, and  $\iota_S$  the  $S$  embedding. The single core, the  $d$ -bit digest core, and the  $c$ -bit capacity core are defined respectively as

$$C_{g,h}(X) = h(g(X) \oplus \iota_S(\pi_S(X))),$$

$$D_d(X) = L_d(\pi_S(C_{g,h}(X))), \quad D_c(X) = \pi_S(C_{g,h}(X)).$$

For the  $L$   $r$ -bit blocks  $B_1, \dots, B_L$  parsed from  $\text{frame}_d(M)$ , the absorbing chain is

$$W_i = X_{i-1} \oplus (B_i \parallel 0^c), \quad X_i = C_{g,h}(W_i), \quad X_0 = \text{IV}.$$

$$H(M) = L_d(\pi_S(X_L)) = D_d(W_L).$$

The maximum number of blocks is given by §2.2.2:

$$\ell_{\max}(r) = \lceil (2^{64} + 128)/r \rceil < 2^{56}.$$

Taking  $c=d+64$ , then

$$c - \log_2 \ell_{\max} > d + 8.$$

All subsequent second-preimage bounds hold for  $\ell \leq \ell_{\max}$ .

### E.9 Lemma E-3 (Single-Core Hit Probability)

Lemma. Let  $W$  be a new core input that triggers no structural bad event. Under the structure-level idealization model, for any fixed  $Z \in \{0,1\}^d$  and any fixed  $Y \in \{0,1\}^c$ , we have

$$\Pr[D_d(W) = Z] = 2^{-d}, \quad \Pr[D_c(W) = Y] = 2^{-c}.$$

Proof. The  $g$  stage diffuses  $W$  to the full state, the mid-section EDM feed-forward XORs  $\pi_S(W)$  into the  $S$  window, and the  $h$  stage then diffuses this constraint to the full state. Under the idealized baseline, the  $S$  window of a new input that triggers no repetition or alignment bad event behaves as uniform  $c$  bits after the  $h$  output; taking its leftmost  $d$  bits gives uniform  $d$  bits. If the concrete permutation has an exploitable deviation, it is not hidden in this uniform term but counted in  $\varepsilon_{\text{str}}$ . QED.

#### E.10 Theorem E-4 (Classical Collision Resistance)

Theorem. Suppose a classical adversary induces  $\sigma$  core evaluations, then

$$\text{Adv}_{\text{coll,cl}}(\sigma) \leq \sigma(\sigma - 1) \cdot 2^{-d-l} + K_{\text{coll}} \cdot \sigma^2 \cdot 2^{-c} + \varepsilon_{\text{str,coll}}(\sigma).$$

Proof. If two distinct messages obtain the same digest, compare forward from the end along the two absorbing chains. If the last divergence occurs in the final digest window, then by Lemma E-3 a  $d$ -bit digest collision is obtained; the union bound over  $\sigma$  candidate digests is  $\text{binom}(\sigma, 2)2^{-d}$ . If the chains realign at an earlier position, then a  $c$ -bit event such as capacity-window equality, capacity-window IV hit, or internal capacity repetition must occur; these events have at most  $O(\sigma^2)$  candidate pairs and are therefore covered by  $K_{\text{coll}}\sigma^2 2^{-c}$ . The suffix-freeness of  $\text{frame}_d$  excludes pure frame-suffix length extension but does not remove capacity-IV-hit-type events, so such events are still retained in the  $c$ -bit term. The concrete-permutation deviation is counted in  $\varepsilon_{\text{str}}^{\text{coll}}$ . QED.

Substituting  $c=d+64$ , the second term is 64 bits lower than the main birthday term, so the classical collision resistance is  $d/2$  bits.

#### E.11 Theorem E-5 (Classical Preimage Resistance)

Theorem. Given a target digest  $Z \in \{0,1\}^d$ , if a classical adversary induces  $\sigma$  candidate core inputs, then

$$\text{Adv}_{\text{pre,cl}}(\sigma) \leq \sigma \cdot 2^{-d} + K_{\text{pre}} \cdot \sigma \ell_{\text{max}} \cdot 2^{-c} + \varepsilon_{\text{str,pre}}(\sigma).$$

Proof. The direct-hit path requires some final candidate  $W$  to satisfy  $D_d(W)=Z$ . By Lemma E-3, each candidate has probability  $2^{-d}$ , and the union over  $\sigma$  gives  $\sigma 2^{-d}$ . If the adversary tries to connect to some legitimate chain from an intermediate state, then the message block can only change the rate region, and the capacity alignment window must hit some fixed  $c$ -bit target; for each candidate there are at most  $\ell_{\text{max}}$  connection positions, so this path is covered by  $K_{\text{pre}}\sigma \ell_{\text{max}} 2^{-c}$ . The concrete-permutation deviation is counted in  $\varepsilon_{\text{str}}^{\text{pre}}$ . QED.

From  $\ell_{\text{max}} < 2^{56}$  and  $c=d+64$ ,

$$\sigma \ell_{\text{max}} \cdot 2^{-c} < \sigma \cdot 2^{-d-8}.$$

The capacity-connection path does not dominate, so the classical preimage resistance is  $d$  bits.

#### E.12 Theorem E-6 (Classical Second-Preimage Resistance)

Theorem. Suppose the target message  $M$  contains  $\ell \leq \ell_{\text{max}}$  blocks after  $\text{frame}_d$ , and a classical adversary induces  $\sigma$  candidate core inputs, then

$$\text{Adv}_{\text{spr,cl}}(\sigma, \ell) \leq \sigma \cdot 2^{-d} + K_{\text{spr}} \cdot \sigma \ell \cdot 2^{-c} + K_{\text{bad}} \cdot \ell^2 \cdot 2^{-c} + \varepsilon_{\text{str,spr}}(\sigma, \ell).$$

Proof. Let the target chain be  $X_0, X_1, \dots, X_\ell$ . When the adversary outputs  $M' \neq M$  with  $H(M')=H(M)$ , if it does not internally merge with the target chain, then it can only directly hit the final  $d$ -bit digest, with probability bounded by  $\sigma 2^{-d}$ . If internal merging occurs, then some candidate chain state must hit some  $D_c(W_i)$  of the target

chain on the  $S$  window; the number of targets is at most  $\ell$ , so the multi-target-hit probability of  $\sigma$  candidates is bounded by  $K_{\text{spr}}\sigma\ell 2^{-c}$ . The probability that the target chain itself exhibits capacity repetition, a capacity IV hit, or another meaningless challenge is bounded by  $K_{\text{bad}}\ell^2 2^{-c}$ . The concrete-permutation deviation is counted in  $\varepsilon_{\text{str}}^{\text{spr}}$ . QED.

Ignoring low-order bad events, the second-preimage complexity exponent is

$$S_{\text{spr,cl}} = \min\{d, c - \log_2 \ell\}.$$

From  $\ell \leq \ell_{\text{max}} < 2^{56}$  and  $c=d+64$ ,

$$\begin{aligned} c - \log_2 \ell &\geq c - \log_2 \ell_{\text{max}} > d + \delta, \\ \min\{d, c - \log_2 \ell_{\text{max}}\} &= d. \end{aligned}$$

Therefore the classical second-preimage resistance is  $d$  bits.

### E.13 Parameter Derivation of the Generic Quantum-Attack Complexity

The quantum collision attack is reckoned by the generic collision search of a  $d$ -bit random function; the digest-path complexity is  $2^{(d/3)}$ , and the capacity-window-path complexity is  $2^{(c/3)}$ . Since  $c=d+64$ , the dominant term is the  $d$ -bit digest:

$$S_{\text{coll},q} = \min\{d/3, c/3\} = d/3.$$

The quantum preimage attack is reckoned by Grover search; the complexity of directly hitting the  $d$ -bit digest is  $2^{(d/2)}$ ; any capacity-connection path that needs to hit the full  $S$  window has complexity  $2^{(c/2)}=2^{(d/2+32)}$  and does not dominate. Therefore

$$S_{\text{pre},q} = d/2.$$

The quantum second-preimage attack considers both the final-digest hit and the multi-target hit on the target-chain capacity window:

$$S_{\text{spr},q} = \min\{d/2, (c - \log_2 \ell_{\text{max}})/2\}.$$

From  $(c - \log_2 \ell_{\text{max}})/2 > d/2 + 4$ , we obtain

$$S_{\text{spr},q} = d/2.$$

The above quantum strength is a claimed strength given jointly by the generic quantum-attack complexity baseline and the capacity multi-target parameters. Since  $C_{g,h}$  consists of two sub-permutation stages  $g$  and  $h$ , this specification does not express this derivation as a tight proof of a single-permutation-relation lifting theorem.

### E.14 Generic Quantum Lifting: Correct Point Counting for the Dual Oracle and a Conservative Proof

Quantum lifting theorem. Suppose the winning witness of  $R$  requires verifying  $k$  random-permutation graph points. The quantum lifting theorem<sup>[21]</sup> guarantees: for any adversary  $A$  making  $q$  forward/inverse quantum queries, there exists an adversary  $B$  making at most  $k$  classical queries, such that

$$\text{Adv}_R^q(A) \leq ((8q + 1)^{2k}/(1 - k^2/2^b)) \cdot \text{Adv}_R^{\text{cl}}(B).$$

Dual-oracle direct-product form. If the witness requires  $k_g$   $g$  graph points and  $k_h$   $h$  graph points, respectively, one can condition sequentially on the two independent random permutations and apply the theorem. Taking  $k = k_g + k_h$  and using  $1 - k^2/2^b$  as a unified conservative denominator gives the same upper bound. This treatment

only adds constants and does not change the exponent of  $q$ .

Proof (first conditioning). Fix the random choice of  $h$  and all the side information formed by the adversary interacting with  $h$  and  $h^{-1}$ , treating them as the auxiliary oracle and work registers of the interactive search game. Since  $g$  and  $h$  are independent,  $g$  remains a uniform random permutation under this condition; applying the quantum lifting theorem to  $g$ , at most  $k_g$   $g$  graph points can be extracted, producing a lifting loss corresponding to  $k_g$ .

Proof (second conditioning). Subsequently fix  $g$  and the graph points extracted in the first round, and apply the same theorem to the independent  $h$ , extracting at most  $k_h$   $h$  graph points. The probabilities of the two good events are multiplied, and the number of queries allocated to  $g$  and  $h$  does not exceed the total number of queries  $q$ , so the total lifting exponent is  $2(k_g+k_h)$ . Lower-bounding the two denominators by a smaller unified conservative denominator gives the formulas of this section. This sequential conditioning relies only on the independence of  $g$  and  $h$  in the dual-random-permutation model; no unconditional claim is made about the deterministic correlation of the actual first and second halves.

Preimage. A candidate core witness contains one  $g$  graph point and one  $h$  graph point, so  $k_g = k_h = 1$ ,  $k = 2$ . The hit probability of a single classical witness is at most  $\delta_{\text{pre}} = 2^{-d} + 2\ell_{\text{max}}2^{-c}$ . Hence

$$\text{Adv}_{\text{pre},q}(q) \leq ((8q + 1)^4 / (1 - 4/2^b)) \cdot \delta_{\text{pre}}.$$

When  $c - \log_2 \ell_{\text{max}} > d$ ,  $\delta_{\text{pre}}$  is dominated by  $2^{-d}$ , and constant success requires  $q = \Omega(2^{d/4})$ .

Collision. A collision witness contains two core witnesses, i.e., two  $g$  graph points and two  $h$  graph points, so  $k_g = k_h = 2$ ,  $k = 4$ . The classical hit probability of a single four-point witness is at most  $\delta_{\text{coll}} = 2^{-d} + 2 \cdot 2^{-c}$ , whence

$$\text{Adv}_{\text{coll},q}(q) \leq ((8q + 1)^8 / (1 - 16/2^b)) \cdot \delta_{\text{coll}}.$$

The main term gives  $q = \Omega(2^{d/8})$ .

Second preimage. The conservative verification game reads both the target core and the candidate core, still giving  $k = 4$ ; the multi-target classical hit probability is at most  $\delta_{\text{spr}} = 2^{-d} + 2\ell 2^{-c}$ . Hence

$$\text{Adv}_{\text{spr},q}(q, \ell) \leq ((8q + 1)^8 / (1 - 16/2^b)) \cdot \delta_{\text{spr}}.$$

we obtain  $q = \Omega(2^{\min\{d, c - \log_2 \ell\}/8})$ ; under the AFS-TrEDM parameters it is  $\Omega(2^{d/8})$ .

Theorem E-7 (baseline I). Under the dual-random-permutation QRPM and the above relation-verification baseline, the conservative provable quantum lower bounds of AFS-TrEDM, in the order “preimage, collision, second preimage,” are  $(h/4, h/8, h/8)$ . This theorem shows that generic lifting can be correctly applied, but also reveals the slack caused by the doubling of  $g/h$  graph points; it does not indicate the existence of an attack reaching these complexities.

### E.15 The Compressed-Oracle/Quantum-Indistinguishability Path and Its Applicability Boundary

The compressed-oracle method establishes search and indistinguishability bounds in the quantum random oracle model (QROM) by recording a database of quantum queries<sup>[22]</sup>. For permutations, it must also simultaneously maintain bijectivity, forward/inverse query consistency, and global correlation after reprogramming. The 2025 work proved quantum indistinguishability from a random oracle for the multi-round standard sponge<sup>[23]</sup>, thereby overcoming the long-standing obstacle that the standard sponge lacked a multi-round quantum lazy-sampling tool.

The generic quantum-indistinguishability bound of this result is itself still loose, and the paper also gives tighter

dedicated preimage and collision bounds. Therefore, it solves the quantum composable-security problem of the standard sponge, rather than being a black-box theorem into which parameters can be directly substituted for any feed-forward variant.

If a sufficiently tight  $\text{Adv}_{q\text{-indiff}}(q)$  could be proven for AFS-TrEDM, then from

$$\text{Adv}_H^P(q) \leq \text{Adv}_{\text{RO}}^P(q) + \text{Adv}_{q\text{-indiff}}(q), \quad P \in \{\text{coll}, \text{pre}, \text{spr}\},$$

one could transfer the  $O(q^3/2^d)$  collision bound and the  $O(q^2/2^d)$  preimage bound of the random oracle to this construction, obtaining the tight provable baseline of  $d/3$ ,  $d/2$ , and  $\min\{d/2, (c - \log_2 \ell)/2\}$ .

However, the standard-sponge proof does not cover the mid-section EDM feed-forward, the  $g/h$  dual permutation, the capacity-window  $S$  squeezing, or the second-preimage target-chain multi-target relation. To establish a compressed-permutation-oracle proof for this construction, one must maintain a linked database among the  $g$  output, the post-feed-forward  $h$  input, and the  $S$  chain value, and handle reprogramming consistency under forward/inverse superposition queries. This work is beyond the direct applicability of the existing standard-sponge theorem, so this specification lists it as a subsequent independent research item and not as a completed proof in the current submission.

## E.16 Constant Tables, Layered Summary of Security Strength, and Scope of Applicability

This section uses three tables to summarize the relation-level reduction constants, the original structure-level parameter constants, and the three-layer classical/quantum security strength, respectively. In the tables, “QRPM generic-lifting baseline I” is the conservative provable lower bound, and “quantum claim (generic attack)” is the attack-complexity baseline adopted by the submission document; the two must not be mixed.

The following table uses  $N = \sigma + 2\ell_{\max}$ , and counts repeated queries, IV-capacity hits, and internal  $S$ -window alignment separately, then uniformly upper-bounds them with the union bound. The coefficients are conservative constants and do not affect the security exponent.

Table E-1. Explicit constants and meanings of the AFS-TrEDM classical relation reduction

| Quantity                               | Explicit value                                                                                                        | Meaning                                                                                                                    |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| $b$                                    | 1600                                                                                                                  | Underlying state width; denominator exponent of the permutation-collision correction term                                  |
| $c$                                    | $h + 64$                                                                                                              | Length of the capacity, feed-forward, and chain-value windows                                                              |
| $\ell_{\max}$                          | $\lceil (2^{64} + 128) / \tau \rceil < 2^{56}$                                                                        | Maximum number of frame blocks under the permitted message length                                                          |
| $N$                                    | $\sigma + 2\ell_{\max}$                                                                                               | Upper bound on the total core candidates of the adversary's original queries and the verification of the two output chains |
| Digest-collision coefficient           | $C(N, 2) \cdot 2^{-d}$                                                                                                | $d$ -bit birthday main term                                                                                                |
| Capacity-alignment coefficient         | $2C(N, 2) \cdot 2^{-c}$                                                                                               | Conservative union term for internal $S$ alignment and IV-capacity hits                                                    |
| Preimage-connection coefficient        | $2N\ell_{\max} \cdot 2^{-c}$                                                                                          | Capacity connection for at most $\ell_{\max}$ intermediate positions                                                       |
| Second-preimage-connection coefficient | $2N\ell \cdot 2^{-c}$                                                                                                 | $c$ -bit multi-target hit of the candidate chain on the target chain                                                       |
| Target-chain bad event                 | $2C(\ell, 2) \cdot 2^{-c}$                                                                                            | Internal $S$ repetition and IV hits of the target chain                                                                    |
| $\varepsilon_{\text{str}}$             | $\varepsilon_{\text{str}, \text{coll}} + \varepsilon_{\text{str}, \text{pre}} + \varepsilon_{\text{str}, \text{spr}}$ | Residual risk of the actual AFS- $p$ -S6 deviating from the dual-random-permutation model                                  |

Table E-2. Structure-level security-bound constants and the three-tier strengths

| Quantity                                                          | Value                                   | Meaning                                                                                      |
|-------------------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------|
| $d$                                                               | 512/768/1024                            | Digest length                                                                                |
| $c$                                                               | $d+64$                                  | Length of the capacity window, the feed-forward window, and the chain-value window           |
| $r$                                                               | $1600-c$                                | Absorbing rate                                                                               |
| $\ell_{\max}$                                                     | $\lceil (2^{64}+128)/r \rceil < 2^{56}$ | Maximum number of frame blocks under the permitted message length                            |
| $K_{\text{coll}}, K_{\text{pre}}, K_{\text{spr}}, K_{\text{bad}}$ | Small constants                         | Chain-localization and bad-event counting constants in the union bound                       |
| $\varepsilon_{\text{str}}$                                        | Constrained by §3.3 and Appendix D      | Residual risk of the concrete AFS- $p$ -S6 deviating from the structure-level ideal baseline |
| Classical collision resistance                                    | $d/2$                                   | 512/768/1024 instances are 256/384/512 bits                                                  |
| Classical preimage resistance                                     | $d$                                     | 512/768/1024 instances are 512/768/1024 bits                                                 |
| Classical second-preimage resistance                              | $\min\{d, c - \log_2 \ell_{\max}\} = d$ | The extra 64 bits of capacity absorb the $\log_2 \ell_{\max}$ loss                           |
| Quantum collision resistance                                      | $d/3$                                   | 512/768/1024 instances are about 170.7/256/341.3 bits                                        |
| Quantum preimage resistance                                       | $d/2$                                   | 512/768/1024 instances are 256/384/512 bits                                                  |
| Quantum second-preimage resistance                                | $d/2$                                   | The capacity multi-target term is at least 4 bits higher than $d/2$                          |

Table E-3. Layered summary of the classical bounds, the conservative quantum proof, and the quantum claimed strength

| Analysis level                  | Security property / baseline     | 512   | 768 | 1024  |
|---------------------------------|----------------------------------|-------|-----|-------|
| Classical dual-RPM reduction    | Collision resistance $d/2$       | 256   | 384 | 512   |
| Classical dual-RPM reduction    | Preimage resistance $d$          | 512   | 768 | 1024  |
| Classical dual-RPM reduction    | Second-preimage resistance $d$   | 512   | 768 | 1024  |
| QRPM generic-lifting baseline I | Collision resistance $d/8$       | 64    | 96  | 128   |
| QRPM generic-lifting baseline I | Preimage resistance $d/4$        | 128   | 192 | 256   |
| QRPM generic-lifting baseline I | Second-preimage resistance $d/8$ | 64    | 96  | 128   |
| Quantum claim (generic attack)  | Collision resistance $d/3$       | 170.7 | 256 | 341.3 |
| Quantum claim (generic attack)  | Preimage resistance $d/2$        | 256   | 384 | 512   |
| Quantum claim (generic attack)  | Second-preimage resistance $d/2$ | 256   | 384 | 512   |

Scope of applicability. The conclusions of this appendix serve to illustrate the logical relationship between the AFS-TrEDM parameter selection and the three classes of security-strength claims: the classical bound is a structure-level idealized derivation plus the  $\varepsilon_{\text{str}}$  deviation term, and the quantum bound is the generic quantum-attack complexity baseline. Any low-complexity structural attack targeting the concrete round count, the concrete S-box, or the concrete linear layer should be counted in  $\varepsilon_{\text{str}}$  and screened by the evidence chain of §3.3 and Appendix D.

Scope of applicability. The three classical items are relation-level conclusions under the dual-random-permutation model plus the  $\varepsilon_{\text{str}}$  deviation term; baseline I is the conservative provable lower bound obtained by generic lifting in the QRPM; the quantum claimed strength is the submission baseline jointly formed by the known generic quantum-attack complexity and structural screening. The three answer different questions and cannot substitute for one another. Any low-complexity attack targeting the round count, the S-box, the linear layer, or the  $g/h$  split relation of the actual AFS- $p$ -S6 should be counted in  $\varepsilon_{\text{str}}$  and continuously evaluated by the evidence chain of §3.3 and Appendix D.